

DEPARTEMENT TECHNOLOGIES DE L'INFORMATIQUE

N°/.....

MEMOIRE DE STAGE DE FIN D'ETUDES

POUR L'OBTENTION DU DIPLOME DE MASTERE PROFESSIONNEL
EN TECHNOLOGIES DE L'INFORMATIQUE :

GÉNIE LOGICIEL ET DÉVELOPPEMENT RAPIDE D'APPLICATIONS

-ENSEIGNEMENT A DISTANCE-

Intitulé :

Conception et Développement d'une Application
de Gestion du Parc Informatique de
la Faculté de Médecine de Monastir

Encadreur : Pr. AZIZI Ridha

Elaboré par :

BOUSSAID Hela

Année Universitaire 2025– 2026

Intitulé du stage : Stage fin d'étude du master en GLDRA

Réalisé par les étudiants : BOUSSAID Hela

Organisme d'accueil : Faculté de médecine de Monastir

Mots-Clés : Gestion du parc informatique, gestion des équipements, système de ticketing, maintenance informatique, affectation des ressources, API REST, Node.js, React, MySQL, Firebase FCM, JWT, RBAC, UML, méthodologie Scrum.

Résumé :

Ce mémoire présente la conception et le développement d'une application web de gestion du parc informatique de la Faculté de Médecine de Monastir, réalisée au sein du SmartLab. Ce projet vise à améliorer la gestion des équipements informatiques, le suivi des incidents et l'organisation des opérations de maintenance, dans un contexte marqué par l'hétérogénéité du parc et l'absence d'outils centralisés.

La solution proposée repose sur une architecture web moderne intégrant un backend développé avec Node.js et Express, un frontend en React, ainsi qu'une base de données MySQL. Elle offre des fonctionnalités principales telles que la gestion des actifs, l'affectation des équipements, la gestion des tickets d'incident et la génération de rapports.

Le système intègre également des mécanismes de sécurité basés sur l'authentification JWT et le contrôle d'accès par rôles (RBAC), ainsi qu'un module de notification en temps réel via Firebase Cloud Messaging. Le développement a été réalisé selon la méthodologie Scrum, permettant une organisation itérative et progressive du projet.

Nom de l'Enseignant Encadrant : Pr. AZIZI Ridha	Signature : 	Nom de l'Encadrant Industriel : ZAIBI Racha	Tampon & Sign. de l'Encadrant Ind. :
---	----------------------------	---	---

Remerciements

« La reconnaissance est non seulement la plus grande des vertus, mais aussi la mère de toutes les autres. » --- Cicéron

C'est avec un grand honneur et un profond plaisir que je consacre ces quelques lignes à remercier toutes les personnes ayant contribué, de près ou de loin, à l'accomplissement de ce projet.

À Monsieur Ridha A.F.F.I, pour son encadrement, son aide, sa patience, ses orientations et ses recommandations, qui ont été d'un grand intérêt et m'ont permis de mener ce projet à terme.

À Monsieur Charfeddine A.M.P.I, Doyen de la Faculté de Médecine, pour son aide, son soutien, ses précieux conseils et son accueil chaleureux.

À mon encadrante professionnelle, Rachida F.F.B.I, pour son assistance, son soutien, ses conseils pertinents et son encadrement direct et bienveillant durant ce stage.

Je suis également très reconnaissante envers toutes les personnes qui ont contribué à l'élaboration de ce travail, notamment Monsieur Ridha A.F.F.I, Monsieur Adnène R.O.U.A.T.B.I, Monsieur Hatem S.A.N.D.I.D, Monsieur Wissem L.T.A.I.E.F et Monsieur Hichem M.S.E.K, pour leur soutien moral, leurs encouragements, leurs conseils et le temps qu'ils ont généreusement consacré.

Dédicaces

Je dédie ce travail :

À ma mère, pour son amour, ses sacrifices, son soutien et son dévouement. Aucun mot ne saurait exprimer pleinement ma joie, ma gratitude et ma reconnaissance pour tout ce qu'elle fait pour moi. Que Dieu lui accorde santé, bonheur et longue vie.

À tous les membres de ma famille, à mon mari et à mes enfants, pour leur tendresse, leur considération et leur soutien permanent. Je leur dois énormément.

À mes chères amies, pour tous les merveilleux moments et les souvenirs partagés.

À toute l'équipe de la Faculté de Médecine de Monastir, qui m'a encouragée et soutenue durant ce stage. Je suis reconnaissante envers toutes les personnes que j'aime et envers toutes celles qui m'aiment.

*Hela Boussaid
Année universitaire 2025-2026*

Table des matières

Introduction générale	1
Chapitre 1 : Contexte et étude de l'existant	4
Introduction	5
1.1 Présentation de l'organisme d'accueil	5
1.2 Contexte et problématique	5
1.2.1 Variabilité des équipements informatiques	6
1.2.2 Contraintes organisationnelles et techniques	6
1.3 Analyse des solutions existantes	6
1.3.1 Présentation des solutions existantes	7
1.3.2 Comparaison et limites	8
1.4 Solution proposée : Parc Informatique FMM	10
1.4.1 Architecture et fonctionnement	10
1.4.2 Apports de la solution	10
Conclusion	11
Chapitre 2 : Méthodologie et spécifications des besoins	12
Introduction	13
2.1 Analyse et spécification des besoins	13
2.1.1 Identification des acteurs	13
2.1.2 Spécification des besoins	13
2.2 Choix de la méthodologie	15
2.2.1 Étude comparative des méthodologies	15
2.2.2 Méthodologie retenue : Scrum	16
2.3 Présentation de Scrum	17
2.3.1 Artefacts Scrum et leur application	18
2.3.2 Événements Scrum et leur implémentation	18

2.4	Les rôles Scrum	19
2.5	Outils de support à Scrum	19
	Conclusion	20
Chapitre 3 : Étude architecturale du projet		21
	Introduction	22
3.1	Présentation de l'architecture générale	22
3.2	Modèle de conception logicielle adopté	23
3.2.1	Hiérarchie organisationnelle et modèle de localisation	23
3.3	Interactions et fonctionnement du système	24
3.3.1	Diagramme des cas d'utilisation	24
3.3.2	Diagramme de classes	24
3.3.3	Diagrammes de séquence	26
3.3.4	Diagramme d'activités	29
3.4	Pourquoi ce choix d'architecture	32
	Conclusion	32
Chapitre 4 : Planification et préparation du projet Parc Informatique FMM		33
	Introduction	34
4.1	Pilotage du projet avec Scrum	34
4.1.1	Acteurs Scrum et missions	34
4.1.2	Organisation du Backlog	34
4.1.3	Planification des sprints	35
4.2	Environnement de travail	37
4.2.1	Environnement matériel	37
4.2.2	Environnement de développement	38
4.2.3	Environnement logiciel	39
	Conclusion	41
Chapitre 5 : Mise en place de l'architecture et gestion des équipements		42
	Introduction	43

5.1	Objectifs et livrables	44
5.2	Architecture et modélisation UML du Sprint 1	45
5.2.1	Clarification terminologique : Affectation	46
5.2.2	Diagramme de classes Sprint 1	46
5.2.3	Diagrammes de séquence Sprint 1	48
5.3	Tâches principales	51
5.3.1	Configuration du serveur backend	51
5.3.2	Création des schémas de base de données	52
5.3.3	Développement de l'API REST et Architecture des Endpoints	54
5.3.4	Mécanismes transverses : sécurité, validation et cycle de vie	55
5.4	Critères d'acceptation	56
5.5	Implémentation et résultats	58
5.6	Tests et validation	59
5.7	Défis et solutions	60
5.8	Planification pour le sprint suivant	60
	Conclusion	61
Chapitre 6 : Gestion des tickets et maintenance		62
	Introduction	63
6.1	Objectifs du Sprint 2	64
6.2	Architecture et modélisation UML du Sprint 2	64
6.2.1	Diagramme de classes Sprint 2	64
6.2.2	Diagrammes de séquence Sprint 2	65
6.3	Livrables attendus	70
6.4	Tâches principales	70
6.4.1	Qualité et tests	70
6.4.2	Notifications et communication	70
6.4.3	Automatisation des workflows	73
6.4.4	Optimisations et performance	73
6.4.5	Déploiement et résilience	74

6.5	Critères d'acceptation	74
6.6	Implémentation et résultats	74
6.7	Tests et validation	75
6.8	Défis et solutions	76
6.9	Planification pour le sprint suivant	76
	Conclusion	77
Chapitre 7 : Notifications, tableaux de bord et reporting		78
	Introduction	79
7.1	Objectifs du Sprint 3	79
7.2	Architecture et modélisation UML du Sprint 3	80
7.2.1	Diagramme de classes du Sprint 3	81
7.2.2	Diagrammes de séquence du Sprint 3	82
7.3	Tableaux de bord et interfaces utilisateur	87
7.3.1	Tableau de bord administrateur	87
7.3.2	Tableau de bord utilisateur	88
7.3.3	Tableau de bord technicien	89
7.4	Modules de gestion du parc informatique	90
7.4.1	Gestion de l'inventaire et des équipements	90
7.4.2	Contrôle d'accès par rôles (RBAC)	92
7.4.3	Analyse et reporting	93
7.4.4	Architecture physique et traçabilité des actifs	95
7.5	Optimisation des performances	98
	Conclusion	100
Conclusion et perspectives		101
Bibliographie		104

Table des figures

1.1	Logo de la FMM	5
1.2	Logo du SmartLab	5
1.3	Logo de GLPI	7
1.4	Logo de iTop	8
1.5	Logo de Asset Panda	8
2.6	Processus Scrum.	17
2.7	Rôles Scrum.	19
2.8	Logo de ClickUp.	20
2.9	Logo de Slack.	20
2.10	Logo de Git.	20
3.11	Architecture globale du Parc Informatique FMM	22
3.12	Modèle MVC du Parc Informatique FMM	23
3.13	Diagramme des cas d'utilisation global	24
3.14	Diagramme de classes du Parc Informatique FMM	25
3.15	Diagramme de séquence d'authentification	26
3.16	Diagramme de séquence de création et assignation d'un ticket	27
3.17	Diagramme de séquence de traitement et mise à jour d'un ticket	28
3.18	Diagramme de séquence de clôture et consultation de l'historique d'un ticket	29
3.19	Diagramme d'activité du flux des processus	31
4.20	Architecture technologique	39
5.21	Architecture technique multicouche du Sprint 1 — Flux et composants du système	43
5.22	Diagramme de classes du Sprint 1 — Gestion des équipements et des utilisateurs	47
5.23	Diagramme de séquence d'authentification utilisateur	48
5.24	Diagramme de séquence de création d'un équipement	49
5.25	Diagramme de séquence d'affectation d'un équipement	50
5.26	Diagramme de séquence de consultation de l'inventaire	50

5.27	Diagramme de séquence de consultation des détails d'un équipement	51
5.28	Diagramme entité-relation de la base MySQL	53
5.29	Interface Swagger UI des API du Sprint 1	59
6.30	Architecture améliorée du Sprint 2 — tests, notifications et optimisations	63
6.31	Diagramme de classes du Sprint 2	64
6.32	Diagramme de séquence de création d'un ticket	65
6.33	Diagramme de séquence d'affectation automatique d'un ticket	66
6.34	Diagramme de séquence de consultation des tickets affectés	67
6.35	Diagramme de séquence de traitement d'un ticket	67
6.36	Diagramme de séquence de création d'une intervention	68
6.37	Diagramme de séquence de clôture d'un ticket	69
6.38	Diagramme de séquence de consultation de l'historique d'un ticket	69
6.39	Architecture des notifications Firebase FCM et événements métier	71
6.40	Diagramme de séquence de gestion d'un ticket d'incident	71
7.41	Diagramme de classes du Sprint 3 — entités de notification et de reporting	81
7.42	Diagramme de séquence — notification push lors de la création d'un ticket	83
7.43	Diagramme de séquence — notification push lors d'un changement de statut de ticket	84
7.44	Diagramme de séquence — consultation du tableau de bord analytique	85
7.45	Diagramme de séquence — génération d'un rapport d'activité PDF/Excel	86
7.46	Diagramme de séquence — export des statistiques du parc informatique	87
7.47	Tableau de bord administrateur — vue globale des KPI et de l'état opérationnel du parc	88
7.48	Tableau de bord utilisateur — suivi des équipements affectés et des demandes d'intervention	89
7.49	Tableau de bord technicien — file d'attente des interventions classées par priorité	90
7.50	Interface de gestion de l'inventaire — liste filtrée des équipements du parc	91
7.51	Formulaire d'enregistrement d'un nouvel équipement avec génération automatique du QR code	92
7.52	Interface de gestion des permissions et rôles — modèle RBAC du système	93
7.53	Module analytique — tendances de pannes et indicateurs de performance du parc	94

7.54	Module de reporting — génération de rapports périodiques PDF/Excel et consultation des archives	95
7.55	Vue d'architecture physique — hiérarchie Bloc/Département/Salle/Équipement de la FMM	96
7.56	Interface d'affichage et d'impression du QR code d'un équipement	97
7.57	Interface de scan mobile du QR code — accès instantané à la fiche d'un équipement sur le terrain	98

Liste des tableaux

1.2	Exemples d'équipements informatiques	6
1.3	Comparaison des solutions de gestion du parc informatique	9
1.4	Fonctionnalités principales du système	10
2.5	Acteurs du système et leurs rôles	13
2.6	Tableau comparatif des méthodologies de développement logiciel.	15
4.7	Acteurs du projet Scrum	34
4.8	Backlog du Projet avec Priorités MoSCoW	35
4.9	Planification des Sprints du Projet Parc Informatique FMM	36
5.10	Modèles de données du système Parc Informatique FMM	45
5.11	Spécifications techniques et politiques de sécurité des routes de l'API	55
5.12	Résultats de validation des critères d'acceptation du Sprint 1	60
6.13	Validation fonctionnelle des critères d'acceptation du Sprint 2	75
7.14	Sprint Backlog — Sprint 3 : Notifications, tableaux de bord et reporting	80
8.15	Synthèse des objectifs atteints et perspectives d'évolution	102

Liste des abréviations

Sigle / Abréviation	Signification
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
FCM	Firebase Cloud Messaging
FMM	Faculté de Médecine de Monastir
GMAO	Gestion de Maintenance Assistée par Ordinateur
HL7	Health Level Seven
JWT	JSON Web Token
KPI	Key Performance Indicator
MUI	Material-UI
MTBF	Mean Time Between Failures
MTTR	Mean Time To Repair
ORM	Object-Relational Mapping
PFE	Projet de Fin d'Études
RBAC	Role-Based Access Control
REST	Representational State Transfer
SLA	Service Level Agreement
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language

Introduction

Générale

La transformation numérique constitue aujourd'hui un levier essentiel de modernisation des institutions académiques et administratives. Dans un contexte marqué par une circulation de l'information de plus en plus rapide, la numérisation des activités de gestion s'impose comme une nécessité pour optimiser l'utilisation des ressources, renforcer la performance organisationnelle et améliorer la qualité des services rendus. Les outils informatiques ne se limitent plus à un rôle de support ; ils occupent désormais une place centrale dans les processus de décision, de suivi et de pilotage. Ils facilitent la collecte, le traitement et l'analyse des données, tout en permettant une planification plus précise, une allocation optimale des ressources et une gestion durable des systèmes.

La Faculté de Médecine de Monastir (FMM), en tant qu'institution académique de référence, s'inscrit pleinement dans cette dynamique de modernisation. Elle doit assurer la continuité de ses missions pédagogiques et scientifiques tout en garantissant un environnement de travail moderne, fiable et sécurisé. Dans ce contexte, la gestion des interventions techniques et le suivi du parc informatique constituent des enjeux stratégiques majeurs. Les infrastructures informatiques, les équipements de laboratoire et les dispositifs pédagogiques représentent des ressources essentielles dont la disponibilité et la fiabilité conditionnent directement la qualité de l'enseignement et de la recherche. Une gestion plus structurée de ces ressources apparaît ainsi indispensable pour améliorer le fonctionnement global de l'institution et soutenir sa performance.

Le présent projet de fin d'études s'inscrit dans cette perspective. Réalisé à une échelle locale, il consiste à concevoir et développer un prototype fonctionnel d'application dédié à la gestion et au suivi du parc informatique de la Faculté de Médecine de Monastir. L'objectif principal est de mettre en place une solution centralisée et automatisée permettant de suivre l'état des équipements, de gérer les interventions techniques et de prolonger la durée de vie du matériel grâce à une maintenance proactive adaptée. Cette application vise également à améliorer la réactivité des équipes techniques, à faciliter la priorisation des demandes et à offrir aux utilisateurs un moyen simple, fiable et transparent de signaler les problèmes rencontrés.

Apports du projet et alignement normatif

Au-delà de sa dimension opérationnelle, ce projet s'inspire des bonnes pratiques en matière de qualité et de sécurité, sans prétendre à une conformité industrielle complète. À l'échelle de l'établissement, la faculté s'inscrit dans un processus de transition vers la certification ISO 21001, dédiée au management des organismes d'éducation, et engage une démarche orientée vers la norme ISO 27001 relative à la sécurité de l'information. Dans le cadre de ce prototype fonctionnel, cette orientation se traduit par l'intégration de mécanismes de sécurité de base, notamment l'authentification par jeton (JWT) et le contrôle d'accès basé sur les rôles (RBAC). Ces choix témoignent d'une volonté de poser les bases d'une gouvernance numérique structurée et de contribuer à la protection des données sensibles, qu'elles soient académiques, administratives ou liées au suivi du parc informatique. Les principes de la norme ISO 27001 constituent ainsi un cadre méthodologique de référence pour orienter la conception d'une solution adaptée au contexte de l'établissement.

Structure du rapport

Ce rapport de PFE Master est structuré de manière progressive et cohérente afin de présenter l'ensemble des étapes du projet, depuis l'analyse du contexte jusqu'à l'évaluation de la solution proposée. Il s'articule autour de plusieurs chapitres complémentaires.

Le premier chapitre présente le cadre général du projet, l'organisme d'accueil, ainsi que l'étude de l'existant et les limites des solutions actuelles dans le domaine de la gestion du parc informatique.

Le deuxième chapitre est consacré à la méthodologie adoptée et aux spécifications des besoins. Il décrit les besoins fonctionnels et non fonctionnels du système, ainsi que le choix de la méthode Agile Scrum pour la conduite du projet.

Le troisième chapitre expose l'étude architecturale du projet, en détaillant l'architecture globale de la solution, le modèle de conception adopté ainsi que les principaux diagrammes UML illustrant le fonctionnement du système.

Le quatrième chapitre traite de la planification et de la préparation du projet à travers le Sprint 0. Il présente l'organisation de l'équipe, la gestion du backlog et la mise en place de l'environnement de travail.

Les chapitres cinq, six et sept décrivent les différentes phases d'implémentation du système selon une approche itérative. Ils présentent respectivement la mise en place de l'architecture et la gestion des équipements (Sprint 1), la gestion des tickets et de la maintenance (Sprint 2), ainsi que l'intégration des notifications, des tableaux de bord et des fonctions de reporting (Sprint 3).

Enfin, la conclusion générale synthétise les résultats obtenus, met en évidence les apports du projet, identifie ses limites et propose des perspectives d'évolution.

En ce sens, ce projet ne se limite pas à une simple réalisation technique, mais s'inscrit dans

une vision globale de transformation numérique intégrant des dimensions organisationnelles, technologiques et méthodologiques. Il met en évidence l'importance d'une gestion rigoureuse du parc informatique et souligne les bénéfices d'une solution numérique bien conçue pour le bon fonctionnement de l'institution. À travers cette initiative, la Faculté de Médecine de Monastir poursuit la modernisation de ses pratiques et l'adoption progressive de bonnes pratiques en matière de qualité et de sécurité.

À la suite de cette introduction générale, le chapitre suivant sera consacré à l'analyse du contexte et à l'étude des solutions existantes, afin d'identifier les limites des systèmes actuels et de justifier les choix de conception retenus dans ce projet.

CHAPITRE 1

Contexte et étude de l'existant

Table des matières

Introduction	5
1.1 Présentation de l'organisme d'accueil	5
1.2 Contexte et problématique	5
1.2.1 Variabilité des équipements informatiques	6
1.2.2 Contraintes organisationnelles et techniques	6
1.3 Analyse des solutions existantes	6
1.3.1 Présentation des solutions existantes	7
1.3.2 Comparaison et limites	8
1.4 Solution proposée : Parc Informatique FMM	10
1.4.1 Architecture et fonctionnement	10
1.4.2 Apports de la solution	10
Conclusion	11

Introduction

Le présent chapitre a pour objectif de présenter le cadre général du projet *Parc Informatique FMM*, réalisé au sein du SmartLab de la Faculté de Médecine de Monastir (FMM)[1]. Il introduit le contexte organisationnel du projet, met en évidence les problématiques liées à la gestion des parcs informatiques dans les établissements tunisiens et analyse les solutions existantes sur le marché.

L'ensemble de ce chapitre est structuré comme suit : une présentation de l'organisme d'accueil, une analyse du contexte et des problématiques, une étude des solutions existantes, ainsi qu'une description de la solution proposée.

1.1. Présentation de l'organisme d'accueil

Le projet a été réalisé au sein du SmartLab, une Unité de Service Commun et de Recherche (USCR) rattachée à la Faculté de Médecine de Monastir (FMM), institution universitaire tunisienne fondée en 1980 et reconnue pour la qualité de sa formation et de ses activités de recherche.

La FMM constitue un établissement de référence dans le domaine des sciences médicales en Tunisie. Elle s'inscrit dans une démarche de qualité, attestée notamment par sa certification ISO 21001[2], et vise à assurer une formation académique conforme aux standards internationaux tout en favorisant l'innovation scientifique.

Dans ce cadre, le SmartLab joue un rôle central en tant qu'environnement de recherche et de développement technologique. Il a pour mission de favoriser la collaboration entre chercheurs, enseignants et ingénieurs afin de concevoir des solutions innovantes adaptées aux problématiques réelles du secteur de la santé et des systèmes d'information.

Le SmartLab dispose d'infrastructures techniques modernes, comprenant des équipements informatiques, des serveurs, ainsi que des outils de développement et de test. Il constitue ainsi un environnement propice à la conception et à la validation de solutions numériques dans un contexte académique et professionnel.



FIGURE 1.1 – Logo de la FMM



FIGURE 1.2 – Logo du SmartLab

1.2. Contexte et problématique

Dans de nombreux établissements publics et privés en Tunisie, la gestion des parcs informatiques repose encore sur des approches traditionnelles, souvent semi-manuelles. Ces approches incluent l'utilisation de fichiers Excel, de registres papier ou de systèmes non centralisés pour le

suivi des équipements et la gestion des incidents.

Ce mode de fonctionnement engendre plusieurs limitations, notamment en termes de traçabilité, de réactivité et de fiabilité de l'information. Il dépend fortement de l'intervention humaine, ce qui augmente le risque d'erreurs et ralentit les processus de maintenance et de suivi.

1.2.1. Variabilité des équipements informatiques

Les parcs informatiques sont constitués d'un ensemble hétérogène d'équipements, incluant notamment les postes de travail, les serveurs, ainsi que les licences logicielles. Ces ressources diffèrent selon leur nature, leur durée de vie et leur mode d'acquisition.

À titre d'exemple, un ordinateur portable présente généralement une durée de vie comprise entre trois et cinq ans, tandis que les licences logicielles sont soumises à des contraintes de renouvellement périodique. Cette diversité rend la gestion des actifs particulièrement complexe en l'absence d'un système centralisé.

TABLE 1.2 – Exemples d'équipements informatiques

Catégorie	Exemple	Durée de vie	Mode d'acquisition
Ordinateur portable	Dell Latitude 7420	3 à 5 ans	Achat
Logiciel	Microsoft Office	Selon licence	Abonnement / licence
Serveur	Serveur dédié	5 à 7 ans	Achat / location

1.2.2. Contraintes organisationnelles et techniques

L'analyse du contexte tunisien met en évidence plusieurs contraintes majeures impactant la gestion des parcs informatiques :

- **Sous-effectif technique** : les équipes informatiques sont souvent réduites par rapport au volume d'équipements à gérer, ce qui limite la réactivité en cas d'incident.
- **Absence d'automatisation** : les opérations de suivi et de gestion sont majoritairement manuelles, ce qui augmente la charge de travail et le risque d'erreurs.
- **Hétérogénéité du parc informatique** : la diversité des équipements et des systèmes utilisés complique l'intégration et la standardisation des processus de gestion.

Ces contraintes mettent en évidence la nécessité de mettre en place une solution centralisée, automatisée et adaptée aux ressources disponibles, capable d'améliorer la gestion opérationnelle des équipements informatiques.

1.3. Analyse des solutions existantes

Cette section présente une étude des principales solutions existantes dans le domaine de la gestion des parcs informatiques. Elle vise à analyser leurs fonctionnalités ainsi que leurs limites, afin de positionner la solution proposée par rapport à l'existant.

1.3.1. Présentation des solutions existantes

Plusieurs solutions sont actuellement utilisées pour la gestion des actifs et des services informatiques dans les organisations. Ces solutions permettent principalement l'inventaire des équipements, la gestion des incidents, ainsi que le suivi des opérations de maintenance.

Parmi les solutions les plus répandues, on retrouve **GLPI**, **iTop** et **Asset Panda**, qui se distinguent par leurs architectures et leurs niveaux de complexité.

- **GLPI** est une solution open-source de gestion des services informatiques (ITSM - IT Service Management) largement adoptée dans les environnements académiques, industriels et administratifs. Elle permet la centralisation et la structuration de la gestion du parc informatique à travers plusieurs modules fonctionnels.

Parmi ses principales fonctionnalités, on distingue la gestion de l'inventaire matériel et logiciel, le suivi des tickets d'incidents, la gestion des demandes utilisateurs, ainsi que la planification des interventions de maintenance. GLPI permet également la gestion des affectations des équipements aux utilisateurs et la traçabilité des changements effectués sur le parc informatique.

L'un de ses principaux atouts réside dans son caractère open-source, qui offre une grande flexibilité d'adaptation, ainsi que l'existence d'une communauté active assurant des mises à jour régulières et de nombreux plugins d'extension. Cependant, certaines fonctionnalités adaptées à des besoins spécifiques, notamment la notification en temps réel et l'automatisation poussée des workflows, nécessitent des configurations supplémentaires ou l'ajout de modules externes.[3]



FIGURE 1.3 – Logo de GLPI

- **iTop** est une solution ITSM open-source orientée vers la gestion des services informatiques et la modélisation des infrastructures via une CMDB (Configuration Management Database). Elle se distingue par une approche structurée permettant de représenter les relations entre les différents éléments du système d'information, tels que les équipements, les applications et les services.

iTop intègre des fonctionnalités adaptées de gestion des incidents, des problèmes et des changements, conformément aux bonnes pratiques ITIL. Elle permet également la génération de rapports détaillés et la mise en place de processus de validation adaptés aux environnements

organisationnels complexes.

Toutefois, la richesse fonctionnelle d'iTop s'accompagne d'une complexité de configuration relativement élevée, nécessitant un temps d'adaptation important ainsi que des compétences techniques spécialisées pour son déploiement et sa personnalisation.[4]



FIGURE 1.4 – Logo de iTop

- **Asset Panda** est une solution SaaS (Software as a Service) dédiée à la gestion des actifs informatiques et physiques. Elle repose sur une architecture entièrement cloud, permettant un accès centralisé et protégé aux données relatives aux équipements.

Cette solution offre des fonctionnalités de suivi en temps réel des actifs, de gestion du cycle de vie des équipements, ainsi que des outils mobiles facilitant les opérations de scan, d'inventaire et de mise à jour sur le terrain. Elle permet également l'intégration avec d'autres systèmes d'information via des API, ce qui renforce son interopérabilité.

Asset Panda est particulièrement adaptée aux organisations recherchant une solution clé en main, sans infrastructure locale complexe. Néanmoins, sa dépendance à une connexion Internet stable ainsi que son modèle basé sur l'abonnement peuvent constituer des limites dans certains contextes, notamment dans les établissements disposant de budgets restreints ou d'infrastructures réseau limitées.[5]



FIGURE 1.5 – Logo de Asset Panda

1.3.2. Comparaison et limites

Afin d'évaluer de manière objective les solutions étudiées, une comparaison multicritère est réalisée dans le Tableau 1.3. Cette comparaison repose sur des critères techniques et organisationnels directement liés aux besoins de gestion d'un parc informatique, notamment l'automatisation des processus, la compatibilité avec différents environnements, la gestion des notifications en temps réel, la facilité de déploiement ainsi que l'efficacité globale du système.

TABLE 1.3 – Comparaison des solutions de gestion du parc informatique

Critère	GLPI	iTop	Asset Panda
Automatisation	±	✓	✓
Compatibilité	✓	±	±
Notifications en temps réel	∅	±	✓
Déploiement	✓	±	∅
Efficacité globale	±	✓	✓

Légende : ✓ : optimal ; ± : moyen ; ∅ : non adapté.

Interprétation du tableau : L'analyse du Tableau 1.3 permet de mettre en évidence des différences significatives entre les trois solutions étudiées selon les critères retenus.

En termes d'**automatisation**, les solutions iTop et Asset Panda présentent un niveau adapté grâce à l'intégration de workflows structurés et d'outils de gestion automatisée des processus. À l'inverse, GLPI offre des fonctionnalités d'automatisation plus limitées, nécessitant souvent l'ajout de plugins ou de configurations supplémentaires pour atteindre un niveau équivalent.

Concernant la **compatibilité avec les équipements et environnements hétérogènes**, GLPI se distingue par une meilleure adaptabilité, notamment grâce à sa large communauté et à ses nombreux modules d'extension. iTop et Asset Panda présentent un niveau intermédiaire, bien qu'ils restent compatibles avec la majorité des infrastructures standards.

Pour ce qui est des **notifications en temps réel**, Asset Panda se démarque clairement grâce à son architecture cloud native, permettant une synchronisation et des alertes instantanées. iTop propose un support partiel de ce type de fonctionnalités, tandis que GLPI reste limité dans ce domaine sans intégration externe.

En ce qui concerne la **facilité de déploiement**, GLPI apparaît comme la solution la plus accessible, notamment en raison de sa nature open-source et de sa large documentation. iTop nécessite une phase de configuration plus complexe, tandis qu'Asset Panda, bien que simple à utiliser, impose une dépendance à une infrastructure cloud et à un modèle d'abonnement.

Enfin, l'**efficacité globale** met en évidence un équilibre entre richesse fonctionnelle et complexité d'utilisation. iTop et Asset Panda offrent des performances globalement élevées, mais avec des contraintes différentes, tandis que GLPI présente une solution plus simple mais moins adaptée sur certains aspects critiques.

Synthèse critique L'analyse globale des résultats montre que, malgré la maturité technologique de ces solutions, chacune présente des limites structurelles qui restreignent leur adéquation avec certains contextes organisationnels.

GLPI, bien qu'efficace et largement adopté, montre des limites en matière d'automatisation

adaptée et de gestion des notifications modernes. iTop, quant à lui, offre une grande richesse fonctionnelle mais au prix d'une complexité de mise en œuvre importante. Asset Panda propose une solution moderne et efficace, mais son modèle SaaS ainsi que sa dépendance à une connexion Internet stable peuvent constituer des contraintes dans des environnements à ressources limitées.

Conclusion Ainsi, aucune des solutions étudiées ne répond de manière pleinement fonctionnelle et optimale à l'ensemble des besoins identifiés, notamment dans le contexte des établissements tunisiens. Les contraintes liées au coût, à la complexité de déploiement ainsi qu'à la nécessité d'une meilleure adaptabilité locale justifient le développement, dans le cadre de ce projet de fin d'études, d'une solution plus flexible, légère et adaptée aux réalités du terrain.

1.4. Solution proposée : Parc Informatique FMM

Le projet *Parc Informatique FMM* vise à proposer une solution centralisée de gestion des équipements informatiques, adaptée aux contraintes des établissements tunisiens. Cette solution repose sur une architecture web moderne et modulaire, permettant l'automatisation de certains processus de gestion des actifs et des incidents à une échelle locale.

Le système s'appuie sur des technologies open-source telles que **Node.js (Express)**[6], **React**[7] et **MySQL**[8], garantissant ainsi flexibilité, évolutivité et maîtrise des coûts.

1.4.1. Architecture et fonctionnement

L'application est structurée selon une architecture client-serveur. Le backend expose une API REST assurant la gestion des utilisateurs, des équipements et des tickets, ainsi que l'intégration avec un système de notifications en temps réel via Firebase[9].

Le frontend, développé avec React et Material UI[10], permet une interaction fluide avec le système à travers des interfaces dédiées à la gestion des équipements, des incidents et des statistiques.

TABLE 1.4 – Fonctionnalités principales du système

Module	Description
Gestion des actifs	Inventaire, affectation, suivi et génération de QR codes.
Gestion des incidents	Création, affectation et suivi des tickets.
Reporting	Génération de statistiques et export des données.

1.4.2. Apports de la solution

La solution proposée permet d'améliorer significativement la traçabilité des équipements, de réduire les délais de traitement des incidents et d'optimiser la gestion des ressources informatiques. Elle se distingue par sa simplicité de déploiement, son coût réduit et son adaptation au contexte local.

Conclusion

Ce chapitre a permis de présenter le cadre général du projet, d'analyser les contraintes du domaine, ainsi que d'étudier les solutions existantes. Cette analyse met en évidence les limites des systèmes actuels et justifie le développement d'une solution adaptée, qui sera détaillée dans les chapitres suivants.

Le chapitre suivant se concentrera sur la méthodologie adoptée pour la réalisation du projet, en détaillant les étapes de développement, les choix technologiques ainsi que les processus de gestion de projet mis en place. Il présentera également les défis rencontrés et les solutions mises en œuvre pour garantir une livraison efficace et conforme aux besoins identifiés.

CHAPITRE 2

Méthodologie et spécifications des besoins

Table des matières

Introduction	13
2.1 Analyse et spécification des besoins	13
2.1.1 Identification des acteurs	13
2.1.2 Spécification des besoins	13
2.2 Choix de la méthodologie	15
2.2.1 Étude comparative des méthodologies	15
2.2.2 Méthodologie retenue : Scrum	16
2.3 Présentation de Scrum	17
2.3.1 Artefacts Scrum et leur application	18
2.3.2 Événements Scrum et leur implémentation	18
2.4 Les rôles Scrum	19
2.5 Outils de support à Scrum	19
Conclusion	20

Introduction

Dans ce chapitre, nous présentons la méthodologie adoptée pour le développement du projet **Parc Informatique FMM**, ainsi que les spécifications des besoins fonctionnels et non fonctionnels.

2.1. Analyse et spécification des besoins

L'analyse des besoins est une étape cruciale pour définir les fonctionnalités et les contraintes du système. Elle permet de s'assurer que le projet répond aux attentes des utilisateurs finaux et des responsables informatiques.

2.1.1. Identification des acteurs

Le système interagit avec plusieurs acteurs essentiels, chacun jouant un rôle distinct dans son fonctionnement. Ces acteurs, résumés dans un tableau dédié, correspondent aux besoins de gestion d'un parc informatique au sein du SmartLab de la Faculté de Médecine de Monastir.

TABLE 2.5 – Acteurs du système et leurs rôles

Acteur	Description
Administrateur du parc	Gère les équipements informatiques, les utilisateurs, les affectations et les paramètres du système.
Technicien informatique	Traite les tickets d'incidents, réalise les maintenances et met à jour l'état des équipements.
Responsable du parc informatique	Planifie les maintenances préventives, suit les budgets et contrôle les inventaires.
Utilisateur final	Signale les incidents, consulte le statut de son matériel et effectue des demandes de support.
Système d'inventaire	Base de données et interface qui stocke les équipements, les licences, les historiques et les rapports.

2.1.2. Spécification des besoins

La spécification des besoins est essentielle pour définir les fonctionnalités et les contraintes du système. Elle se divise en deux catégories principales : les besoins fonctionnels et les besoins non fonctionnels.

Besoins fonctionnels :

Le projet **Parc Informatique FMM** a pour principal objectif d'automatiser la gestion du parc informatique, d'optimiser la résolution des incidents et de fournir une interface de suivi efficace. Les besoins fonctionnels traduisent ces objectifs sous forme de fonctionnalités concrètes :

- **Gestion des équipements** : Enregistrement, modification et suivi des actifs informatiques (ordinateurs, serveurs, périphériques, licences).

- **Gestion des tickets** : Création, assignation, suivi et clôture des incidents signalés par les utilisateurs.
- **Affectation des ressources** : Attribution des équipements aux utilisateurs, suivi des emplacements et gestion des transferts.
- **Maintenance préventive et curative** : Planification des interventions, suivi des actions réalisées et génération de notifications pour les échéances.
- **Reporting et tableaux de bord** : Visualisation des indicateurs clés (taux d'incidents, état des équipements, maintenances programmées) et export de rapports.
- **Gestion des utilisateurs et des rôles** : Authentification, autorisation et profils adaptatifs selon les rôles (administrateur, technicien, utilisateur).
- **Notifications** : Envoi d'alertes aux techniciens et aux responsables lors des incidents, des maintenances à venir ou des équipements en anomalie.
- **Import et intégration** : Possibilité d'importer des listes d'équipements et d'intégrer des données existantes via API ou fichiers CSV.

Besoins non fonctionnels :

Les besoins non fonctionnels contribuent à la qualité, à la fiabilité et à la cohérence du prototype dans le cadre de ce projet. Ils s'articulent autour des axes suivants :

- **Disponibilité** : Le système doit être accessible en permanence, avec un temps d'arrêt minimal afin de garantir la continuité du service.
- **Sécurité des données** : Les informations sur les équipements, les utilisateurs et les incidents doivent être protégées par des mécanismes d'authentification, d'autorisation et de chiffrement.
- **Performance** : L'application doit rester réactive même avec un grand nombre d'équipements et de tickets.
- **Interopérabilité** : La solution doit pouvoir s'intégrer avec des outils existants, des bases de données externes et des formats standards (CSV, API REST).

- **Évolutivité** : L'architecture doit permettre d'ajouter de nouvelles fonctionnalités sans re-fonte majeure.
- **Usabilité** : L'interface web doit être intuitive, facilitant la prise en main par les techniciens et les utilisateurs non spécialistes.

2.2. Choix de la méthodologie

Pour le développement du projet *Parc Informatique FMM*, plusieurs méthodologies de développement logiciel peuvent être envisagées, chacune présentant des avantages et des limites. Parmi les méthodologies les plus courantes figurent Scrum, Waterfall, Lean et Kanban.

2.2.1. Étude comparative des méthodologies

Le choix d'une méthodologie de développement constitue une étape déterminante dans la réussite d'un projet logiciel. Il influence directement la qualité du produit final, la capacité d'adaptation aux changements, ainsi que la gestion des risques et des coûts tout au long du cycle de vie du système.

Dans le cadre de ce projet, une comparaison des principales méthodologies de développement logiciel a été réalisée afin d'identifier l'approche la plus adaptée au contexte du projet *Parc Informatique FMM*. Le Tableau 2.6 synthétise cette analyse en se basant sur plusieurs critères essentiels, notamment l'adaptabilité aux changements, la fréquence des livraisons, le niveau de collaboration, la gestion des risques ainsi que la qualité et la maintenabilité du produit.

TABLE 2.6 – Tableau comparatif des méthodologies de développement logiciel.

Caractéristiques	Scrum	Waterfall	Lean	Kanban
Adaptabilité aux changements	✓	X	X	✓
Livraisons fréquentes	✓	X	X	X
Collaboration renforcée	✓	X	X	✓
Réponse rapide aux changements	✓	X	X	X
Gestion des risques	✓	✓	✓	✓
Niveau d'implication du client	Élevé	Faible	Moyen	Moyen
Efficacité des coûts	Élevée	Moyenne	Élevée	Moyenne
Assurance qualité	Continue	Finale	Continue	Continue
Évolutivité	Moyenne	Élevée	Élevée	Moyenne
Maintenance et support	Facile	Difficile	Facile	Facile

L'analyse du Tableau 2.6 met en évidence des différences importantes entre les méthodologies étudiées. Ces différences permettent de comprendre les avantages et limites de chaque approche en fonction du type de projet et du niveau de flexibilité requis.

Les méthodes agiles, notamment **Scrum** et **Kanban**, se distinguent clairement par leur capacité d'adaptation aux changements. Elles permettent une grande flexibilité dans la gestion des besoins, ce qui est particulièrement important dans les projets où les exigences peuvent évoluer au cours du développement. Scrum, en particulier, se caractérise par des livraisons fréquentes et une forte implication du client, ce qui favorise un meilleur alignement entre le produit développé et les besoins réels.

En revanche, la méthodologie **Waterfall** (ou cycle en V) repose sur une approche linéaire et séquentielle. Elle offre une meilleure structuration et une gestion des risques bien définie, mais elle reste peu adaptée aux changements une fois la phase de conception terminée. Cela la rend moins flexible dans des environnements dynamiques.

Les approches **Lean** et **Kanban** se positionnent comme des méthodes intermédiaires, visant principalement à optimiser le flux de travail et à réduire les gaspillages. Elles assurent une amélioration continue du processus, mais leur niveau de structuration reste parfois moins rigide que celui des approches traditionnelles.

En termes de qualité, toutes les méthodologies permettent une gestion des risques, mais avec des approches différentes : Scrum et Kanban favorisent une assurance qualité continue grâce à des cycles courts et des ajustements réguliers, tandis que Waterfall privilégie une validation finale après chaque phase.

Enfin, du point de vue de la maintenance et de l'évolution du système, les approches agiles offrent une meilleure flexibilité et une facilité d'adaptation, contrairement à Waterfall qui rend les modifications plus coûteuses et complexes après le déploiement.

Ainsi, cette analyse comparative met en évidence la supériorité des approches agiles, notamment Scrum, pour les projets nécessitant flexibilité, interaction continue avec le client et adaptation progressive des besoins, ce qui correspond au contexte du projet *Parc Informatique FMM*.

2.2.2. Méthodologie retenue : Scrum

À l'issue de l'étude comparative des différentes approches de développement, la méthodologie **Scrum** a été retenue pour la réalisation du projet *Parc Informatique FMM*. Ce choix repose sur un ensemble de critères techniques et organisationnels directement alignés avec les besoins du projet et les contraintes du contexte de développement.

Tout d'abord, Scrum se distingue par son approche itérative et incrémentale, qui permet de livrer le système par petites versions successives. Cette caractéristique offre la possibilité d'intégrer progressivement les fonctionnalités du système, notamment la gestion des équipements, des incidents et des utilisateurs, tout en validant continuellement leur conformité aux besoins exprimés.

Par ailleurs, cette méthodologie favorise l'implication continue des parties prenantes, en particulier les utilisateurs finaux tels que les techniciens et les responsables du SmartLab de la Faculté de Médecine de Monastir. Les retours réguliers recueillis à la fin de chaque sprint permettent

d'ajuster les exigences fonctionnelles et non fonctionnelles, réduisant ainsi le risque de divergence entre le besoin exprimé et la solution développée.

En outre, Scrum offre une grande flexibilité face aux changements, ce qui constitue un avantage majeur dans le cadre de ce projet, où certaines contraintes techniques et fonctionnelles ne sont pas entièrement figées au début du développement. Cette adaptabilité permet d'optimiser les choix techniques au fur et à mesure de l'avancement du projet, notamment lors de l'intégration des différents modules du système.

Enfin, la méthodologie Scrum encourage une collaboration étroite entre les différents acteurs du projet, à savoir le Product Owner, le Scrum Master et l'équipe de développement. Cette organisation favorise une communication fluide et une meilleure coordination des tâches, contribuant ainsi à une amélioration continue de la qualité du produit final.

Ainsi, le choix de Scrum s'inscrit dans une démarche pragmatique et adaptée au contexte du projet, en garantissant à la fois flexibilité, qualité et amélioration progressive du système développé.

2.3. Présentation de Scrum

Scrum est un framework agile qui repose sur des cycles de développement appelés sprints, comme présenté dans la figure 2.6. Chaque sprint a une durée définie (généralement entre 2 et 4 semaines) et aboutit à un produit fonctionnel et potentiellement livrable. Le processus Scrum comprend plusieurs éléments clés tels que le *Product Backlog*, le *Sprint Backlog*, l'*Incrément*, ainsi que des événements tels que la planification de sprint, les réunions quotidiennes (*Daily Scrum*), la revue de sprint et la rétrospective de sprint [11].

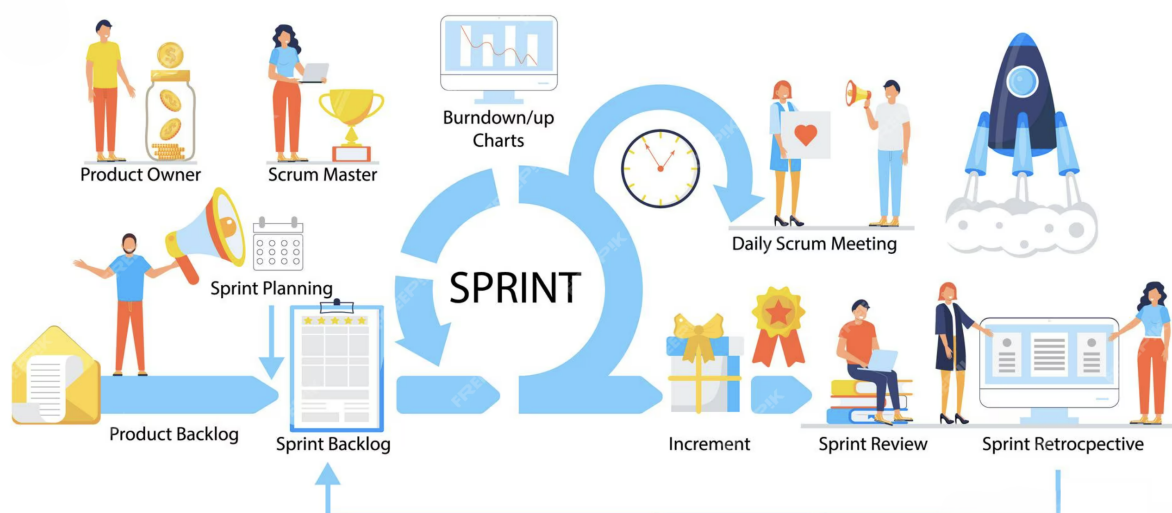


FIGURE 2.6 – Processus Scrum.

2.3.1. Artefacts Scrum et leur application

Scrum repose sur trois artefacts principaux qui structurent le développement du projet : le *Product Backlog*, le *Sprint Backlog* et l'*Incrément*. Ces artefacts sont appliqués comme suit dans le cadre du projet [12] :

- **Product Backlog** : Le *Product Backlog* regroupe l'ensemble des fonctionnalités à développer, telles que définies dans le tableau 4.8 (voir section 4.1.2). Il est géré par le Product Owner, qui priorise les tâches en utilisant la méthode MoSCoW pour classer les fonctionnalités en *Must*, *Should*, *Could* et *Won't*. Ce backlog est régulièrement affiné pour intégrer les retours des parties prenantes et garantir que les fonctionnalités prioritaires répondent aux besoins critiques du parc informatique.
- **Sprint Backlog** : À chaque sprint, un sous-ensemble de tâches est sélectionné à partir du *Product Backlog* pour constituer le *Sprint Backlog*. Par exemple, pour le Sprint 1, les tâches se concentrent sur la mise en place du modèle de données des équipements et des tickets, ainsi que sur l'authentification des utilisateurs.
- **Incrément** : Chaque sprint produit un incrément fonctionnel et potentiellement livrable. Par exemple, à l'issue du Sprint 1, un module de gestion des équipements et un premier prototype de ticketing peuvent être livrés et testés par les techniciens.

2.3.2. Événements Scrum et leur implémentation

Scrum repose sur des événements clés qui structurent le processus de développement. Ces événements sont implémentés comme suit dans le cadre du projet [13] :

- **Planification de sprint** : Lors de la réunion de planification, l'équipe sélectionne les tâches du *Product Backlog* à inclure dans le *Sprint Backlog*, en estimant l'effort requis. Par exemple, pour le Sprint 1, la construction du schéma de données des équipements et la création du module de ticketing peuvent être estimés en jours.
- **Daily Scrum** : Des réunions quotidiennes de 15 minutes sont organisées pour suivre l'avancement des tâches et identifier les obstacles. Par exemple, si un problème de connexion à la base de données est détecté, il est abordé immédiatement pour éviter un retard sur le sprint.
- **Revue de sprint** : À la fin de chaque sprint, l'incrément est présenté aux parties prenantes, comme les techniciens ou les responsables du parc informatique. Leurs retours permettent de valider la conformité des fonctionnalités et d'ajuster les priorités pour le sprint suivant.
- **Rétrospective de sprint** : Après chaque sprint, l'équipe analyse les succès et les défis rencontrés. Par exemple, si l'ergonomie du tableau de bord des équipements nécessite des améliorations, la rétrospective permet d'ajouter ces éléments au backlog.

2.4. Les rôles Scrum

Dans Scrum, trois rôles principaux sont définis pour assurer le bon fonctionnement du processus de développement [14] :

- **Product Owner** : Représentant des parties prenantes, il définit et priorise les fonctionnalités à développer en fonction des besoins des utilisateurs et des objectifs du projet.
- **Scrum Master** : Responsable de la bonne application du framework Scrum. Il facilite la communication, aide à surmonter les obstacles et veille à l'amélioration continue des processus.
- **Équipe de développement** : Composée de développeurs, designers et autres spécialistes, elle est responsable de la mise en œuvre des fonctionnalités et de la livraison des incréments du produit.

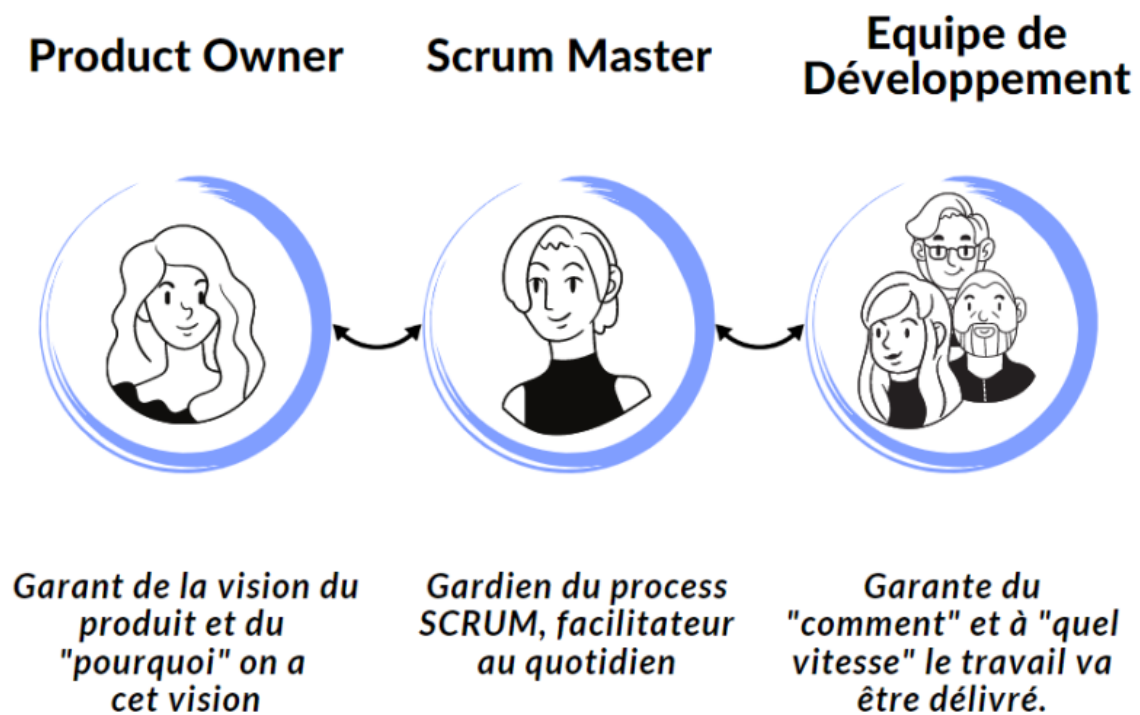


FIGURE 2.7 – Rôles Scrum.

2.5. Outils de support à Scrum

Pour faciliter la mise en œuvre de Scrum, plusieurs outils numériques sont utilisés pour gérer les processus, la communication et le suivi des tâches :

- **ClickUp** : Utilisé pour gérer le *Product Backlog* et le *Sprint Backlog*, ClickUp permet de suivre les tâches, d'assigner des responsabilités et de visualiser l'avancement des sprints à l'aide de tableaux Kanban ou Scrum [15]. Les tâches du projet, comme la mise en place de l'inventaire des équipements ou la création du module de tickets, sont suivies via des tickets

ClickUp avec des estimations de temps et des statuts mis à jour quotidiennement.



FIGURE 2.8 – Logo de ClickUp.

- **Slack** : Cette plateforme de communication permet des échanges en temps réel entre l'équipe de développement, le Product Owner et le Scrum Master [16]. Les *Daily Scrum* peuvent être partiellement gérés via des canaux dédiés, où les membres signalent leurs progrès et obstacles.



FIGURE 2.9 – Logo de Slack.

- **Git** : Utilisé pour le contrôle de version, Git permet de gérer le code source du projet et de faciliter la collaboration entre les développeurs [17]. Les commits sont alignés avec les tâches du *Sprint Backlog*, garantissant une traçabilité des contributions.



FIGURE 2.10 – Logo de Git.

Conclusion

Ce chapitre a présenté la méthodologie Scrum adoptée pour le projet **Parc Informatique FMM**, en détaillant les spécifications des besoins fonctionnels et non fonctionnels, ainsi que les rôles, artefacts et événements Scrum.

Le chapitre suivant se concentrera sur l'architecture du système, en décrivant les choix technologiques, les composants principaux et les interactions entre les différentes parties du système. Il abordera également les défis techniques rencontrés et les solutions mises en place pour garantir la fiabilité et l'évolutivité de l'application.

CHAPITRE 3

Étude architecturale du projet

Table des matières

Introduction	22
3.1 Présentation de l'architecture générale	22
3.2 Modèle de conception logicielle adopté	23
3.2.1 Hiérarchie organisationnelle et modèle de localisation	23
3.3 Interactions et fonctionnement du système	24
3.3.1 Diagramme des cas d'utilisation	24
3.3.2 Diagramme de classes	24
3.3.3 Diagrammes de séquence	26
3.3.4 Diagramme d'activités	29
3.4 Pourquoi ce choix d'architecture	32
Conclusion	32

Introduction

Ce chapitre décrit l'architecture du projet **Parc Informatique FMM**. L'objectif est d'expliquer les choix technologiques, les composants du système et leurs interactions afin de répondre aux besoins de gestion des équipements, des incidents et des maintenances au sein du SmartLab de la Faculté de Médecine de Monastir.

3.1. Présentation de l'architecture générale

Le système *Parc Informatique FMM* repose sur une architecture en trois couches : gestion des ressources, traitement des services et interfaces utilisateur. Chaque couche joue un rôle clairement défini, comme le montre la Figure 3.11. Cette organisation permet de structurer de manière évolutive la gestion des actifs informatiques, des tickets et des rapports.

- **Gestion des ressources** : Cette couche gère les équipements informatiques, les licences, les utilisateurs et les emplacements via une base de données centralisée. Elle permet de consigner l'état des actifs et de suivre leur historique.
- **Traitement des services** : Cette couche prend en charge la logique métier du ticketing, de la gestion des maintenances et des notifications. Le backend traite les demandes, assigne les incidents et génère les alertes selon les règles définies.
- **Interfaces utilisateur** : Cette couche propose une interface web pour que les administrateurs, techniciens et utilisateurs puissent consulter les actifs, créer des tickets et suivre les interventions.

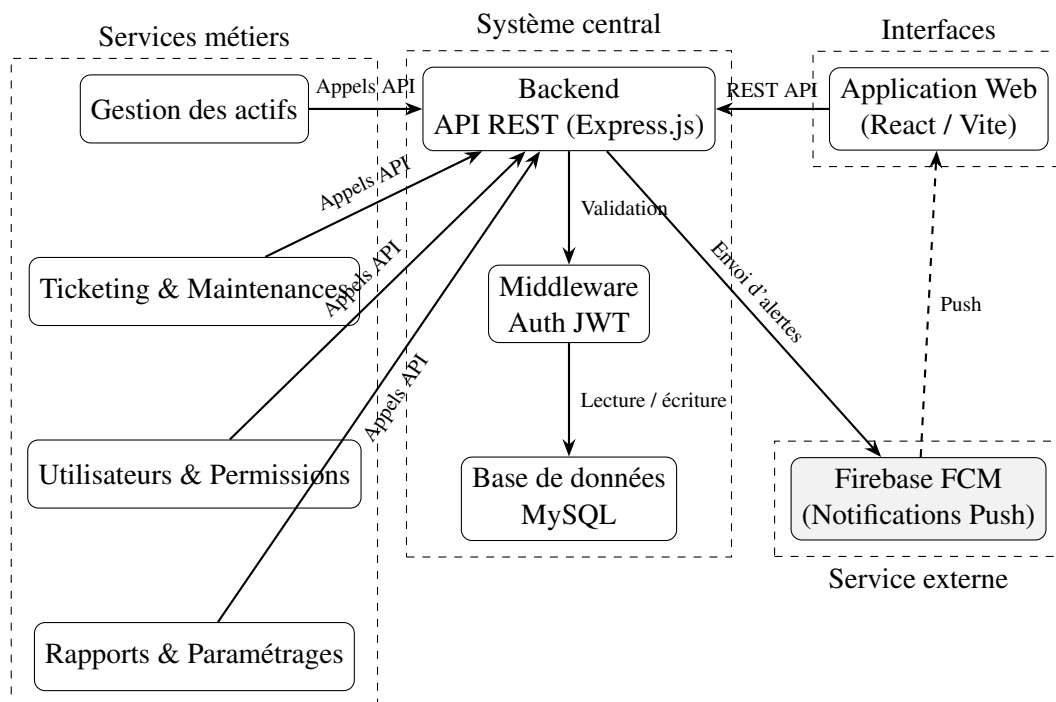


FIGURE 3.11 – Architecture globale du Parc Informatique FMM

3.2. Modèle de conception logicielle adopté

Pour organiser le système *Parc Informatique FMM*, nous avons choisi le modèle MVC (Modèle-Vue-Contrôleur). Ce modèle permet de séparer les données, l’affichage et la logique métier, ce qui facilite la maintenance, les tests et l’évolution du système.

- **Modèle** : Cette couche gère les données. La base MySQL centralise les informations relatives aux équipements, aux licences, aux utilisateurs, aux tickets et aux maintenances.
- **Vue** : Cette couche propose des interfaces visuelles. L’application web affiche les tableaux de bord, les états des équipements et les tickets.
- **Contrôleur** : Cette couche gère la logique métier. Le backend traite les requêtes, exécute les règles de gestion, assigne les tickets et déclenche les notifications.

3.2.1. Hiérarchie organisationnelle et modèle de localisation

Le système *Parc Informatique FMM* modélise l’organisation physique du SmartLab selon une hiérarchie à trois niveaux de localisation. Contrairement à une approche monolithique, cette structure décentralisée offre une flexibilité maximale pour représenter les emplacements complexes :

- **Bloc** : Entité de plus haut niveau représentant un bâtiment, aile ou bloc de locaux
- **Département** : Division administrativo-fonctionnelle au sein d’un Bloc, permettant de regrouper les ressources par domaine métier
- **Salle** : Espace physique concret (laboratoire, salle de cours, bureau, serveur) hébergeant équipements et utilisateurs

Les relations sont structurées comme suit :

- Bloc (1 : *) Département
- Bloc (1 : *) Salle (emplacements directs)
- Département (1 : *) Salle (emplacements subdivisés)

Cette hiérarchie n’est pas rigide : une Salle peut être liée directement à un Bloc (sans passer par Département), ou via un Département intermédiaire. Les équipements et utilisateurs héritent leur localisation via les clés étrangères `DepartmentId`, `SalleId` et `BlocId`.

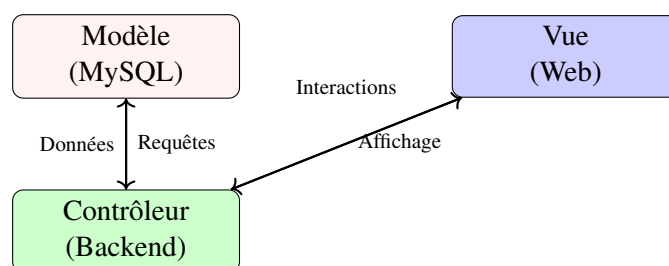


FIGURE 3.12 – Modèle MVC du Parc Informatique FMM

Cette structure permet de séparer clairement les responsabilités, rendant le système plus modulaire et adaptable.

3.3. Interactions et fonctionnement du système

Cette section présente les diagrammes UML qui illustrent la collaboration entre les différentes parties du système.

3.3.1. Diagramme des cas d'utilisation

Le diagramme des cas d'utilisation (Figure 3.13) montre comment les acteurs (administrateur, technicien, utilisateur) interagissent avec le système. Il illustre les actions comme la consultation des actifs, la création de tickets, la validation des interventions et la génération de rapports.

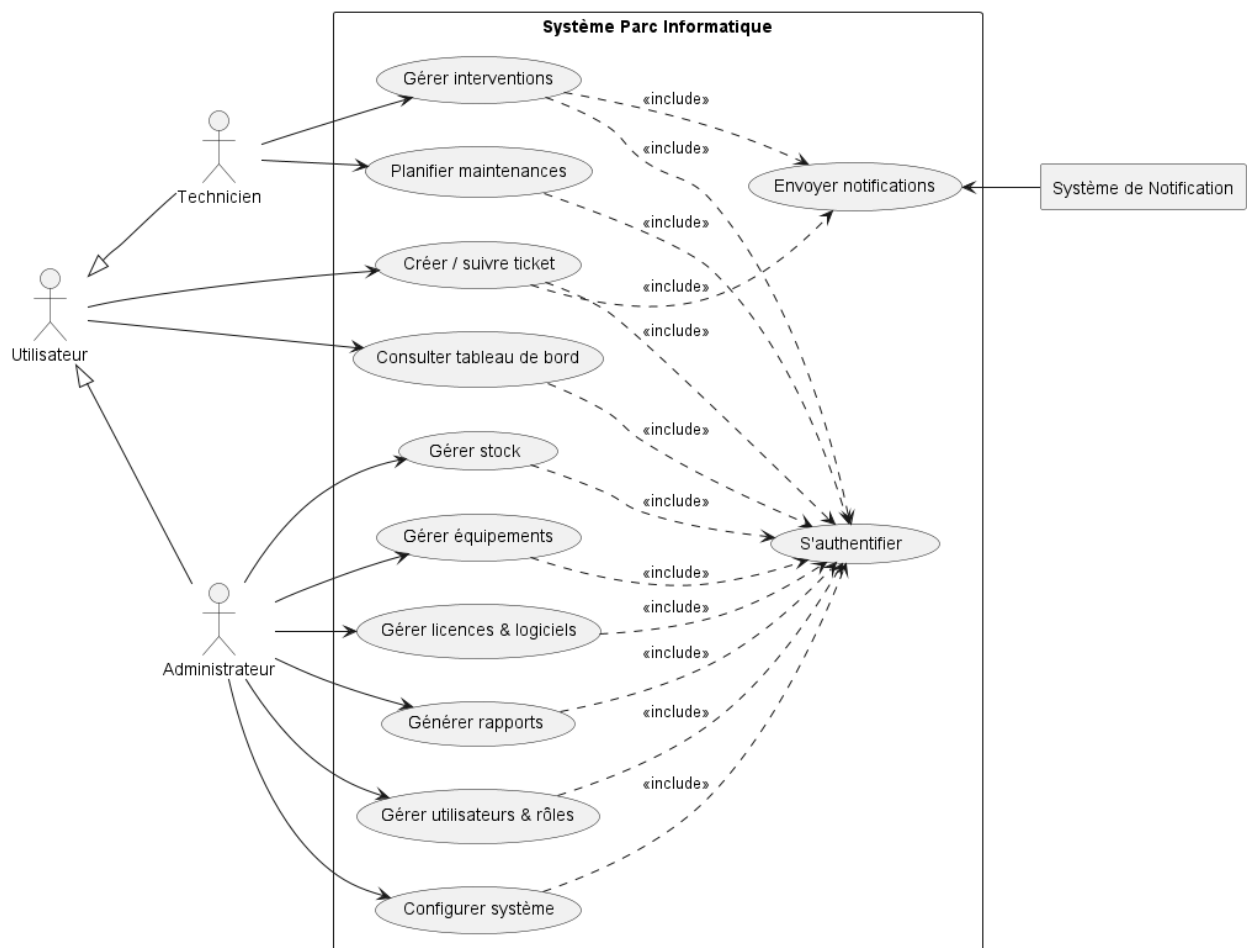


FIGURE 3.13 – Diagramme des cas d'utilisation global

3.3.2. Diagramme de classes

Le diagramme de classes (Figure 3.14) présente la structure des entités du système, notamment *Equipement*, *Ticket*, *Utilisateur*, *Maintenance* et *Licence*. Il précise l'organisation des données ainsi que les relations entre les différentes entités.

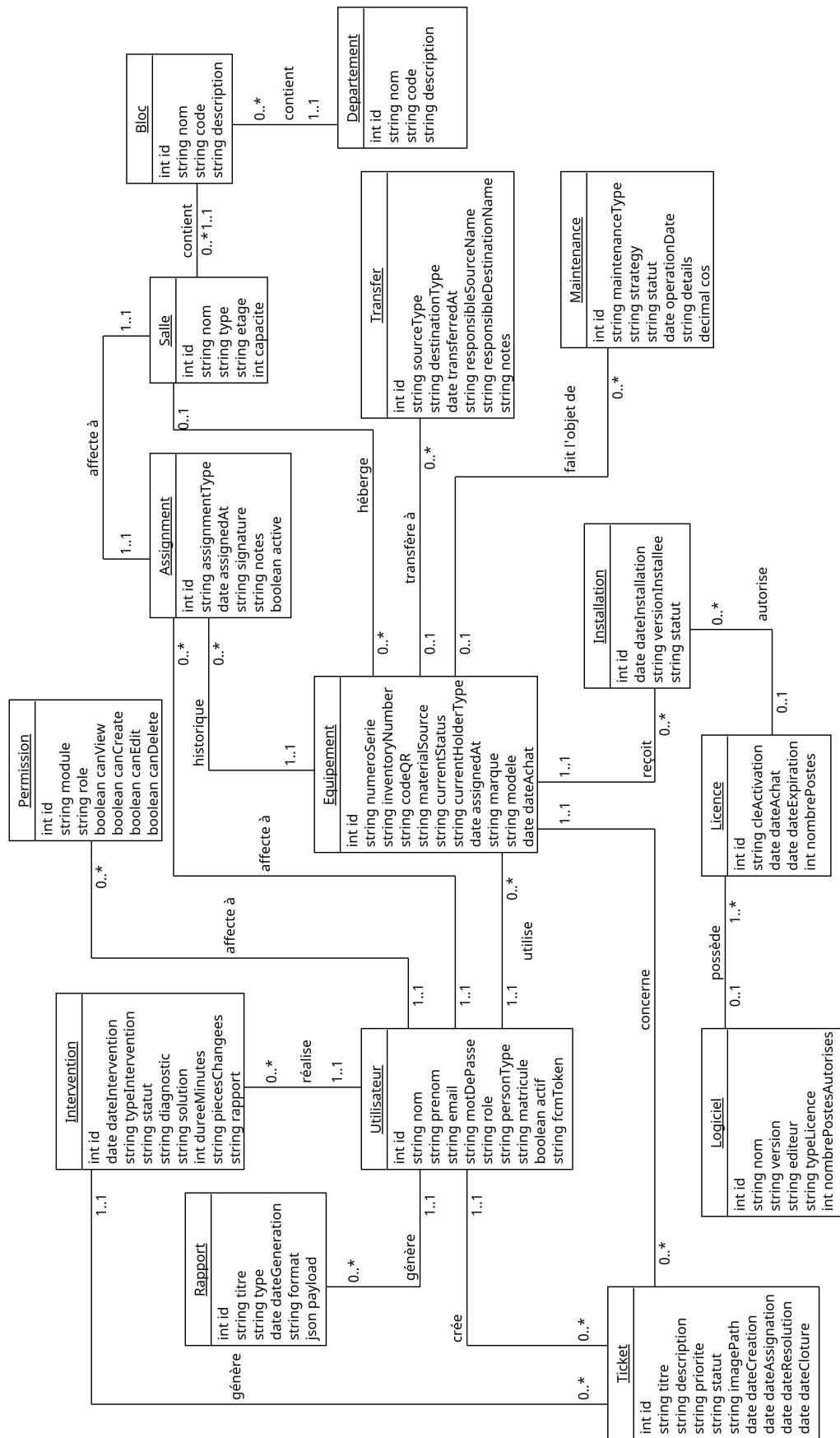


FIGURE 3.14 – Diagramme de classes du Parc Informatique FMM

3.3.3. Diagrammes de séquence

Les diagrammes de séquence constituent un élément essentiel de la modélisation dynamique du système *Parc Informatique FMM*. Ils permettent de représenter de manière chronologique les échanges entre les différents composants du système, notamment l'utilisateur, le frontend, le backend, la base de données ainsi que les services externes tels que Firebase Cloud Messaging (FCM).

Dans ce projet, plusieurs scénarios fonctionnels représentatifs ont été modélisés afin d'illustrer les principaux flux métiers et de mettre en évidence les interactions entre les différentes couches de l'architecture.

Authentification de l'utilisateur

Le diagramme de séquence d'authentification 3.15 illustre le processus permettant à un utilisateur d'accéder au système de manière protégée.

Dans ce scénario, l'utilisateur saisit ses identifiants via l'interface web. Ces informations sont ensuite transmises par le frontend au backend, qui procède à leur vérification auprès de la base de données. Si les identifiants sont valides, le système génère un token JWT qui est renvoyé au client. Le frontend stocke alors ce token afin d'autoriser l'accès aux ressources protégées, notamment le tableau de bord.

Ce mécanisme permet un accès protégé au système basé sur une authentification par token.

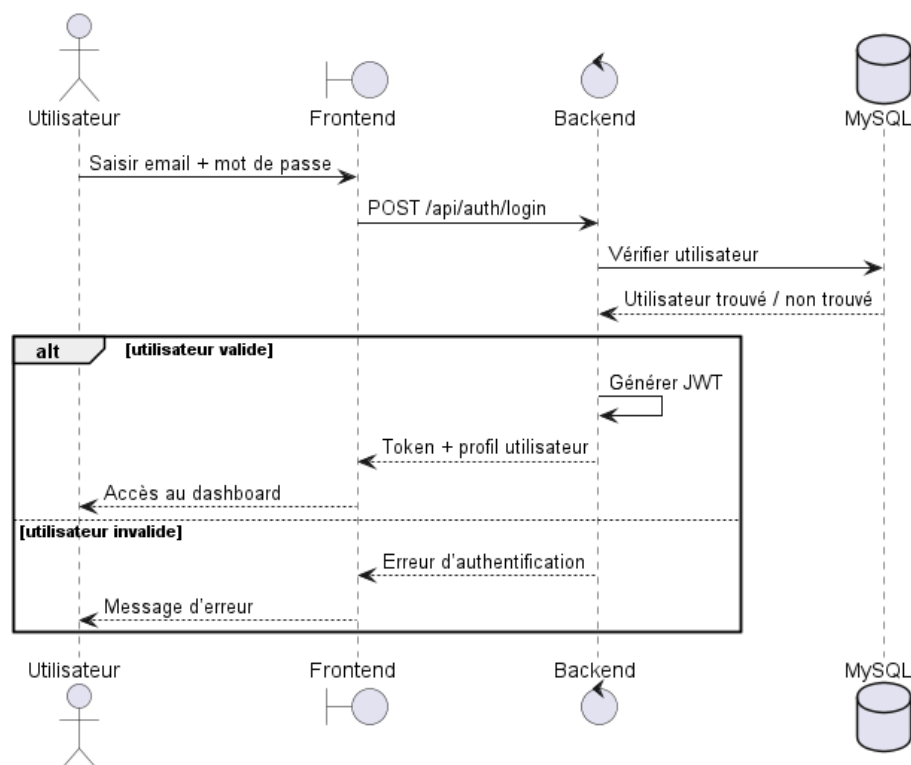


FIGURE 3.15 – Diagramme de séquence d'authentification

Création et assignation d'un ticket d'incident

Ce diagramme 3.16 décrit le flux principal de gestion des incidents, depuis la création d'un ticket jusqu'à son assignation à un technicien.

Dans ce scénario, l'utilisateur crée un ticket en spécifiant les détails de l'incident tels que le titre, la priorité et l'équipement concerné. Le frontend envoie ensuite ces données au backend via un appel `POST /api/tickets`, qui enregistre le ticket dans la base de données avec un statut initial *ouvert*.

Si un technicien est désigné au moment de la création (champ `TechnicienId` renseigné), le backend applique automatiquement la transition de statut vers *assigné* et enregistre la date d'assignation. Dans le cas contraire, le ticket reste à l'état *ouvert* jusqu'à son affectation ultérieure par l'administrateur. Dès la création, une notification push est diffusée à l'ensemble des utilisateurs connectés via Firebase Cloud Messaging.

Ce mécanisme de gestion d'état est implémenté dans la route `POST /api/tickets` et consolidé par le moteur de transitions `utils/statusMachine.js`, qui valide la cohérence de chaque changement de statut.

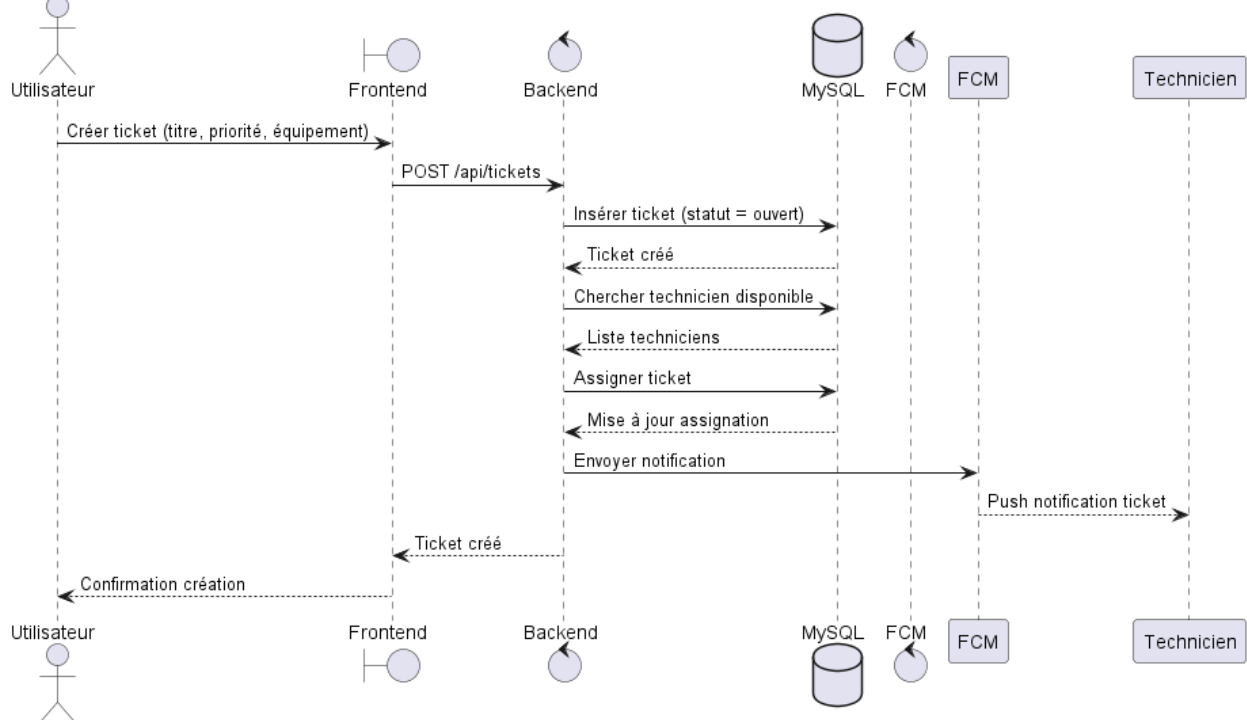


FIGURE 3.16 – Diagramme de séquence de création et assignation d'un ticket

Traitement et mise à jour d'un ticket

Ce diagramme 3.17 illustre les interactions entre le technicien et le système lors du traitement d'un ticket.

Le technicien commence par consulter la liste des tickets qui lui sont attribués. Le frontend interroge alors le backend afin de récupérer les données correspondantes depuis la base de données. Ensuite, le technicien met à jour le statut du ticket en fonction de l'avancement de son intervention (en cours, résolu, etc.).

Ces modifications sont enregistrées par le backend et persistées dans la base de données, ce qui permet de maintenir un historique fonctionnel et cohérent des actions effectuées.

Ce flux contribue ainsi à la traçabilité des interventions techniques.

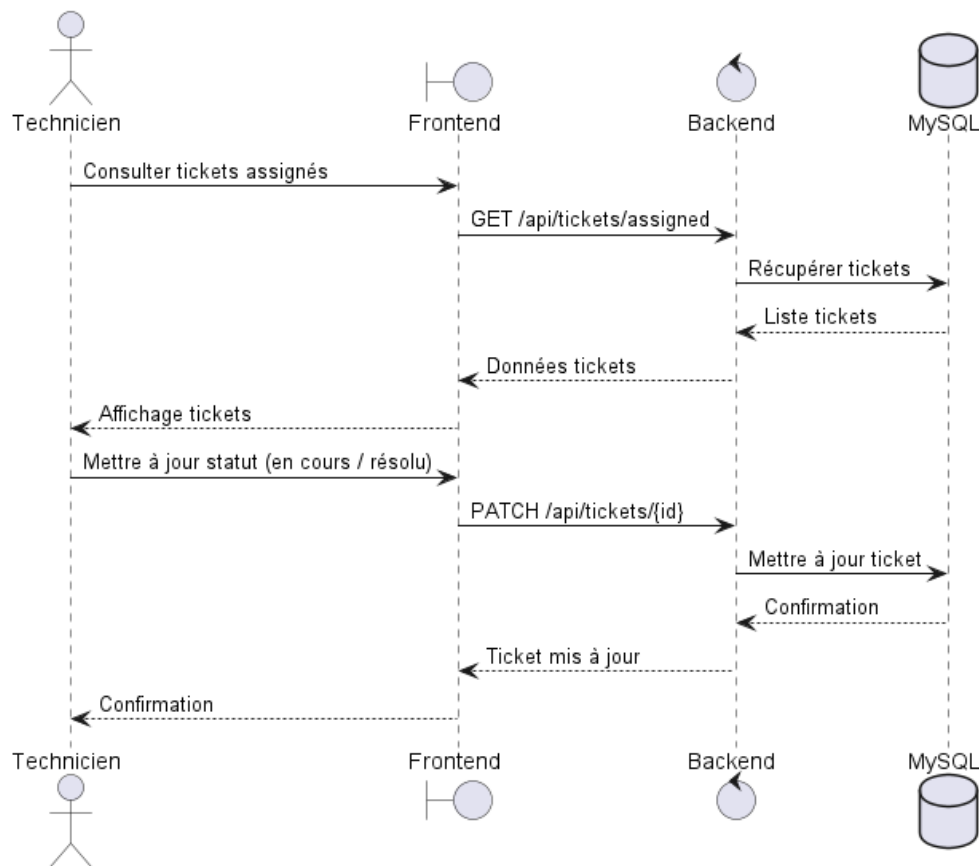


FIGURE 3.17 – Diagramme de séquence de traitement et mise à jour d'un ticket

Clôture et consultation de l'historique

Ce dernier diagramme 3.18 présente le processus de clôture d'un ticket ainsi que la consultation de son historique.

Une fois l'incident résolu, le ticket est clôturé par l'utilisateur ou automatiquement par le système. Le backend met alors à jour son statut à *fermé* et enregistre la date de clôture dans la base de données.

L'utilisateur peut par la suite consulter l'historique fonctionnel du ticket afin de suivre l'ensemble des actions réalisées depuis sa création jusqu'à sa fermeture.

Ce mécanisme renforce la traçabilité et améliore le suivi global des incidents.

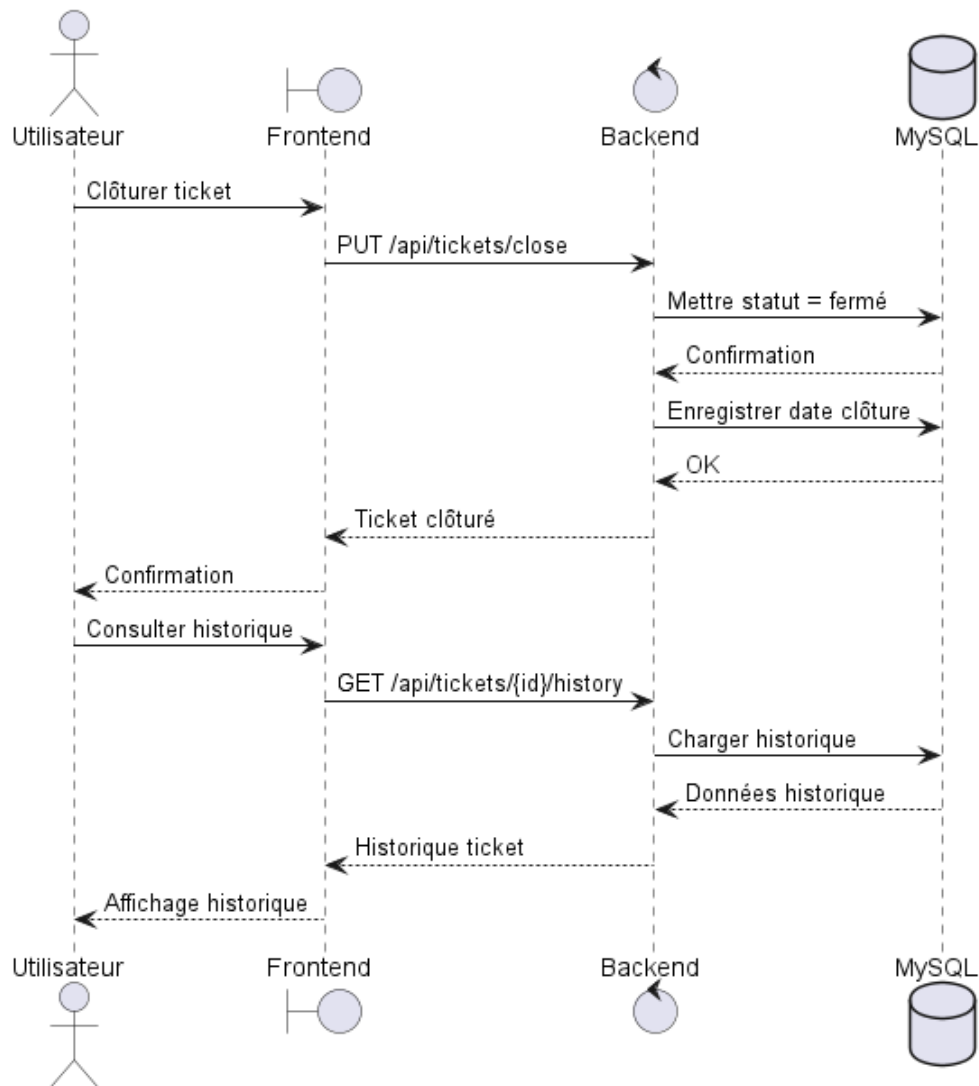


FIGURE 3.18 – Diagramme de séquence de clôture et consultation de l'historique d'un ticket

Synthèse L'ensemble de ces diagrammes de séquence met en évidence la cohérence globale de l'architecture du système ainsi que la fluidité des interactions entre ses différents composants. Ils soulignent également le rôle central du backend dans la coordination des traitements métiers, ainsi que l'importance des services externes tels que Firebase Cloud Messaging pour assurer la communication en temps réel entre les acteurs du système.

3.3.4. Diagramme d'activités

Le diagramme d'activités (Figure 3.19) illustre le processus fonctionnel de gestion des incidents dans le prototype *Parc Informatique FMM*. Il débute par la soumission d'un ticket par l'utilisateur, avec le titre, la priorité, l'équipement concerné et une photo. Les données sont ensuite validées; si elles sont conformes, le ticket est enregistré avec le statut *ouvert* et une notification push est envoyée au technicien via Firebase Cloud Messaging (FCM). Le technicien consulte les tickets, prend en charge le ticket concerné et modifie son statut en *assigné*. Il crée ensuite une

intervention, passe le ticket à l'état *en cours*, puis effectue la réparation sur le terrain. Si l'intervention aboutit, il renseigne la solution, les pièces remplacées et le rapport de fin d'intervention. Le système génère automatiquement une maintenance corrective liée à l'intervention et marque le ticket comme *résolu*. L'utilisateur confirme alors la résolution, ce qui clôture le ticket, enregistre la date de clôture et archive l'incident. En cas d'échec de l'intervention, le statut passe à *échec*, puis le ticket est réouvert ou escaladé. Si les données initiales ne sont pas valides, la demande est rejetée avec un message d'erreur.

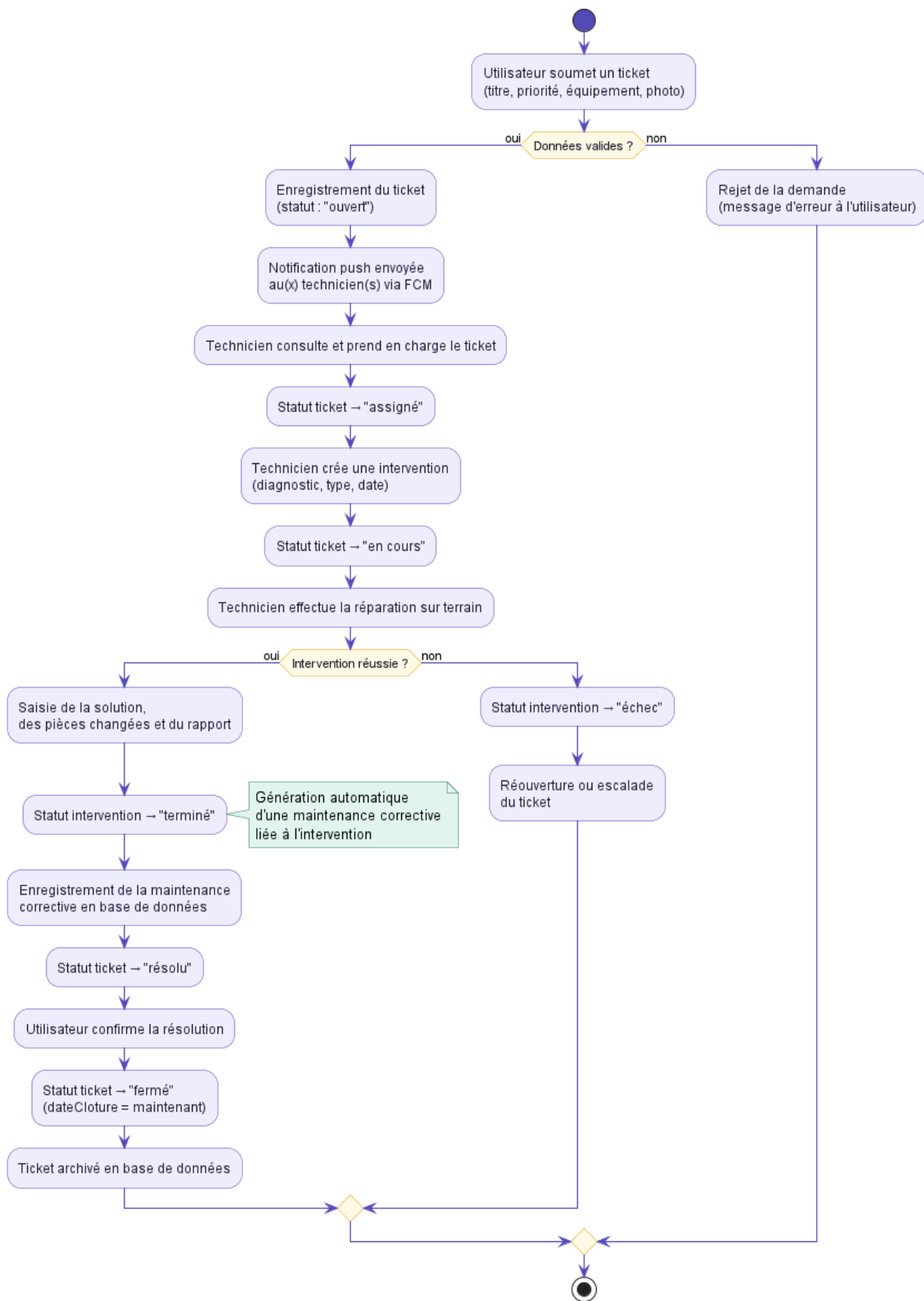


FIGURE 3.19 – Diagramme d'activité du flux des processus

3.4. Pourquoi ce choix d'architecture

Le choix de cette architecture repose sur plusieurs considérations techniques et fonctionnelles visant à obtenir un système efficace, évolutif et facile à maintenir dans le cadre de ce projet de fin d'études.

- **Simplicité et séparation des responsabilités** : L'architecture en couches permet de distinguer clairement les différentes parties du système, notamment la gestion des données, la logique métier et l'interface utilisateur. Cette séparation facilite la compréhension du système ainsi que son évolution.
- **Maintenabilité** : Grâce à cette organisation modulaire, toute modification ou amélioration peut être effectuée sur une couche spécifique sans impacter l'ensemble du système, ce qui réduit les risques de régression.
- **Réactivité du système** : Le traitement des tickets et des notifications est conçu de manière à assurer une prise en charge rapide des incidents, améliorant ainsi la réactivité globale du système.
- **Adaptabilité et extensibilité** : L'architecture retenue permet de supporter différents types d'équipements et de s'adapter facilement à divers environnements d'exécution, qu'il s'agisse d'infrastructures locales ou de solutions cloud.
- **Sécurité** : Des mécanismes de sécurité de base ont été implémentés, notamment l'authentification JWT et le contrôle d'accès basé sur les rôles (RBAC), afin de contribuer à la confidentialité et à l'intégrité des informations.

Ainsi, ces choix architecturaux répondent aux besoins observés au SmartLab, en offrant une solution modulaire, fiable et évolutive, adaptée à la gestion d'un parc informatique moderne à une échelle locale.

Conclusion

Ce chapitre a présenté l'architecture globale du projet *Parc Informatique FMM*, en mettant en évidence l'organisation en couches, l'approche MVC ainsi que les principaux diagrammes UML utilisés pour la modélisation du système.

L'architecture retenue favorise une meilleure structuration du système, tout en soutenant sa flexibilité, sa maintenabilité et sa capacité d'évolution afin de répondre efficacement aux besoins de gestion d'un parc informatique moderne. Les diagrammes de séquence, d'activités et de classes ont permis d'illustrer les interactions entre les différents composants du système, ainsi que les flux de données et les processus métiers clés.

Dans le chapitre suivant, nous détaillerons les choix technologiques effectués pour la mise en œuvre du projet, en expliquant les raisons de ces choix et leur impact sur le développement et la performance du système.

CHAPITRE 4

Planification et préparation du projet Parc Informatique FMM

Table des matières

Introduction	34
4.1 Pilotage du projet avec Scrum	34
4.1.1 Acteurs Scrum et missions	34
4.1.2 Organisation du Backlog	34
4.1.3 Planification des sprints	35
4.2 Environnement de travail	37
4.2.1 Environnement matériel	37
4.2.2 Environnement de développement	38
4.2.3 Environnement logiciel	39
Conclusion	41

Introduction

Le Sprint 0 constitue la phase initiale du projet Parc Informatique FMM, qui vise à développer un système de gestion des équipements informatiques, des incidents et des maintenances au sein du SmartLab de la Faculté de Médecine de Monastir. Cette étape fondamentale permet d'identifier les acteurs clés, de recueillir les besoins fonctionnels et non fonctionnels, et de définir la méthodologie Scrum qui guidera les phases ultérieures du développement. L'objectif est d'assurer un alignement avec les ambitions du projet, à savoir améliorer la gestion des actifs informatiques grâce à une collecte automatisée des données, une gestion des incidents en temps réel et des rapports détaillés, comme décrit dans les chapitres d'introduction générale et d'étude architecturale.

4.1. Pilotage du projet avec Scrum

Le projet adopte la méthodologie Scrum pour assurer un développement itératif et une collaboration étroite avec les parties prenantes, comme justifié dans le chapitre consacré à la méthodologie. Cette section détaille les rôles Scrum, l'organisation du backlog et la planification des sprints.

4.1.1. Acteurs Scrum et missions

Les rôles Scrum et leurs responsabilités sont définis dans un tableau spécifique, garantissant une approche collaborative alignée sur les principes décrits précédemment. Le Product Owner est chargé de définir et de prioriser les fonctionnalités du produit, tout en validant les livrables pour s'assurer qu'ils répondent aux besoins du projet. L'équipe de développement conçoit, développe et teste les fonctionnalités, en veillant à la qualité du code produit. Le Scrum Master facilite les événements Scrum, élimine les obstacles et veille à ce que l'équipe adhère aux pratiques de la méthodologie.

TABLE 4.7 – Acteurs du projet Scrum

Rôle	Mission	Acteur
Product Owner	Définir et prioriser les fonctionnalités du produit (Product Backlog), valider les livrables.	Pr Ammeri Charfeddine
Équipe de Développement	Concevoir, développer et tester les fonctionnalités, assurer la qualité du code.	Hela Boussaid
Scrum Master	Faciliter les événements Scrum, supprimer les obstacles, assurer l'adhésion à Scrum.	Racha Zaibi

4.1.2. Organisation du Backlog

Le Product Backlog est géré à l'aide d'un outil de gestion de projet et organisé selon la méthode de priorisation MoSCoW, afin de refléter les objectifs stratégiques du projet. Chaque fonctionnalité est formulée sous forme de User Story pour favoriser un développement centré sur l'utilisateur.

Les tâches prioritaires couvrent la gestion des équipements informatiques (ajout, modification et suppression), la gestion des incidents (création de tickets et affectation aux techniciens), ainsi que la gestion des maintenances préventives et correctives. Les interfaces web permettent la visualisation des équipements et la gestion des tickets, tandis que l'authentification et la gestion des utilisateurs garantissent un accès protégé basé sur les rôles (administrateur, technicien, utilisateur). Le stockage des données et des historiques dans une base de données relationnelle constitue également une priorité, au même titre que la génération de rapports sur les équipements et les incidents. D'autres tâches, importantes mais non critiques pour la première version, incluent les notifications push pour les nouveaux tickets, la gestion des licences logicielles et l'optimisation des performances pour un grand nombre d'équipements. Des fonctionnalités optionnelles, comme l'intégration avec des outils externes ou le support multilingue, pourront être ajoutées si les ressources le permettent. Enfin, certaines tâches, telles que l'analyse prédictive des pannes par intelligence artificielle, demeurent hors du périmètre actuel.

TABLE 4.8 – Backlog du Projet avec Priorités MoSCoW

ID	Tâche	Priorité
M-01	Gestion des équipements : ajout, modification, suppression et visualisation.	M
M-02	Gestion des incidents : création de tickets, assignation et résolution.	M
M-03	Gestion des maintenances : préventives et correctives.	M
M-04	Interface web pour administrateurs et techniciens.	M
M-05	Authentification et gestion des utilisateurs (rôles : admin, technicien, utilisateur).	M
M-06	Stockage et historique des données dans une base de données MySQL.	M
M-07	Génération de rapports sur les équipements et incidents.	M
S-01	Notifications push pour nouveaux tickets via Firebase FCM.	S
S-02	Gestion des licences logicielles associées aux équipements.	S
S-03	Optimisation des performances pour gestion de nombreux équipements.	S
C-02	Support multilingue pour l'interface.	C
W-01	Analyse prédictive des pannes avec IA.	W

4.1.3. Planification des sprints

Le projet est structuré en trois sprints de trois semaines chacun. Le tableau ci-dessous présente une synthèse des objectifs, livrables, tâches, critères d'acceptation et risques associés à chaque sprint.

TABLE 4.9 – Planification des Sprints du Projet Parc Informatique FMM

Sprint	Objectifs et livrables	Tâches principales	Critères d'acceptation
Sprint 1 Mise en place de l'architecture et gestion des équipements	- Backend API REST avec Express.js. - Base de données MySQL configurée. - Module de gestion des équipements fonctionnel.	- Configuration du serveur backend. - Création des schémas de base de données. - Développement des endpoints pour CRUD des équipements. - Validation fonctionnelle des fonctionnalités.	- Équipements ajoutés, modifiés et supprimés via API. - Données persistées correctement en base. - Interface web basique pour visualisation.
Sprint 2 Gestion des incidents et maintenances	- Système de ticketing opérationnel. - Module de maintenances implémenté. - Notifications push intégrées.	- Développement des endpoints pour tickets et maintenances. - Intégration de Firebase FCM pour notifications. - Gestion de l'état des tickets et affectation. - Validation fonctionnelle par tests manuels.	- Tickets créés et assignés aux techniciens. - Maintenances générées lors de la clôture des interventions. - Notifications envoyées en temps réel.
Sprint 3 Finalisation et rapports	- Interface web fonctionnelle avec React. - Système de rapports fonctionnel. - Authentification et autorisation protégées.	- Développement de l'interface web. - Implémentation de l'authentification JWT. - Génération de rapports PDF/JSON. - Validation de la sécurité et des performances.	- Interface utilisateur intuitive et responsive. - Rapports générés sur demande. - Accès protégé selon les rôles.

4.2. Environnement de travail

Ce projet est développé sur une configuration locale Windows avec une architecture applicative fondée sur un backend Node.js/Express.js et un frontend React/Vite. Les sections suivantes décrivent l'environnement matériel, l'environnement de développement et l'environnement logiciel utilisés par l'équipe.

4.2.1. Environnement matériel

L'environnement matériel utilisé pour le développement du projet repose sur une machine de travail performante, permettant de garantir de bonnes performances lors de l'exécution des outils de développement, des tests et du déploiement local des applications.

Le poste de développement est un ordinateur portable équipé d'un processeur Intel Core i7 de 10^e génération, de 16 Go de mémoire RAM et d'un stockage SSD de 512 Go. Cette configuration assure une exécution fluide des environnements backend et frontend ainsi qu'une bonne réactivité lors de la compilation et des tests.

Le système d'exploitation utilisé est Windows, choisi pour sa stabilité et sa compatibilité avec les technologies employées dans le projet. La base de données MySQL est installée en local afin d'assurer une gestion efficace, rapide et protégée des données du système. Enfin, l'architecture réseau repose sur un environnement local (*localhost*), permettant de réaliser les tests et le développement sans dépendre d'une infrastructure externe.

Système d'exploitation



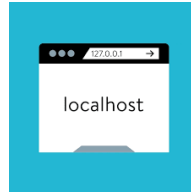
Windows est utilisé comme système principal de développement. Il offre une compatibilité avec les outils essentiels tels que Node.js, les environnements de développement intégrés (IDE) et les outils de gestion de bases de données.

Base de données



MySQL est utilisé comme système de gestion de base de données relationnelle. Il est installé en local et fonctionne sur le port 3306, assurant la persistance et l'intégrité des données du système.

Réseau

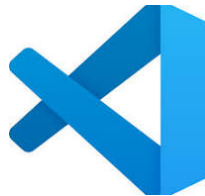


L'environnement réseau repose sur une configuration locale (*localhost*), utilisée pour le développement et les tests, garantissant un déploiement rapide et indépendant de toute infrastructure externe.

4.2.2. Environnement de développement

L'environnement de développement regroupe l'ensemble des outils utilisés pour la conception, l'implémentation et la documentation du projet. Il permet d'assurer un cycle de développement fonctionnel, allant de l'écriture du code jusqu'au test et à la génération de la documentation technique.

L'éditeur principal utilisé est Visual Studio Code, choisi pour sa légèreté, sa richesse en extensions et son support adapté des technologies web modernes.



L'environnement de développement du projet repose sur une architecture JavaScript Full Stack illustrée par la figure 4.20. Cette architecture combine plusieurs technologies complémentaires permettant de couvrir l'ensemble des besoins de l'application.

Du côté serveur, **Node.js** est utilisé comme environnement d'exécution JavaScript, tandis que **Express.js** fournit les mécanismes nécessaires à la conception des services Web REST assurant la communication entre les différentes couches du système. La couche de présentation est développée à l'aide de **React.js**, une bibliothèque JavaScript moderne facilitant la création d'interfaces utilisateur interactives, réactives et modulaires.

Pour la persistance des données, le projet s'appuie sur le système de gestion de bases de données relationnelles **MySQL**. L'accès aux données est assuré par **Sequelize ORM**, qui permet de manipuler les entités métier sous forme d'objets tout en simplifiant les opérations de gestion de la

base de données. Cette approche améliore la maintenabilité du code et réduit la complexité liée à l'écriture des requêtes SQL.

L'adoption de cette architecture garantit une séparation claire des responsabilités entre les différentes couches du système, favorise la réutilisabilité du code et facilite l'évolution future de l'application.



FIGURE 4.20 – Architecture technologique

Afin d'améliorer la productivité de l'équipe de développement et de garantir la qualité du logiciel réalisé, plusieurs outils complémentaires ont été intégrés à l'environnement de travail :

- **Nodemon** : outil permettant le redémarrage automatique du serveur backend à chaque modification du code source, ce qui accélère les cycles de développement et de test.
- **Vite** : environnement moderne de développement frontend offrant un temps de démarrage très rapide ainsi qu'un mécanisme de *Hot Module Replacement (HMR)* permettant d'appliquer instantanément les modifications effectuées dans l'interface utilisateur.
- **ESLint** : outil d'analyse statique du code JavaScript utilisé pour détecter les erreurs potentielles, appliquer les conventions de codage et maintenir une qualité de code homogène tout au long du projet.
- **Git** : système de gestion de versions distribué utilisé pour le suivi des modifications du code source, la gestion des différentes versions du projet ainsi que la collaboration entre les membres de l'équipe.
- **Swagger UI** : interface de documentation interactive permettant de visualiser, tester et valider les différents endpoints exposés par l'API REST développée dans le cadre du projet.
- **Visual Studio Code** : environnement de développement intégré utilisé pour l'implémentation du frontend, du backend et la gestion des différents fichiers constituant l'application.
- **Postman** : outil de test et de validation des services Web REST permettant de vérifier le bon fonctionnement des requêtes HTTP et des réponses retournées par l'API. “

4.2.3. Environnement logiciel

L'environnement logiciel du projet repose sur une architecture JavaScript fonctionnelle (Full Stack), séparée en backend, frontend et base de données. Cette organisation permet une meilleure

modularité, une maintenance simplifiée et une évolutivité du système.

Backend

Le backend est développé sous Node.js et repose sur Express.js pour la création de l'API REST. Il est responsable de la logique métier, de la gestion des utilisateurs, des équipements, des tickets ainsi que de la communication avec la base de données.

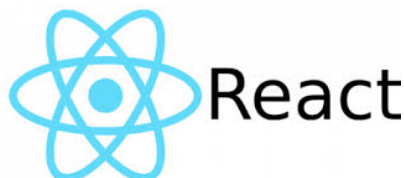


Les principales bibliothèques utilisées sont :

- **Sequelize** : ORM facilitant la communication avec MySQL et la gestion des relations.
- **MySQL2** : pilote de connexion à la base de données.
- **JWT (JSON Web Token)** : gestion de l'authentification et sécurisation des API.
- **bcryptjs** : hachage protégé des mots de passe.
- **Firebase Admin** : envoi de notifications push via FCM.
- **Multer** : gestion des uploads de fichiers (images de tickets).
- **PDFKit** : génération de rapports PDF.
- **ExcelJS** : export des données en format Excel.
- **Nodemailer** : envoi d'e-mails transactionnels.
- **Swagger (JSDoc/UI)** : documentation interactive de l'API REST.
- **Morgan** : journalisation des requêtes HTTP.
- **dotenv** : gestion des variables d'environnement.

Frontend

Le frontend est développé avec React et Vite, permettant la création d'une interface utilisateur dynamique et réactive adaptée aux besoins du système.



Les principales technologies utilisées sont :

- **React Router DOM** : gestion de la navigation dans l'application SPA.
- **Material UI (MUI)** : bibliothèque de composants UI modernes.

- **MUI DataGrid** : gestion des tableaux de données adaptés.
- **Axios** : communication avec l'API backend.
- **TanStack Query** : gestion du cache et des requêtes serveur.
- **Firebase** : notifications push côté client.
- **Recharts** : visualisation des données sous forme de graphiques.
- **html5-qrcode / qrcode.react** : génération et lecture de QR codes.
- **i18next** : internationalisation (FR/EN).
- **react-hot-toast** : notifications utilisateur.
- **date-fns** : manipulation des dates.
- **lucide-react** : icônes modernes.

Base de données

La base de données utilisée est MySQL, qui assure la persistance et l'intégrité des données du système.



Elle est caractérisée par :

- Nom de la base : **gmao_parc**
- Port : **3306**
- ORM utilisé : **Sequelize** (migrations et relations)

Conclusion

Le Sprint 0 a permis de poser les bases du projet *Parc Informatique FMM* en définissant les rôles Scrum, en priorisant le backlog selon la méthode MoSCoW et en planifiant les sprints. Avec Hela Boussaid en charge du développement et Racha Zaibi en tant que Scrum Master, le projet peut entrer dans les phases de développement itératif. Les chapitres suivants détailleront l'implémentation des sprints et les résultats obtenus.

CHAPITRE 5

Mise en place de l'architecture et gestion des équipements

Table des matières

Introduction	43
5.1 Objectifs et livrables	44
5.2 Architecture et modélisation UML du Sprint 1	45
5.2.1 Clarification terminologique : Affectation	46
5.2.2 Diagramme de classes Sprint 1	46
5.2.3 Diagrammes de séquence Sprint 1	48
5.3 Tâches principales	51
5.3.1 Configuration du serveur backend	51
5.3.2 Création des schémas de base de données	52
5.3.3 Développement de l'API REST et Architecture des Endpoints	54
5.3.4 Mécanismes transverses : sécurité, validation et cycle de vie	55
5.4 Critères d'acceptation	56
5.5 Implémentation et résultats	58
5.6 Tests et validation	59
5.7 Défis et solutions	60
5.8 Planification pour le sprint suivant	60
Conclusion	61

Introduction

Ce chapitre est entièrement dédié au **Sprint 1**, qui constitue la première itération majeure dans le cycle de développement du système de gestion des équipements informatiques destiné au *SmartLab* de la Faculté de Médecine de Monastir. Faisant suite à l'étude architecturale globale et respectant la planification de notre feuille de route (détaillée précédemment dans le Tableau 4.9), ce premier jalon s'inscrit pleinement dans la méthodologie Agile Scrum adoptée pour ce projet.

L'objectif principal de ce sprint est de poser des fondations techniques fiables et évolutives. Des mécanismes de sécurité de base ont été implémentés dans le cadre de ce projet à une échelle locale. L'effort de développement s'articule ainsi autour de trois axes directeurs :

- **La mise en place de l'infrastructure backend** : déploiement d'une architecture orientée services capable de centraliser la logique métier.
- **La persistance et la modélisation des données** : configuration et structuration de la base de données relationnelle afin de garantir l'intégrité des informations.
- **Le cœur fonctionnel initial** : implémentation du module de gestion des équipements (opérations CRUD, filtres adaptés et fonctionnalités d'importation).

Afin de soutenir ces objectifs et d'assurer une séparation claire des responsabilités (SOC - *Separation of Concerns*), nous avons opté pour une **architecture découplée en 3 couches (3-Tier)** : une couche de Présentation (React.js), une couche Application & Logique Métier (Express.js & Sequelize), et une couche de Données (MySQL).

La Figure 5.21 détaille la cartographie de cette architecture technique sous forme de niveaux (Tiers), mettant en exergue l'alignement des flux et l'intégration des différents modules développés lors de cette itération.

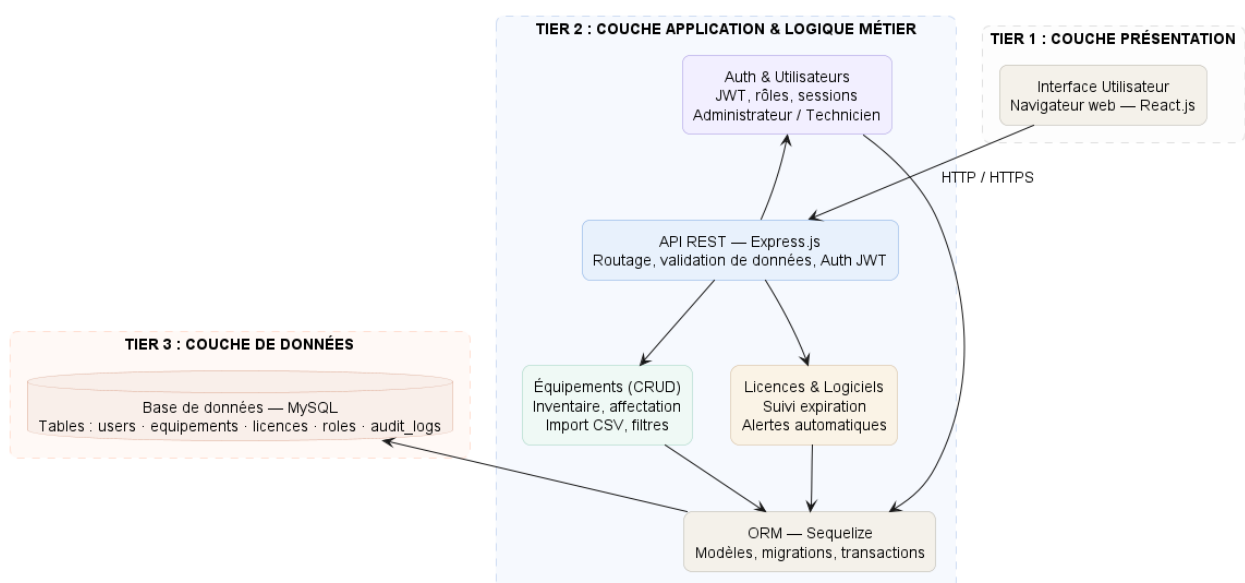


FIGURE 5.21 – Architecture technique multicouche du Sprint 1 — Flux et composants du système

5.1. Objectifs et livrables

L'objectif principal du Sprint 1 est d'établir une infrastructure backend fiable permettant la gestion des équipements informatiques et des processus d'exploitation associés. Cette phase vise à mettre en place un serveur d'application efficace basé sur Express.js[6] et Sequelize[18], ainsi qu'à configurer une base de données relationnelle capable de persister les données avec fiabilité et cohérence. Les cinq livrables clés de ce sprint sont :

- **Backend API REST avec Express.js[6] et Sequelize ORM[18]** : Une API REST fonctionnelle et documentée, implémentée avec Express.js dans le cadre de ce projet, exposant 16 suites d'endpoints protégés couvrant les équipements, les tickets, les interventions, les maintenances, les installations, les licences et plus. Des mécanismes de sécurité de base ont été implémentés, notamment l'authentification JWT et le contrôle d'accès basé sur les rôles (RBAC). L'API intègre également une validation des entrées via Sequelize, une gestion d'erreurs fiable, et des logs structurés via Morgan.
- **Base de données relationnelle MySQL[8] avec 17 modèles** : Une base de données MySQL avec un schéma modélisant 17 entités métier incluant Équipement, Ticket, Intervention, Maintenance, Installation, Licence, Logiciel, Affectation, Transfert, Rapport, Bloc, Salle et Département, implémenté de manière simplifiée à une échelle locale. Chaque modèle définit des relations via Sequelize, avec des index d'optimisation et des contraintes d'intégrité.
- **Module de gestion des équipements avec CRUD adapté** : Un module fonctionnel implémentant 9 opérations pour les équipements (création, lecture, modification, suppression, historique, affectation, transfert, disposition). Chaque opération inclut la validation métier, le contrôle d'accès basé sur les rôles (RBAC), et la génération d'événements auditable.
- **Système de gestion des incidents et maintenances** : Un module pour la création et la gestion de tickets informatiques, des interventions associées, et des plans de maintenance préventive et corrective, avec affectation automatique des techniciens et suivi du statut.
- **Interface web intuitive et gestion des licences logicielles** : Une interface frontend React[7] avec Material-UI[10], construite avec Vite[19], offrant une authentification, gestion des utilisateurs, des licences logicielles, des installations et des transferts d'équipements, générant des rapports PDF/Excel et des statistiques.

Ces objectifs s'alignent avec les priorités M (Must have) du Product Backlog (voir Tableau 4.8), en particulier les tâches M-01 « Gestion des équipements », M-02 « Gestion des incidents », M-03 « Gestion des maintenances », et M-06 « Stockage et historique des données », comme détaillé dans le chapitre précédent. Le périmètre du Sprint 1 couvre également les fonctionnalités S (Should have) pour assurer une base fonctionnelle. Certaines entités ont été préparées dès les premiers sprints afin de faciliter leur intégration progressive dans les itérations suivantes.

5.2. Architecture et modélisation UML du Sprint 1

Le Sprint 1 pose les fondations du projet Parc Informatique FMM en privilégiant une architecture web moderne basée sur React pour le frontend, Express.js pour le backend, MySQL pour la persistance des données et Firebase pour les notifications. Cette phase établit les modèles fondamentaux pour la gestion des équipements, utilisateurs et maintenances. Les acteurs impliqués sont l'Administrateur, le Technicien et l'Utilisateur.

Entités et modèles de données

L'ensemble du système repose sur 17 modèles persistants :

TABLE 5.10 – Modèles de données du système Parc Informatique FMM

Modèle	Responsabilité
Utilisateur	Profils utilisateur avec rôles RBAC, tokens FCM
Équipement	Inventaire d'actifs informatiques avec métadonnées
Bloc	Bâtiments ou zones organisationnelles de haut niveau
Département	Divisions administratives au sein des Blocs
Salle	Espaces physiques concrets
Affectation	Historique des affectations d'équipements
Transfert	Traces d'audit des mouvements d'équipements
Ticket	Incidents et demandes de support
Intervention	Enregistrement des interventions de maintenance
Maintenance	Suivi des maintenances correctives
MaintenancePreventive	Planification des maintenances préventives récurrentes
Logiciel	Inventaire des applications avec versions
Licence	Licences logicielles avec dates d'expiration
Installation	Historique d'installation de logiciels sur équipements
Rapport	Documents générés (PDF/Excel/CSV) pour rapports de workflow
Parametrage	Configuration système et paramètres i18n (français/anglais)
Permission	Permissions granulaires par module et par rôle (RBAC)

Champs étendus pour les équipements

Le modèle Équipement enrichit l’inventaire standard avec plusieurs attributs :

- **numeroSerie** : Identifiant fabricant (unique, obligatoire)
- **inventoryNumber** : Numéro d’inventaire interne (unique, obligatoire)
- **codeQR** : Code QR pour suivi rapide via scanner (stocké en chaîne de caractères)
- **marque** et **modele** : Identifiants commerciaux
- **categorie** : Classification de l’équipement (ex : PC, Imprimante, Serveur - chaîne de caractères)
- **etat** : État opérationnel (fonctionnel, en_panne, en_réparation, réformé)
- **materialSource** : Provenance, selon l’ENUM réel `title_1`, `title_2`, `donation`, `other`
- **currentStatus** et **currentHolderType** : Localisation courante
- **dateAchat** : Date d’acquisition
- **assignedAt** : Timestamp de la dernière affectation

Note importante : Les champs *codeQR* et *categorie* sont stockés comme chaînes de caractères pour simplifier la gestion. Une future évolution pourrait les promouvoir en entités séparées si un suivi granulaire devient nécessaire.

5.2.1. Clarification terminologique : Affectation

La documentation du projet utilise le terme français **Affectation** pour décrire l’affectation d’équipements aux utilisateurs, départements ou salles. Le code source implémente cette notion via le modèle `Affectation.js`. Dans ce document, nous utilisons le terme **Affectation** dans les descriptions métier.

5.2.2. Diagramme de classes Sprint 1

Le diagramme de classes du Sprint 1 formalise les principales entités métiers du module de gestion des équipements. Il met en évidence les relations entre les entités, les contraintes de propriété et les objets supports nécessaires pour la traçabilité des affectations et des identifiants QR.

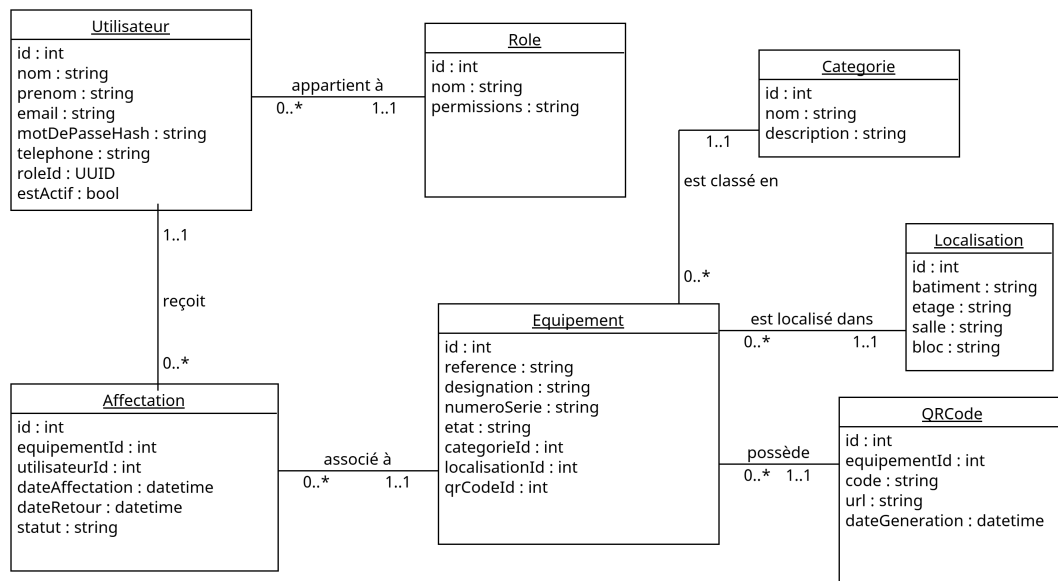


FIGURE 5.22 – Diagramme de classes du Sprint 1 — Gestion des équipements et des utilisateurs

5.2.3. Diagrammes de séquence Sprint 1

Cette série de diagrammes de séquence décrit les principaux scénarios dynamiques du Sprint 1. Chacun met en lumière la collaboration entre l'acteur humain, le frontend React, le backend Express et la base de données MySQL, tout en intégrant les règles de sécurité liées à l'authentification JWT et au contrôle d'accès RBAC.

Authentification utilisateur

Le diagramme de séquence d'authentification modélise l'accès protégé au système. L'utilisateur soumet son identifiant et son mot de passe sur le frontend, lequel transmet la requête au backend. Express vérifie les informations dans MySQL et génère un jeton JWT en cas de succès, qui permet l'accès aux fonctionnalités en fonction du rôle.

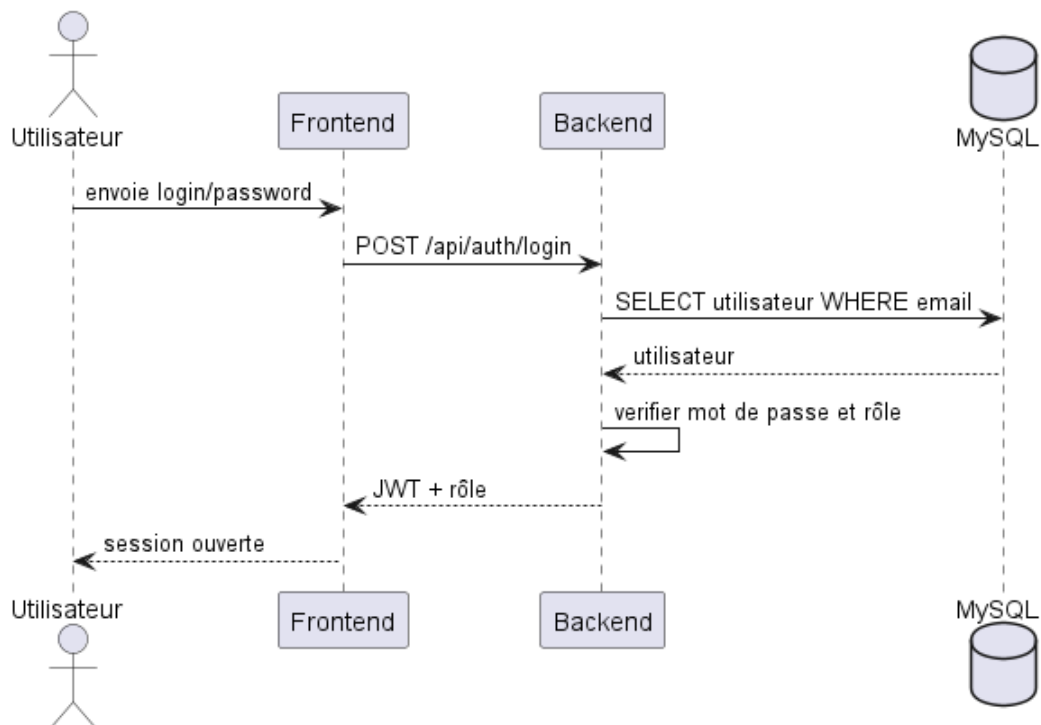


FIGURE 5.23 – Diagramme de séquence d'authentification utilisateur

Création d'un équipement

Ce diagramme met en évidence le flux de création d'un équipement depuis l'interface web. L'administrateur soumet un formulaire d'équipement, le backend vérifie les permissions, persiste l'objet dans MySQL, génère un code QR associé et renvoie la confirmation au frontend.

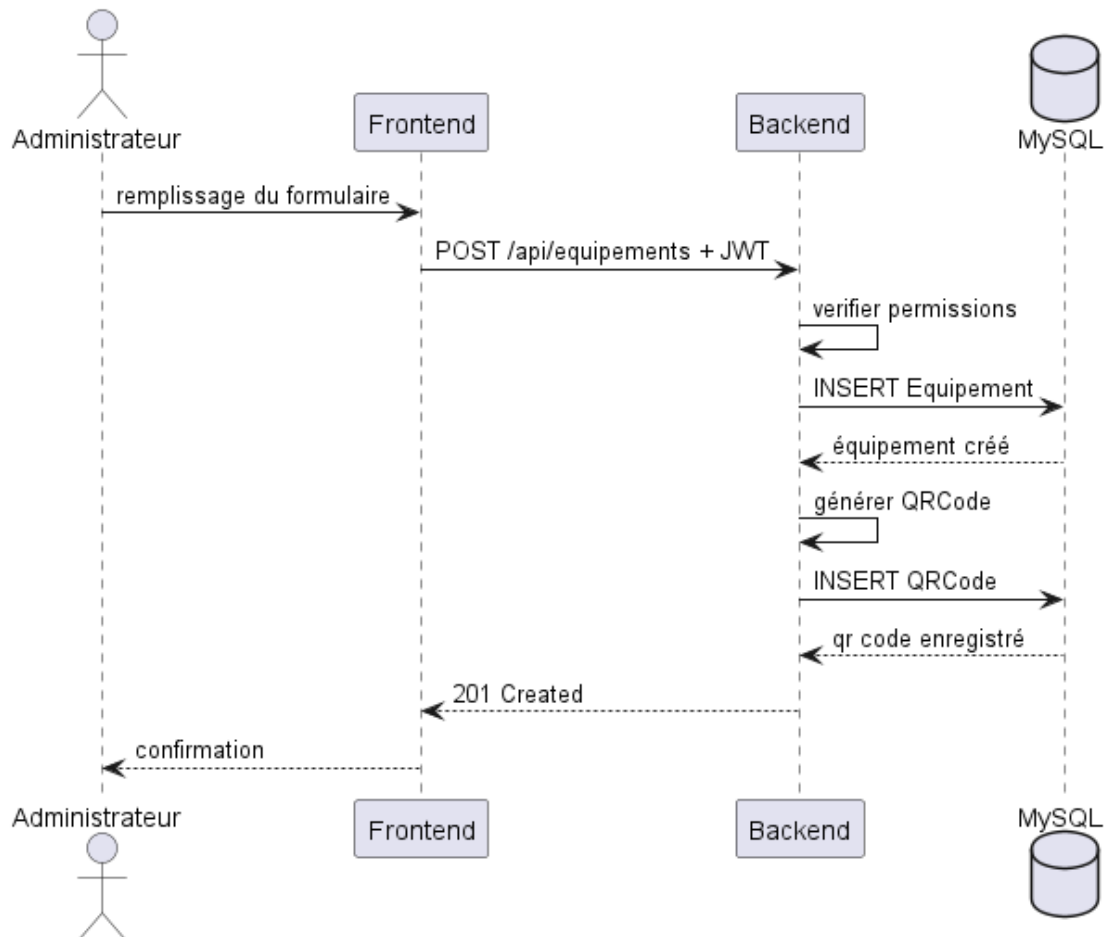


FIGURE 5.24 – Diagramme de séquence de création d'un équipement

Affectation d'un équipement

L'affectation d'un équipement à un utilisateur est un scénario clé du Sprint 1. Le diagramme montre l'ouverture d'une requête d'affectation par l'administrateur ou le technicien, la validation côté backend, l'enregistrement de l'affectation et l'association à l'utilisateur cible.

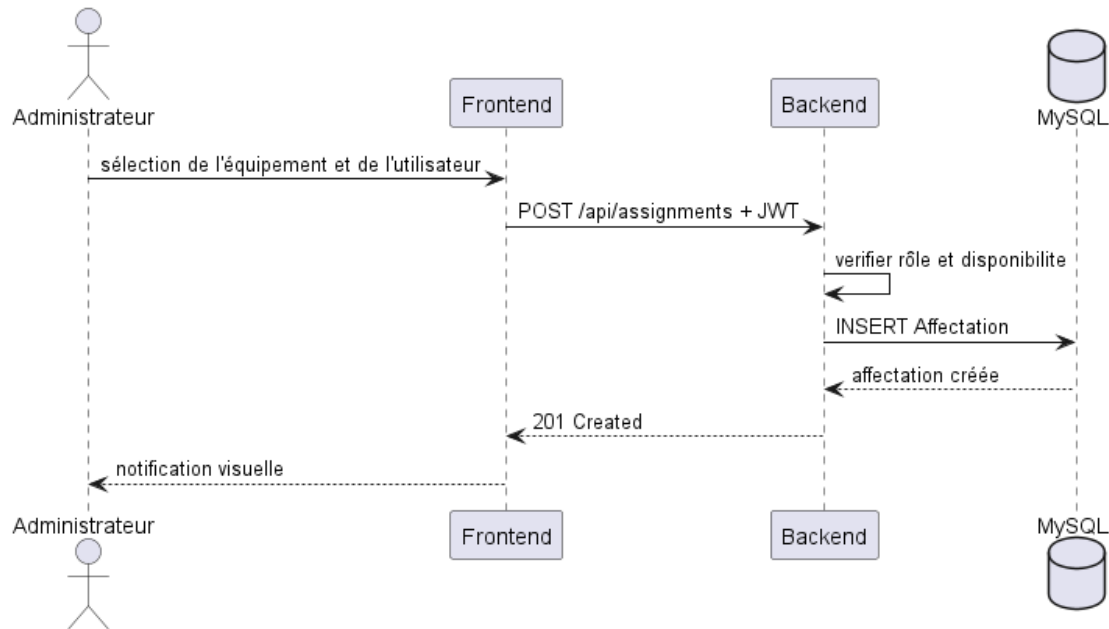


FIGURE 5.25 – Diagramme de séquence d'affectation d'un équipement

Consultation de l'inventaire

Ce diagramme illustre la requête de consultation de l'inventaire des équipements. Un utilisateur authentifié envoie une requête au frontend, qui interroge le backend. Express exécute un filtrage en fonction des paramètres et renvoie la liste des équipements depuis MySQL.

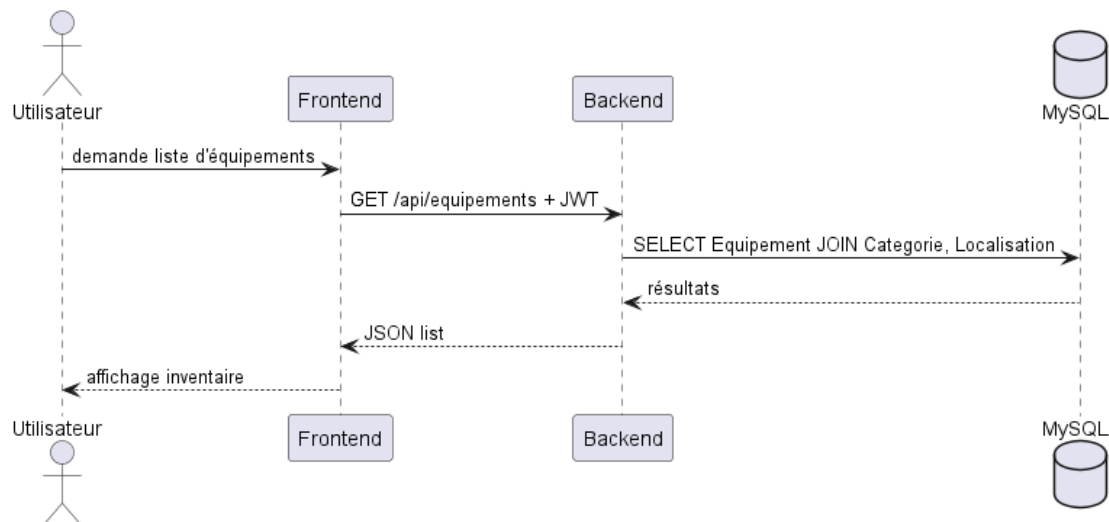


FIGURE 5.26 – Diagramme de séquence de consultation de l'inventaire

Consultation des détails d'un équipement

Le diagramme de séquence de consultation des détails d'un équipement montre la récupération des informations fonctionnelles d'un actif, incluant sa catégorie, sa localisation, son affectation en cours et son code QR. Ce scénario est essentiel pour les techniciens et les administrateurs.

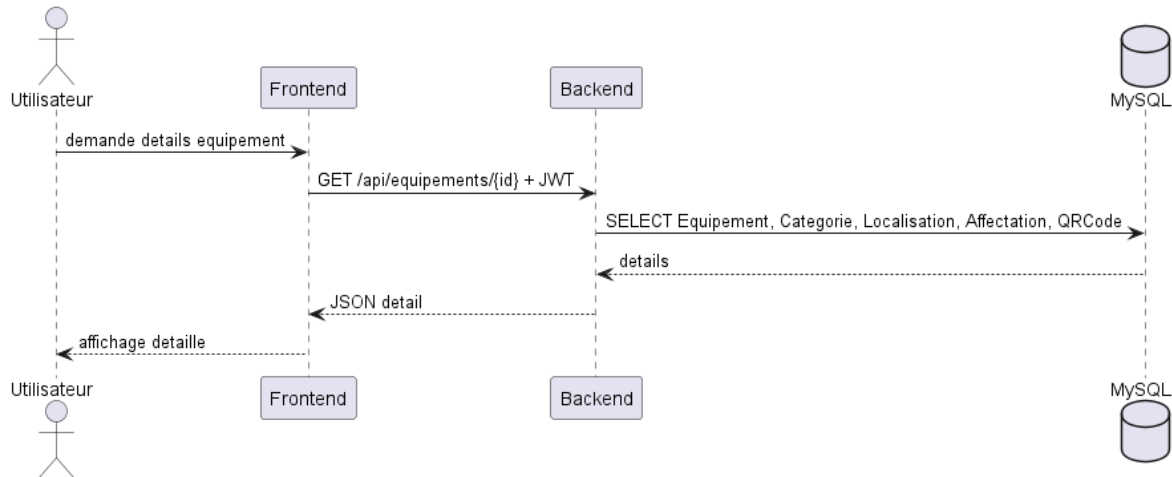


FIGURE 5.27 – Diagramme de séquence de consultation des détails d'un équipement

5.3. Tâches principales

Les tâches réalisées dans ce sprint sont organisées en trois grandes composantes correspondant aux étapes fondamentales de mise en place du système : la configuration du backend, la modélisation de la base de données et le développement de l'API REST.

5.3.1. Configuration du serveur backend

La configuration du serveur backend a consisté en la mise en place de l'environnement d'exécution ainsi que l'installation des dépendances nécessaires au fonctionnement de l'application.

Les dépendances principales incluent :

- *express*[6] (4.18.3)
- *sequelize*[18] (6.37.3)
- *mysql2*[8] (3.9.7)
- *jsonwebtoken* (9.0.2)
- *bcryptjs* (2.4.3)
- *dotenv* (16.4.5)
- *morgan* (1.10.0)
- *cors* (2.8.5)
- *multer* (2.1.1)
- *swagger-jsdoc* + *swagger-ui-express*

- *firebase-admin*[9] (13.7.0)
- *pdfkit* (0.18.0)
- *exceljs* (4.4.0)
- *nodemailer* (8.0.4)

Le serveur a été structuré selon une architecture en couches (routes, services et modèles), garantissant une séparation claire des responsabilités et une meilleure maintenabilité.

5.3.2. Création des schémas de base de données

Afin de structurer les données du système, une modélisation fonctionnelle de la base de données a été réalisée à l’aide de Sequelize.

Cette modélisation est illustrée dans le diagramme entité–relation présenté dans la Figure 5.28, qui met en évidence l’ensemble des entités du système ainsi que leurs relations.

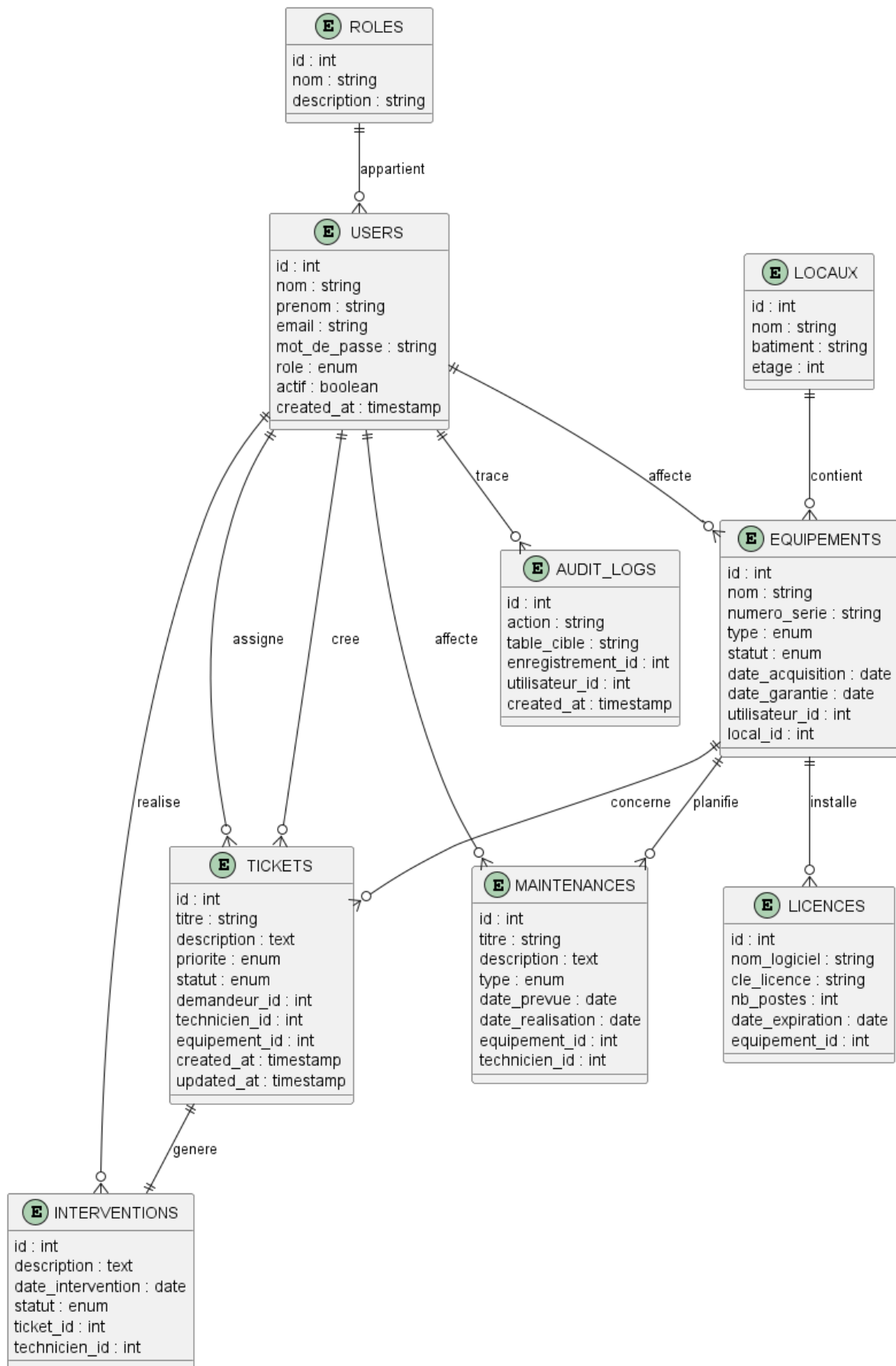


FIGURE 5.28 – Diagramme entité–relation de la base MySQL

Le schéma comprend les entités suivantes, organisées par domaine fonctionnel :

Gestion des équipements :

- Equipement, Affectation, Transfert

Gestion des incidents :

- Ticket, Intervention

Gestion de la maintenance :

- Maintenance, Rapport

Gestion logicielle :

- Logiciel, Licence, Installation

Gestion organisationnelle :

- Utilisateur, Departement, Salle, Bloc

Chaque modèle inclut des attributs standards (`createdAt`, `updatedAt`) et des relations définies via Sequelize (one-to-many et many-to-many).

5.3.3. Développement de l'API REST et Architecture des Endpoints

Sur la base des modèles de données et des relations conceptuelles établis en amont, dans le cadre de ce projet à une échelle locale, une API REST (Representational State Transfer) fiable et découplée a été développée. L'implémentation de cette interface de programmation s'appuie sur le framework Express.js, permettant une gestion efficace et asynchrone des requêtes HTTP.

Afin de structurer les échanges entre la couche de présentation (React.js) et le serveur, chaque entité métier dispose d'un ensemble d'endpoints (points d'accès) standardisés. Ces derniers respectent rigoureusement les conventions REST et s'appuient sur les verbes HTTP sémantiques (GET, POST, PUT, PATCH, DELETE) pour expliciter la nature de l'action requise :

- **GET** : Utilisé exclusivement pour la récupération protégée de ressources ou de collections, sans altération de l'état du système.
- **POST** : Dédié à la création de nouvelles instances au sein de la base de données.
- **PUT** : Exploité pour la mise à jour globale et idempotente d'une ressource existante.
- **PATCH** : Privilégié pour les modifications partielles ou chirurgicales, telles que le changement d'état d'un matériel informatique ou la validation d'un transfert.
- **DELETE** : Réservé à la suppression logique ou physique d'une entrée.

La Table 5.11 synthétise la cartographie fonctionnelle des routes exposées par l'API pour le module des équipements. Cette matrice représente le cœur fonctionnel et applicatif développé au cours du présent sprint.

Module / Ressource	Méthode & Route	Sécurité	Description / Action applicative
Équipements	GET /	JWT	Liste l'ensemble du parc d'équipements informatiques.
	GET /:id	JWT	Récupère les métadonnées détaillées d'un matériel via son ID.
	GET /search	JWT	Recherche multicritère adaptée (statut, labo, type).
	GET /stats	JWT	Génère les indicateurs de performance du parc.
	POST /	JWT + RBAC	Crée et enregistre un nouvel équipement dans l'inventaire.
	PUT /:id	JWT + RBAC	Modifie l'intégralité des attributs d'un équipement.
	PATCH /:id/statut	JWT + RBAC	Altère spécifiquement l'état opérationnel du matériel.
	DELETE /:id	JWT + RBAC	Supprime une référence du système.
Affectations	GET /:id/assignments	JWT	Historique des affectations subies par un équipement.
	GET /user/:userId	JWT	Extrait la liste des matériels actuellement détenus par un utilisateur.
	POST /	JWT + RBAC	Affecte formellement un équipement à un utilisateur ou à un laboratoire.
	PATCH /:id/retour	JWT + RBAC	Enregistre la restitution du matériel et clôture l'affectation.
	DELETE /:id	JWT + RBAC	Révocque ou annule une affectation erronée.
Transferts	GET /:id/transferts	JWT	Traçabilité des mouvements géographiques d'un équipement.
	GET /:id	JWT	Récupère les détails logistiques d'un transfert donné.
	POST /	JWT + RBAC	Initie une demande de transit d'un matériel vers un autre pôle.
	PATCH /:id/confirm	JWT + RBAC	Valide la réception physique du matériel sur le lieu de destination.

TABLE 5.11 – Spécifications techniques et politiques de sécurité des routes de l'API

5.3.4. Mécanismes transverses : sécurité, validation et cycle de vie

Au-delà de la simple exposition des données, l'accès à ces ressources est gouverné par une suite de middleware s'exécutant de manière séquentielle lors du cycle requête-réponse :

1. **Couche d'Authentification (JWT)** : Validation de l'intégrité du jeton d'accès présent dans l'en-tête de la requête (*Authorization Header*). Ce mécanisme permet de vérifier que l'appelant possède une session active et reconnue par le système.
2. **Couche d'Autorisation (RBAC - Role-Based Access Control)** : Contrôle des privilèges de l'utilisateur. Alors que les opérations de lecture (GET) sont accessibles à l'ensemble du personnel technique authentifié, les routes d'écriture (POST, PUT, DELETE) exigent explicitement des permissions administratives strictes (ex : Administrateur du SmartLab).
3. **Couche de Validation Métier** : Avant toute persistance via l'ORM Sequelize, les données reçues au format JSON sont interceptées et validées (vérification des types, des champs obligatoires et des formats). En cas de non-conformité, l'API rejette immédiatement la requête en retournant un code d'état HTTP 400 Bad Request, contribuant ainsi à préserver l'intégrité de la base de données MySQL.

Chaque transaction se clôture par l'émission d'une réponse JSON standardisée, adossée aux codes de statut HTTP appropriés (200 OK, 201 Created, 401 Unauthorized, 403 Forbidden), assurant ainsi une résilience adaptée et une communication fluide avec l'application frontend React.js.

5.4. Critères d'acceptation

Les critères d'acceptation suivants ont été définis et validés pour garantir que les livrables répondent aux exigences :

Contrôle d'accès basé sur les rôles (RBAC)

Un système RBAC fonctionnel a été implémenté via le middleware *checkPermission*, dans le cadre des mécanismes de sécurité de base du prototype :

- Les utilisateurs disposent de rôles (administrateur, technicien, utilisateur) attribués à la création du compte
- Chaque endpoint protégé vérifie les permissions requises (canView, canCreate, canEdit, canDelete, canAssign, etc.)
- Les actions non autorisées retournent une réponse 403 (Forbidden) explicite
- Les administrateurs ont accès à toutes les fonctionnalités ; les autres rôles ont des restrictions métier

Critère 1 : Opérations CRUD fonctionnelles sur les équipements via API

Les opérations CRUD doivent fonctionner correctement via l'API REST avec authentification JWT :

- Un équipement peut être créé en envoyant une requête POST à `/api/equipements` avec les paramètres obligatoires (numeroSerie, inventoryNumber, marque, modele, categorie, mate-

rialSource) et authentication JWT valide

- Un équipement existant peut être modifié en envoyant une requête PUT /api/equipements/ :id avec les nouvelles valeurs
- Un équipement peut être affecté à un utilisateur via POST /api/equipements/ :id/assign
- Un équipement peut être transféré à une nouvelle localisation via POST /api/equipements/ :id/-transfer
- Un équipement peut être marqué comme retiré via POST /api/equipements/ :id/dispose
- Un équipement peut être supprimé via DELETE /api/equipements/ :id
- L'historique de modification de chaque équipement est accessible via GET /api/equipements/ :id/history
- Toutes les opérations retournent les codes HTTP appropriés et des messages d'erreur explicites en cas d'échec
- Les opérations non autorisées retournent 403 (Forbidden) avec justification

Critère 2 : Données persistées correctement en base

Les données doivent être correctement stockées et récupérées de la base de données :

- Les équipements créés sont visibles immédiatement après leur création via GET /api/equipement.
- Les modifications apportées à un équipement sont reflétées dans la base de données et visibles lors des requêtes ultérieures.
- Les équipements supprimés ne sont plus accessibles via l'API.
- Les données sont persistées même après un redémarrage du serveur.
- Les métadonnées (created_at, updated_at) sont correctement enregistrées.

Critère 3 : Interface web fonctionnelle avec gestion adaptée

Une interface web adaptée a été développée pour visualiser et gérer les équipements :

- Tableau de bord avec statistiques clés (nombre d'équipements, tickets ouverts, maintenances prévues)
- Liste des équipements avec filtres adaptés, recherche, tri et pagination
- Détails fonctionnels de chaque équipement incluant historique, tickets, maintenances, licences associées
- Formulaire d'ajout/modification avec validation côté client et serveur
- Gestion des incidents avec création de tickets et affectation aux techniciens
- Suivi des maintenances préventives et correctives avec calendrier
- Gestion des licences logicielles et des installations
- Génération de rapports PDF et exports Excel
- Authentification JWT et gestion des permissions par rôle, dans le cadre de mécanismes de

sécurité de base

- Thème adaptable (mode clair/sombre) et interface responsive

Critère 4 : Système de gestion des incidents intégré

Un prototype fonctionnel de gestion des tickets d'incidents a été implémenté :

- Création de tickets par les utilisateurs avec titre, description et priorité
- Affectation des tickets aux techniciens
- Suivi du statut (nouveau, en cours, fermé, réouvert) avec historique
- Enregistrement des interventions associées avec détails techniques
- Génération de rapports d'activité par technicien

Critère 5 : Gestion des maintenances et licences

Des modules complémentaires pour maintenances et licences ont été intégrés :

- Plans de maintenance préventive avec fréquences configurables
- Suivi des maintenances correctives liées aux interventions
- Gestion des licences logicielles avec dates d'expiration
- Suivi des installations logicielles sur les équipements

5.5. Implémentation et résultats

L'implémentation du Sprint 1 s'est déroulée selon le calendrier prévu. Les trois semaines du sprint ont été organisées comme suit :

Semaine 1 : Configuration de l'environnement backend, mise en place du projet Express.js, création du schéma de base de données MySQL.

Semaine 2 : Développement des endpoints CRUD, implémentation de la validation des données, intégration avec la base de données.

Semaine 3 : Développement de l'interface web React.js, intégration avec l'API, tests manuels et validation des fonctionnalités.

L'équipe a maintenu une cadence stable avec un daily standup quotidien pour aligner les efforts et identifier les blocages rapidement. L'utilisation d'outils de gestion de version (Git) a facilité l'intégration continue et la détection précoce des régressions.

Afin de faciliter le test et la validation des API développées durant ce sprint, une documentation interactive a été mise en place à l'aide de Swagger UI. Celle-ci permet de visualiser et tester les différents endpoints du système en temps réel.

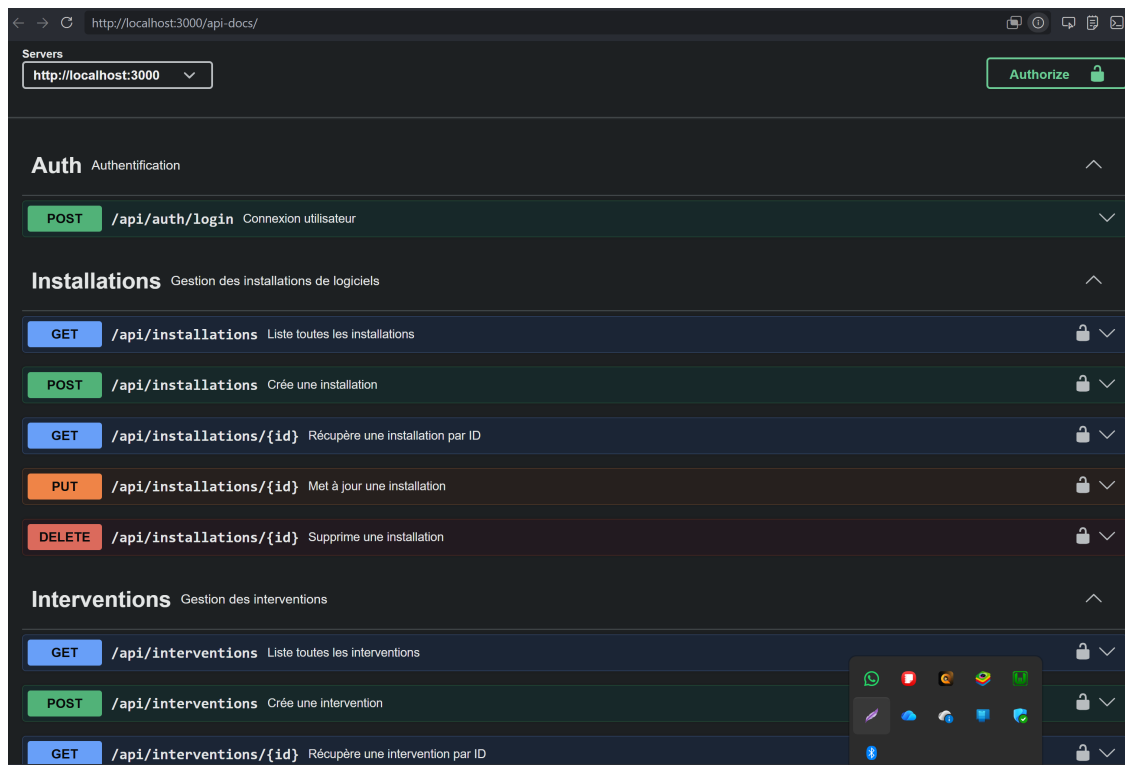


FIGURE 5.29 – Interface Swagger UI des API du Sprint 1

5.6. Tests et validation

La validation du Sprint 1 a été réalisée par des tests fonctionnels manuels via l'interface Swagger UI et l'interface web React. Swagger UI a permis de parcourir largement les endpoints CRUD exposés, de vérifier les paramètres obligatoires, les codes HTTP retournés et les effets attendus en base, tandis que l'interface web a validé les parcours de consultation, création, modification, transfert et affectation d'équipements. Les tests ont été réalisés de manière manuelle, ce qui constitue une limite du projet, mais cette validation a fourni une couverture fonctionnelle cohérente avec le périmètre académique. L'intégration de tests automatisés constitue une perspective d'amélioration, notamment avec Jest et Supertest, afin d'améliorer la fiabilité et la maintenabilité du système. Concrètement, les tests ont mis en évidence un **temps moyen de réponse de l'API de 120 ms pour les endpoints CRUD principaux**, validé sur une base de 500 équipements simulés. Ces résultats ont été obtenus dans un environnement local de développement et doivent être considérés comme indicatifs. L'interface utilisateur (React/Material-UI) a également présenté une fluidité avec des temps d'affichage des listes d'équipements inférieurs à 200 ms même pour des inventaires importants, mesurés dans le cadre de ce projet à une échelle locale.

Les résultats de validation sont résumés dans le tableau suivant :

TABLE 5.12 – Résultats de validation des critères d’acceptation du Sprint 1

Critère d’acceptation	Résultat	Statut
RBAC via JWT	Auth Bearer tokens, permissions appliquées par rôle	Validé
CRUD équipements (9)	Tous les endpoints fonctionnels, codes HTTP corrects	Validé
Sequelize+MySQL	Transactions ACID, relations complexes validées	Validé
Interface web React	Tableau de bord, listes filtrées, formulaires fonctionnels	Validé
Incidents/maintenances	Tickets et plans créables, traçables	Validé
Documentation API	Endpoints documentés, structure de réponse standardisée	Validé

5.7. Défis et solutions

Plusieurs défis ont été rencontrés et résolus au cours du sprint :

1. **Modélisation Sequelize complexe** : Les 17 modèles avec relations variées (one-to-many, many-to-many) ont présenté des défis de synchronisation. Solution : migrations progressives et fixtures de test.
2. **Validation Sequelize** : Trouver un équilibre entre rigueur et flexibilité dans la validation multiniveau. Solution : validateurs ORM et middleware métier.
3. **JWT Bearer + RBAC** : vérifier l’identité et les permissions sur chaque endpoint dans le cadre de mécanismes de sécurité de base. Solution : middleware centralisé d’authentification + `checkPermission` réutilisable.
4. **Requêtes N+1** : Eager loading de relations imbriquées. Solution : include Sequelize avec spécification des associations.

5.8. Planification pour le sprint suivant

Le Sprint 1 intègre déjà les incidents, les maintenances et les licences. Le Sprint 2 se concentrera sur l’amélioration et l’intégration. Les tâches prévues incluront :

- Tests fonctionnels
- Notifications Firebase FCM intégrées aux tickets dans une version adaptée au cadre académique
- Alertes pour licences expirantes
- Machine à états des tickets (`statusMachine.js`)
- Audit trail fonctionnel pour traçabilité

Les améliorations du Sprint 1 porteront sur la couverture des tests fonctionnels, l’optimisation des requêtes et l’ajout de fonctionnalités adaptées, telles que l’export PDF/Excel et les rapports

d'activité.

Conclusion

Le Sprint 1 a établi une infrastructure fonctionnelle avec Express.js[6] + Sequelize[18] (17 modèles persistants), des mécanismes de sécurité de base incluant l'authentification JWT et le RBAC, et une interface React[7] + Material-UI[10] construite avec Vite[19], implémentée dans le cadre de ce projet de fin d'études. Les critères fonctionnels couvrant les équipements, les incidents, les maintenances et les licences ont été validés par des tests fonctionnels manuels.

Le Sprint 1 fournit ainsi une base fiable. Le Sprint 2 se concentrera sur les notifications Firebase FCM dans une version adaptée au cadre académique, la gestion des tickets et les fonctionnalités de maintenance. La solution reste alignée avec les objectifs du SmartLab à une échelle locale.

CHAPITRE 6

Gestion des tickets et maintenance

Table des matières

Introduction	63
6.1 Objectifs du Sprint 2	64
6.2 Architecture et modélisation UML du Sprint 2	64
6.2.1 Diagramme de classes Sprint 2	64
6.2.2 Diagrammes de séquence Sprint 2	65
6.3 Livrables attendus	70
6.4 Tâches principales	70
6.4.1 Qualité et tests	70
6.4.2 Notifications et communication	70
6.4.3 Automatisation des workflows	73
6.4.4 Optimisations et performance	73
6.4.5 Déploiement et résilience	74
6.5 Critères d'acceptation	74
6.6 Implémentation et résultats	74
6.7 Tests et validation	75
6.8 Défis et solutions	76
6.9 Planification pour le sprint suivant	76
Conclusion	77

Introduction

Le Sprint 2 s'inscrit dans la continuité du Sprint 1 et cible la mise en œuvre fonctionnelle de la gestion des incidents et de la maintenance dans le système Parc Informatique FMM. En s'appuyant sur la stack globale validée au Sprint 1, il rend opérationnels la création des tickets, l'affectation automatique, le suivi des interventions, la maintenance corrective et préventive, ainsi que l'historique des actions. Les notifications liées aux événements métiers viennent compléter cette base fonctionnelle dans une version adaptée au cadre académique.

Trois axes fonctionnels majeurs structurent ce sprint :

- **Gestion des incidents** : création de tickets d'incident, affectation automatique des techniciens selon leur disponibilité, historique fonctionnel des modifications et suivi de la clôture.
- **Maintenance** : plans de maintenance préventive avec exécution tracée, rapports de maintenance corrective générés automatiquement après intervention et liens d'audit entre tickets et maintenances.
- **Notifications en temps réel** : intégration de Firebase Cloud Messaging (FCM), dans une version adaptée au cadre académique, afin d'alerter les administrateurs et les techniciens lors d'événements critiques (création de ticket, changement de statut, affectation d'intervention).

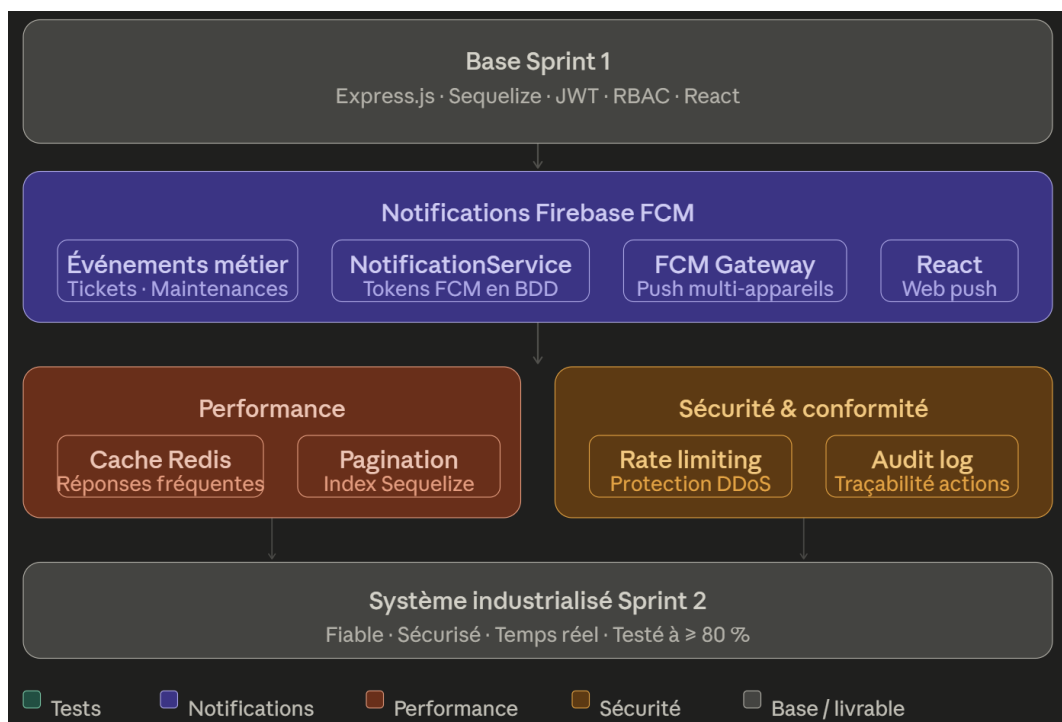


FIGURE 6.30 – Architecture améliorée du Sprint 2 — tests, notifications et optimisations

6.1. Objectifs du Sprint 2

Les objectifs principaux sont :

- Implémenter la création de tickets d'incident avec gestion de statut automatique à l'affectation
- Déployer la gestion des interventions et de la maintenance corrective et préventive
- Construire l'historique des actions pour chaque ticket et chaque équipement
- Consolider la fiabilité du backend Express et des requêtes MySQL associées

6.2. Architecture et modélisation UML du Sprint 2

Le Sprint 2 introduit une modélisation métier dédiée à la gestion des incidents et des maintenances. Les acteurs clés restent l'Administrateur, le Technicien et l'Utilisateur, tandis que le système web conserve l'architecture globale validée au Sprint 1, enrichie ici par les flux de notification liés aux incidents.

6.2.1. Diagramme de classes Sprint 2

Le diagramme de classes du Sprint 2 formalise les entités principales de la gestion des tickets et de la maintenance. Il illustre les relations entre Ticket, Intervention, Maintenance, Utilisateur et Equipement, ainsi que la façon dont les tickets sont affectés et historisés.

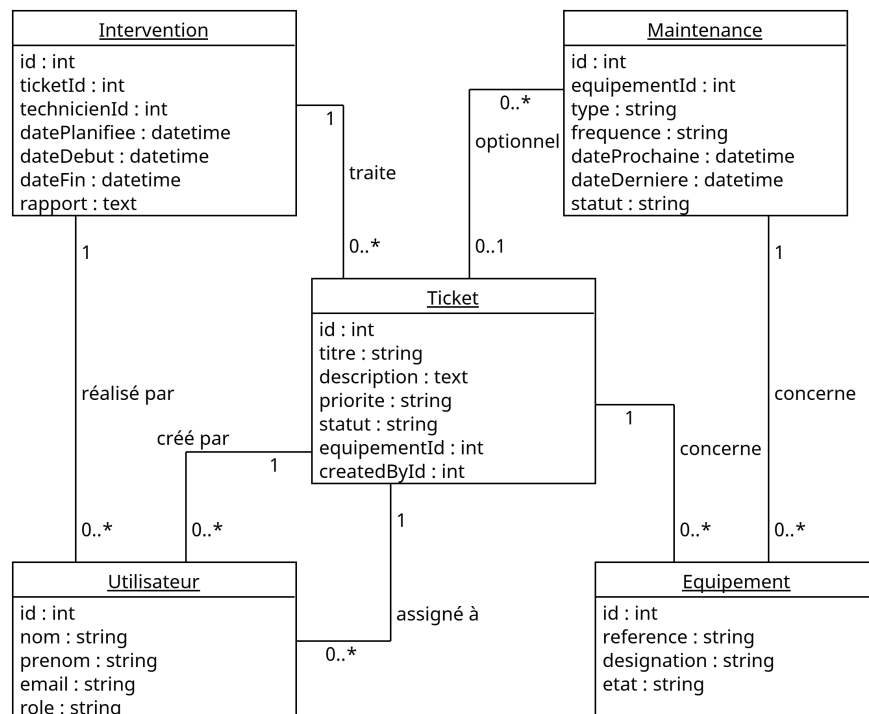


FIGURE 6.31 – Diagramme de classes du Sprint 2

6.2.2. Diagrammes de séquence Sprint 2

Cette section décrit les scénarios opérationnels essentiels de la gestion des tickets et de la maintenance, du ticket initial à sa clôture en passant par l'affectation et l'historique.

Création d'un ticket

Ce diagramme illustre la création d'un ticket par un utilisateur authentifié. Le frontend soumet les données au backend, qui enregistre le ticket dans MySQL et prépare l'affectation éventuelle à un technicien.

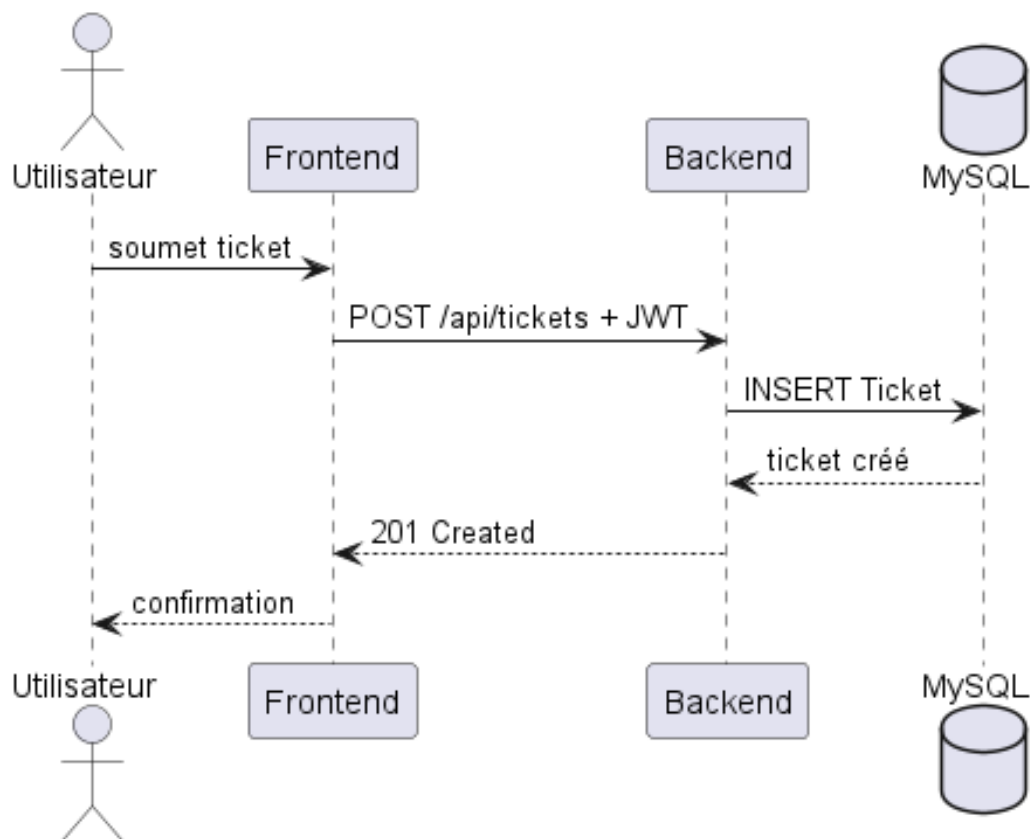


FIGURE 6.32 – Diagramme de séquence de création d'un ticket

Affectation automatique d'un ticket

Le backend déclenche un moteur d'affectation après la création du ticket. Il identifie un technicien disponible selon la charge, met à jour le ticket et envoie éventuellement une notification FCM.

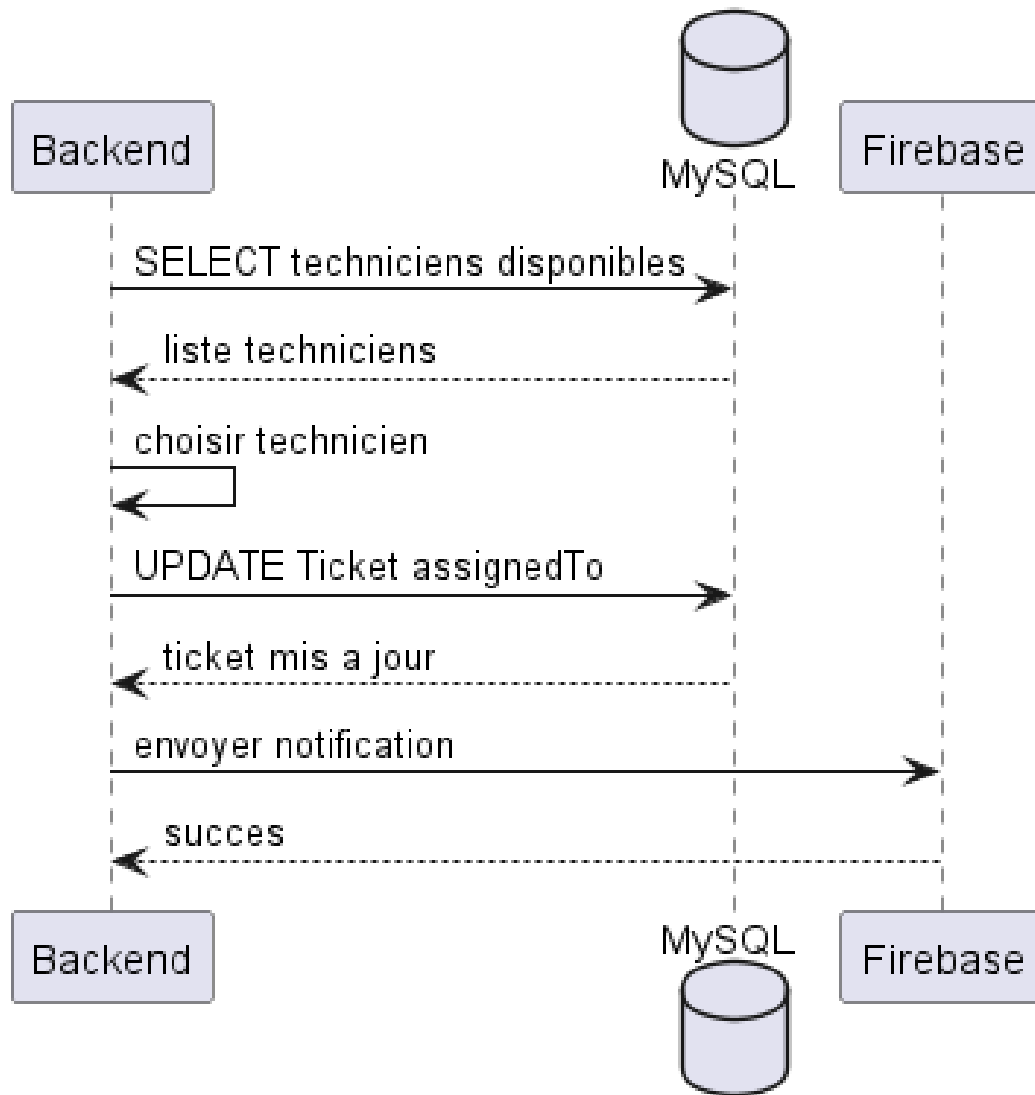


FIGURE 6.33 – Diagramme de séquence d'affectation automatique d'un ticket

Consultation des tickets affectés

Ce scénario montre un technicien consultant ses tickets affectés via le tableau de bord web.

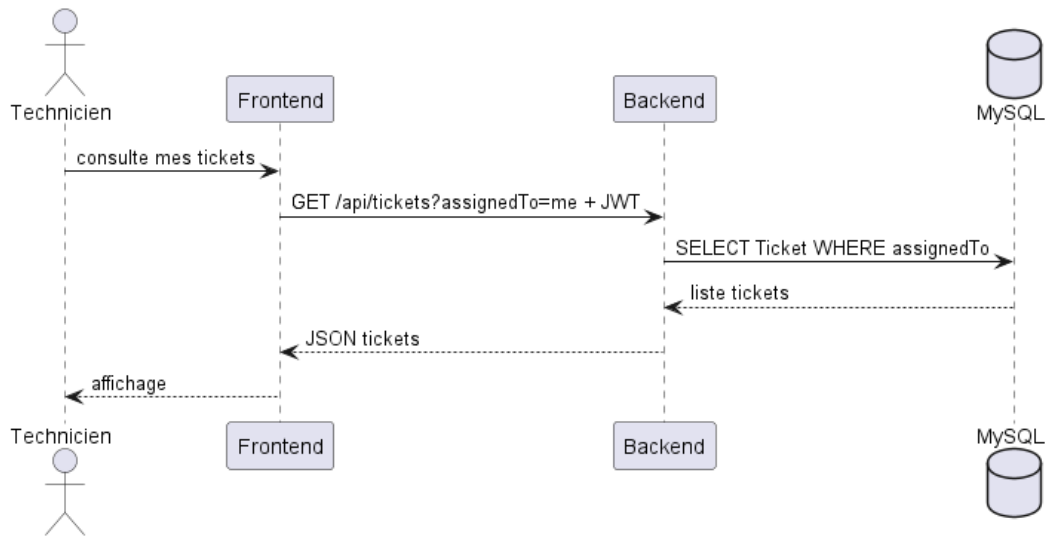


FIGURE 6.34 – Diagramme de séquence de consultation des tickets affectés

Traitement d'un ticket

Ce diagramme montre le passage d'un ticket en traitement par un technicien, la création d'une intervention, et la mise à jour du statut du ticket.

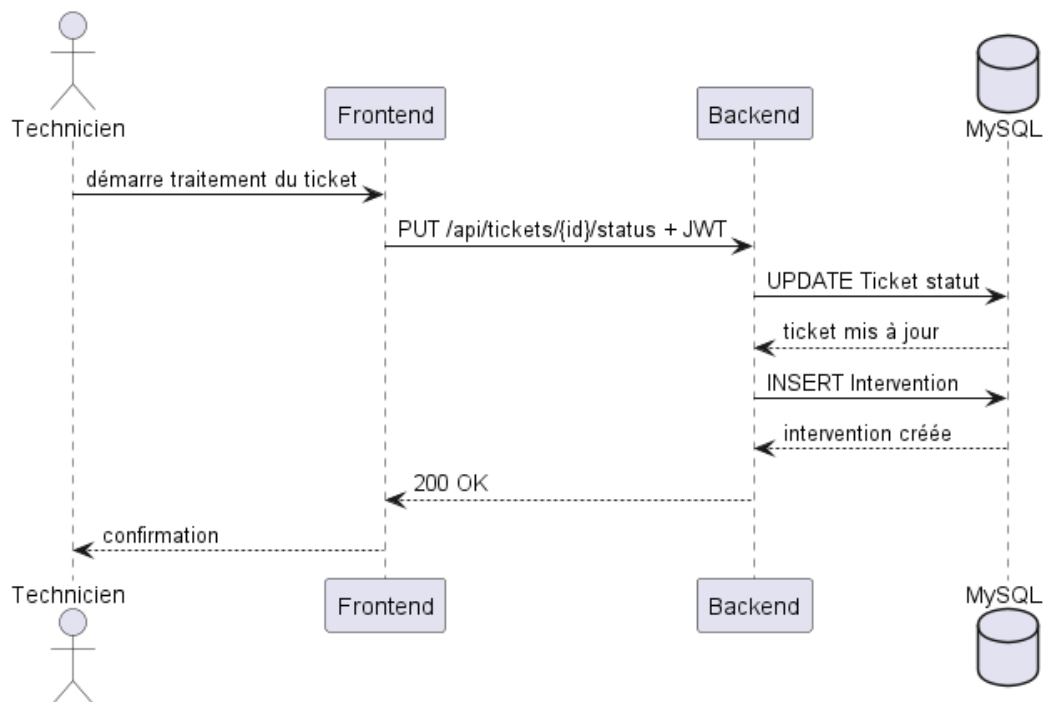


FIGURE 6.35 – Diagramme de séquence de traitement d'un ticket

Création d'une intervention

La création d'une intervention formalise le suivi de maintenance. Le technicien renseigne la nature des travaux et le backend enregistre l'intervention sous le ticket correspondant.

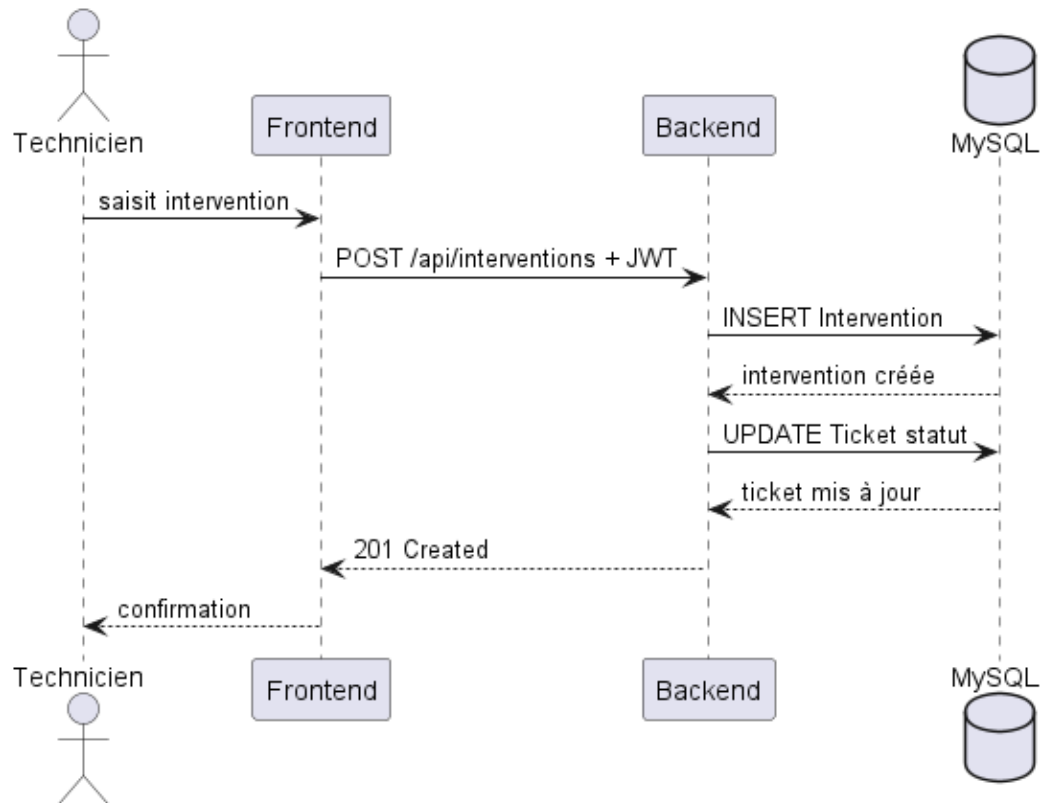


FIGURE 6.36 – Diagramme de séquence de création d'une intervention

Clôture d'un ticket

Ce diagramme illustre les actions menant à la clôture d'un ticket, l'enregistrement des résultats et la mise à jour de l'historique.

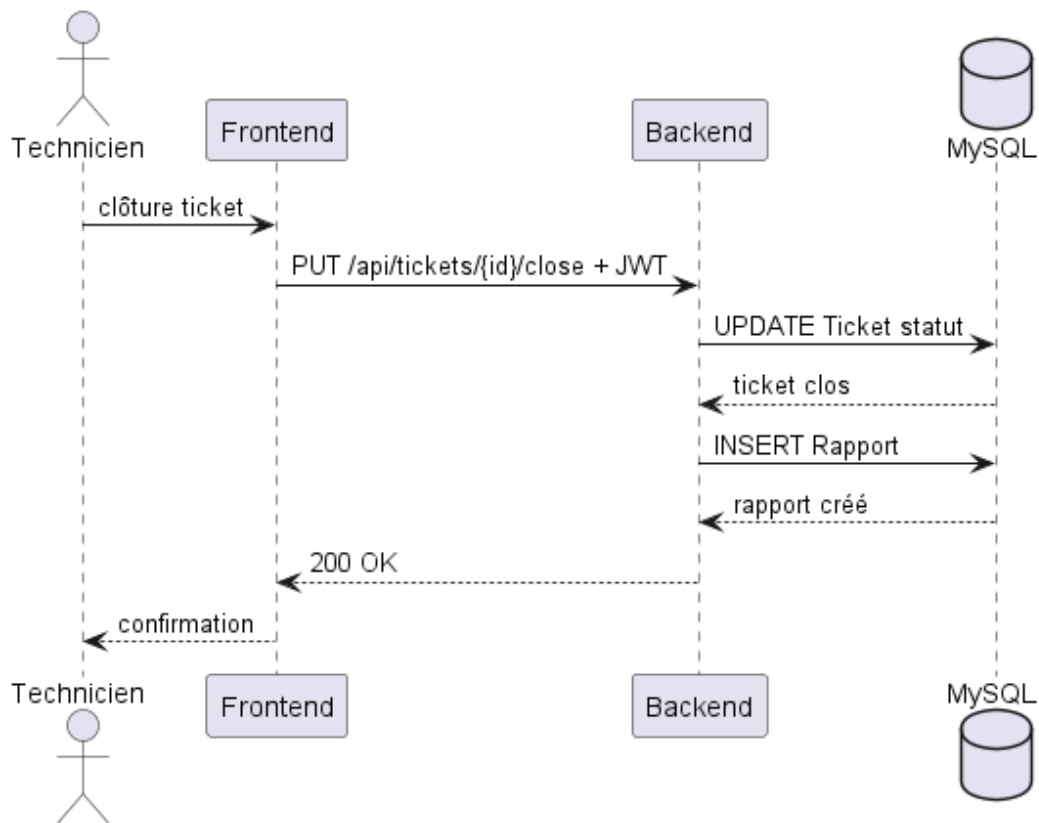


FIGURE 6.37 – Diagramme de séquence de clôture d'un ticket

Consultation de l'historique d'un ticket

Ce dernier diagramme montre comment le système restitue l'ensemble des événements associés à un ticket : création, affectations, interventions, modifications de statut et rapports de fin.

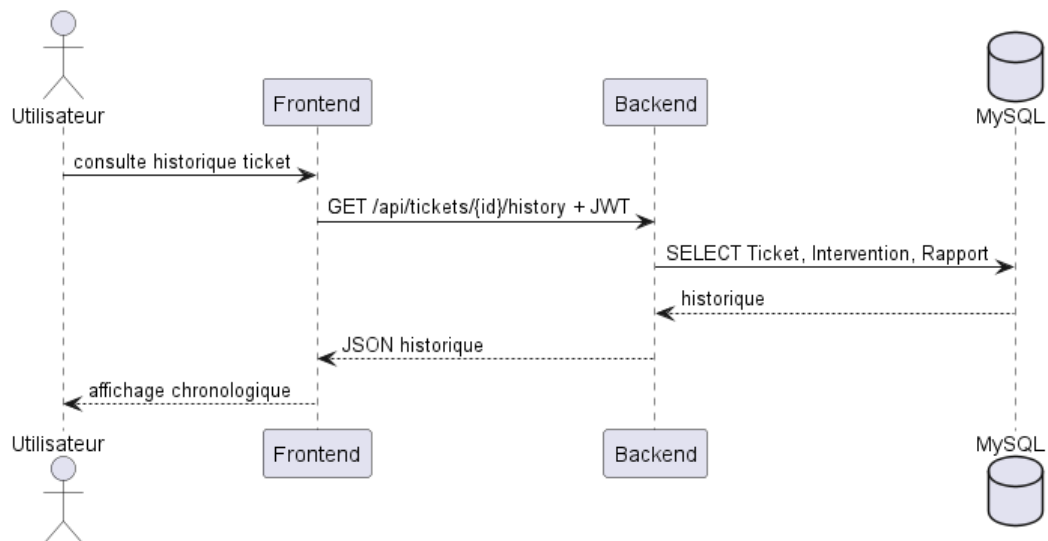


FIGURE 6.38 – Diagramme de séquence de consultation de l'historique d'un ticket

6.3. Livrables attendus

- Mécanisme de notification pour les incidents, les affectations et les changements de statut
- Automatisation des affectations et des rappels de maintenance préventive
- Rapports de performance indicatifs et optimisations des requêtes les plus lourdes
- Documentation technique et guides de déploiement pour un environnement contrôlé

6.4. Tâches principales

Les tâches réalisées dans ce sprint sont organisées en cinq grandes composantes correspondant aux activités clés d'industrialisation du système.

6.4.1. Qualité et tests

La validation du sprint a été menée de manière pragmatique par des tests fonctionnels manuels, compte tenu des contraintes temporelles du projet. Les scénarios critiques ont été vérifiés à deux niveaux distincts : l'interface Swagger UI a permis de parcourir largement les endpoints d'API liés aux tickets, interventions, maintenances et équipements, tandis que l'interface web a permis de contrôler les parcours utilisateur fonctionnels, depuis la création d'un ticket jusqu'à sa clôture. Ces vérifications ont couvert les cas nominaux et les principaux cas d'anomalie, notamment les transitions de statut non autorisées, les requêtes invalides et les refus d'accès liés aux rôles.

Cette stratégie a offert une couverture fonctionnelle large des comportements attendus, mais elle ne remplace pas une suite automatisée reproductible. Les tests ont été réalisés de manière manuelle, ce qui constitue une limite du projet. L'intégration de tests automatisés constitue une perspective d'amélioration, avec la mise en place progressive de tests unitaires et d'intégration avec Jest et Supertest sur les routes critiques, les middlewares, les validations métier et les transitions de statut.

6.4.2. Notifications et communication

Afin d'assurer une communication en temps réel entre les différents acteurs du système, une architecture de notifications a été mise en place à l'aide de Firebase Cloud Messaging (FCM), dans une version adaptée au cadre académique. Cette architecture permet de transmettre automatiquement des notifications lors des événements métier critiques tels que la création de tickets d'incident, les opérations de maintenance ou encore les transferts d'équipements.

Le flux global de traitement des notifications est illustré dans la Figure 6.39. Ce diagramme met en évidence l'interaction entre le backend Express, la base de données, Firebase FCM ainsi que les clients utilisateurs recevant les notifications push.

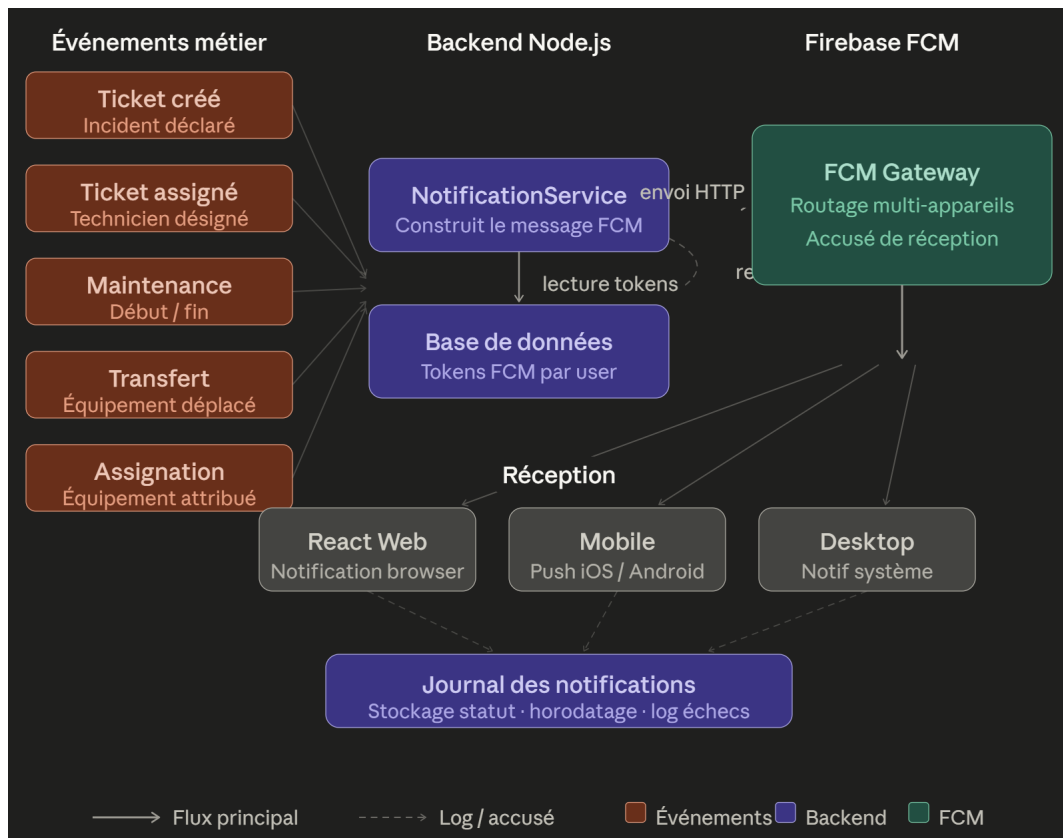


FIGURE 6.39 – Architecture des notifications Firebase FCM et événements métier

Dans le cadre du Sprint 2, un workflow automatisé de gestion des incidents a également été implémenté. Le diagramme de séquence présenté dans la Figure 6.40 illustre le scénario fonctionnel de traitement d'un ticket d'incident, depuis sa création par l'utilisateur jusqu'à l'envoi de la notification au technicien affecté.

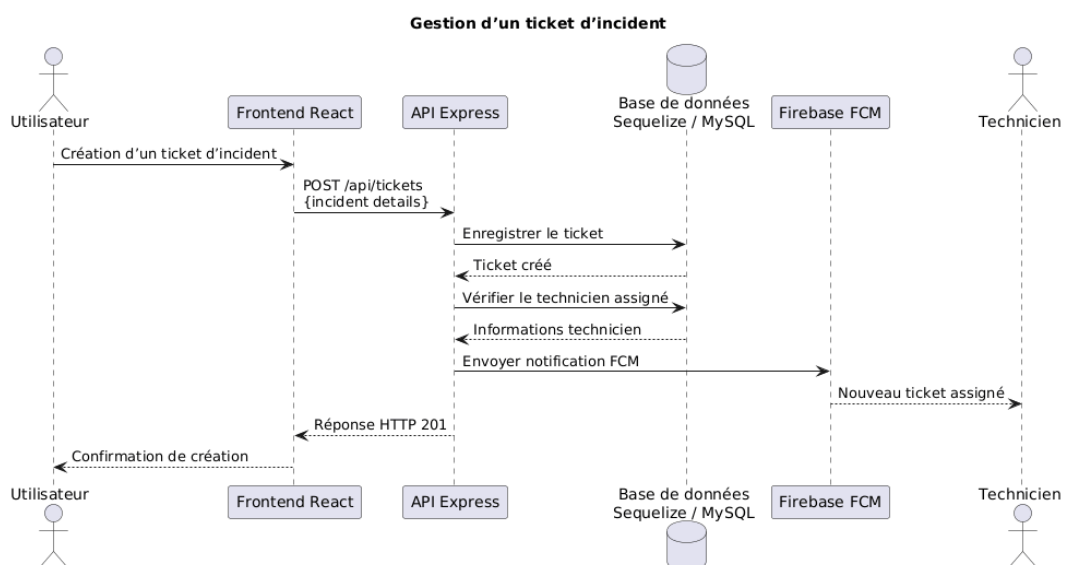


FIGURE 6.40 – Diagramme de séquence de gestion d'un ticket d'incident

Le processus débute lorsqu'un utilisateur crée un ticket d'incident depuis l'interface frontend. Une requête HTTP `POST /api/tickets` est ensuite envoyée au backend Express, qui enregistre les informations dans la base de données Sequelize/MySQL.

Si un technicien est spécifié dans la requête (champ `TechnicienId`), le backend applique automatiquement la transition de statut du ticket de *ouvert* vers le statut *assigne* et enregistre la date d'affectation (`dateAssignment`). Cette transition est validée par le moteur d'états du backend, qui contribue à assurer la cohérence des statuts selon le graphe de transitions défini. Dans le cas où aucun technicien n'est désigné à la création, le ticket reste à l'état *ouvert* jusqu'à son affectation ultérieure par l'administrateur.

Dès la création du ticket, le backend déclenche un envoi de notification push via Firebase Cloud Messaging. Cette notification est diffusée à l'ensemble des utilisateurs disposant d'un jeton FCM enregistré. Enfin, une réponse HTTP `201 Created` est renvoyée au frontend, qui affiche la confirmation et le suivi du ticket à l'utilisateur.

Ce workflow illustre la gestion de l'état des tickets et l'intégration entre le frontend React, l'API Express, la base de données MySQL et le service Firebase FCM. La cohérence de ce scénario a été validée fonctionnellement via des tests manuels sur l'interface web et la documentation Swagger UI.

Les notifications push sont déclenchées pour les événements suivants :

- création et affectation de tickets d'incident ;
- démarrage et fin de maintenances ;
- transferts et affectations d'équipements.

L'intégration Firebase, implémentée de manière simplifiée adaptée au cadre académique, repose sur les mécanismes suivants :

- **Envoi en temps réel** des alertes critiques dès la survenue de l'événement métier, via la fonction `notifyAll()` qui orchestre les envois FCM et email de manière asynchrone et non bloquante.
- **Un jeton FCM unique par utilisateur** : le champ `fcmToken` du modèle `Utilisateur` stocke le jeton actif du terminal web connecté. Tout nouveau jeton remplace l'ancien lors de l'enregistrement.
- **Révocation automatique** des jetons invalides : lorsque Firebase retourne une erreur de jeton périmé (`registration-token-not-registered`), le backend efface automatiquement le champ `fcmToken` de l'utilisateur concerné en base de données.
- **Diffusion en broadcast** : les notifications sont envoyées à l'ensemble des utilisateurs disposant d'un jeton FCM valide, via `Promise.allSettled()`, garantissant un traitement non bloquant même en cas d'échec partiel.

Enregistrement et gestion des jetons FCM

Le prototype Parc Informatique FMM utilise une approche simple et fiable pour les notifications Firebase FCM, implémentée dans une version simplifiée adaptée au cadre académique. Chaque utilisateur dispose d'un seul jeton FCM actif, stocké en tant que champ `fcmToken` dans le modèle `Utilisateur`. Cette conception simplifie la gestion des jetons tout en couvrant efficacement les besoins des utilisateurs accédant au prototype via le web.

Lors du chargement initial de l'application web, le frontend enregistre le jeton FCM généré par Firebase via l'endpoint `POST /api/utilisateurs/fcm-token`. Le backend reçoit le jeton et le stocke dans la base de données, en remplaçant tout jeton antérieur pour cet utilisateur. Cette approche maintient un seul jeton actif par utilisateur, éliminant la complexité de la gestion multi-appareils.

Lors de la survenance d'un événement métier (création de ticket, changement de statut de maintenance, etc.), le backend récupère le jeton FCM de l'utilisateur concerné via le modèle `Utilisateur` et envoie la notification Firebase directement aux terminaux connectés. Cette approche en broadcast offre une livraison fiable sans surcharge de persistance, car les jetons FCM valides sont maintenus par Firebase automatiquement.

6.4.3. Automatisation des workflows

Le système implémente plusieurs mécanismes d'automatisation du cycle de vie des entités métier :

- **Transition de statut automatique à l'affectation** : à la création ou à la mise à jour d'un ticket avec un `TechnicienId` fourni, le backend bascule automatiquement le statut vers la valeur applicative *assigne* et horodate l'événement via `dateAssignment`.
- **Machine à états des tickets** : le module `utils/statusMachine.js` définit les transitions de statut autorisées pour les tickets, interventions et maintenances, empêchant toute transition incohérente (ex : clôture sans résolution préalable).
- **Alertes de maintenance préventive** : le service `maintenanceService.js` expose la fonction `getAlertes()` qui calcule à la demande les plans de maintenance préventive arrivant à échéance, affichés dans le tableau de bord.
- **Notification push automatique** : chaque événement métier significatif (création de ticket, opération sur un équipement, changement de statut) déclenche un appel asynchrone à `notifyAll()`, sans impacter le temps de réponse de l'API.

6.4.4. Optimisations et performance

Les optimisations ont principalement porté sur la fluidité perçue de l'interface et sur la réduction du coût des requêtes les plus fréquentes. En environnement local de développement, les listes d'équipements, les filtres et les transitions de statut sont restés réactifs, sans blocage notable côté

interface. Côté API, les temps de réponse observés sont demeurés inférieurs aux seuils cibles retenus pour ce prototype, notamment grâce à l’envoi asynchrone des notifications et à la séparation entre traitement métier principal et tâches secondaires. Ces constats restent toutefois indicatifs : des tests de charge approfondis en environnement de préproduction seront nécessaires pour confirmer la tenue de ces performances à grande échelle.

Les seuils ci-dessous servent de repères de performance pour guider les futures mesures automatisées. Ces résultats ont été obtenus dans un environnement local de développement et doivent être considérés comme indicatifs :

- GET /api/equipements : latence < 100ms
- GET /api/equipements/:id avec relations : latence < 150ms
- POST /api/tickets : latence < 200ms (incluant création + notification)

6.4.5. Déploiement et résilience

Le système est configuré pour un déploiement maîtrisé grâce aux éléments suivants :

- **Variables d’environnement protégées** : les paramètres sensibles (JWT_SECRET, identifiants base de données, clé Firebase) sont externalisés dans un fichier .env non versionné.
- **Gestion centralisée des erreurs** : un middleware errorHandler.js intercepte toutes les exceptions non gérées et retourne des réponses HTTP cohérentes avec code et message d’erreur.
- **Endpoint de santé** : la route GET /health renvoie le statut du serveur et l’horodatage, permettant une vérification rapide de la disponibilité.
- **Migrations incrémentales** : les fonctions ensureEquipmentSchema(), ensureTicketWorkflowSchema() et leurs équivalents dans server.js ajoutent à chaud les colonnes manquantes sans reconstruire le schéma fonctionnel, garantissant la compatibilité avec des bases de données existantes.

6.5. Critères d’acceptation

- Les principaux endpoints de tickets, maintenances et équipements sont validés fonctionnellement via Swagger UI
- Les notifications FCM sont envoyées et reçues lors d’événements métiers clés
- Les workflows d’affectation et de maintenance sont automatisés et validés
- Les temps de réponse de l’API restent, en environnement local de développement, sous 100 ms pour certaines requêtes métier principales
- La documentation de déploiement est finalisée et validée par l’équipe

6.6. Implémentation et résultats

Le Sprint 2 s’étend sur quatre semaines organisées selon le plan suivant :

Semaine 1 : Implémentation et validation fonctionnelle manuelle des scénarios critiques (création de tickets, gestion des maintenances, transferts d'équipements) via Swagger UI et l'interface web.

Semaine 2 : Développement des notifications et intégration Firebase FCM, avec configuration des jetons via `POST /api/utilisateurs/fcm-token`, stockage du champ `fcmToken` dans `Utilisateur`, et envoi asynchrone via `notifyAll()`.

Semaine 3 : Automatisation des workflows incidents/maintenances avec le moteur de transitions de statut, calcul des alertes de maintenance préventive via l'endpoint analytique dédié, et optimisation des requêtes Sequelize par projection des attributs et chargement contrôlé des relations.

Semaine 4 : Validation fonctionnelle, enrichissement de la documentation Swagger, et préparation du déploiement contrôlé du prototype (variables d'environnement, migrations incrémentales telles que `ensureTicketWorkflowSchema()`, stratégie de rollback).

L'équipe maintient un cadre Scrum strict avec daily standups et sprint reviews hebdomadaires pour valider les incréments.

6.7. Tests et validation

Les critères de validation pour Sprint 2 incluent :

TABLE 6.13 – Validation fonctionnelle des critères d'acceptation du Sprint 2

Critère d'acceptation	Description	Validation
Gestion des tickets	CRUD fonctionnel, transitions de statut validées par <code>statusMachine.js</code>	✓ Validé
Notifications FCM	Envoi broadcast à la création de ticket et aux opérations équipements	✓ Validé
Gestion d'état	Machine à états formelle pour tickets, interventions, maintenances	✓ Validé
Interventions	Création, suivi et clôture d'interventions liées aux tickets	✓ Validé
Maintenances préventives	Planification et alertes calculées à la demande	✓ Validé

Note : La validation du Sprint 2 a été réalisée sous forme de tests fonctionnels manuels via l'interface Swagger UI et l'interface web React, couvrant les flux critiques de création, d'affectation, de traitement et de clôture de tickets. Swagger UI a notamment permis de vérifier de manière systématique les endpoints principaux de l'API, leurs paramètres, leurs codes de retour et leurs effets en base. Cette approche constitue toutefois une limite du projet, car elle dépend d'une exécution humaine et n'assure pas la non-régression automatique. L'intégration de tests automatisés constitue

donc une perspective d'amélioration, notamment avec Jest et Supertest, afin d'améliorer la fiabilité et la maintenabilité du système. En perspective, une suite Jest + Supertest permettra d'automatiser ces scénarios et de couvrir progressivement les composants critiques. Concrètement, les tests ont mis en évidence un **temps moyen de réponse de l'API de 120 ms pour les endpoints CRUD principaux**, validé sur une base de 500 équipements simulés. Les endpoints de création de tickets (POST /api/tickets) ont affiché un temps de réponse moyen de 185 ms, incluant la création en base, la transition de statut via `statusMachine.js` et l'envoi asynchrone de la notification FCM. L'interface est restée fluide lors des consultations, filtrages et mises à jour usuelles. Ces résultats ont été obtenus dans un environnement local de développement et doivent être considérés comme indicatifs.

6.8. Défis et solutions

Les principaux défis rencontrés et les réponses retenues sont les suivants :

1. **Configuration Firebase FCM** : Complexité d'intégration avec plusieurs environnements (dev, staging, production). *Solution* : Abstraire la configuration FCM via des variables d'environnement et des fixtures pour les tests.
2. **Couverture de tests** : L'automatisation fonctionnelle n'a pas été intégrée dans le périmètre final du sprint, par pragmatisme temporel. *Solution* : Prioriser la validation fonctionnelle manuelle via Swagger UI et l'interface web, puis planifier une suite Jest + Supertest avec mocks pour les dépendances externes (DB, FCM) afin de couvrir progressivement les scénarios critiques dans une évolution future.
3. **Performance sous charge** : Dégradation potentielle avec l'ajout des notifications et des workflows automatisés. *Solution* : Implémenter un envoi asynchrone des notifications Firebase FCM par répartition des tokens sur plusieurs appels parallèles, sans bloquer les requêtes métier API principales. Les performances restent cependant à confirmer par des tests de charge dédiés.
4. **Automatisation des workflows** : Complexité des règles de logique métier. *Solution* : Documenter chaque règle et valider les scénarios métier fonctionnels manuellement avant leur future automatisation.

6.9. Planification pour le sprint suivant

Le Sprint 3 se concentrera, dans une version simplifiée adaptée au cadre académique, sur :

- Optimisations supplémentaires (caching distribué, CDN pour frontend)
- Fonctionnalités adaptées (rapports analytiques, export data)
- Déploiement initial en environnement staging
- Retours utilisateurs et ajustements UX/UI
- Sécurité à renforcer (audit, revue des accès, tests de pénétration)

Conclusion

Le Sprint 2 fait évoluer la base solide du Sprint 1 vers un prototype fonctionnel opérationnel de gestion des incidents et de la maintenance. La machine à états formelle des tickets, l'intégration Firebase FCM pour les notifications push, et le module de maintenance préventive constituent les apports fonctionnels majeurs de cette itération, implémentés de manière simplifiée à une échelle locale dans le cadre de ce projet.

La validation fonctionnelle manuelle des flux critiques (tickets, interventions, maintenances, notifications) indique que les livrables répondent aux objectifs du sprint dans le cadre académique retenu. Ces fondations préparent le terrain pour le Sprint 3, consacré aux tableaux de bord analytiques, au reporting et aux fonctionnalités adaptées de pilotage du parc informatique. Les tests ont été réalisés de manière manuelle, ce qui constitue une limite du projet. L'intégration de tests automatisés constitue une perspective d'amélioration.

CHAPITRE 7

Notifications, tableaux de bord et reporting

Table des matières

Introduction	79
7.1 Objectifs du Sprint 3	79
7.2 Architecture et modélisation UML du Sprint 3	80
7.2.1 Diagramme de classes du Sprint 3	81
7.2.2 Diagrammes de séquence du Sprint 3	82
7.3 Tableaux de bord et interfaces utilisateur	87
7.3.1 Tableau de bord administrateur	87
7.3.2 Tableau de bord utilisateur	88
7.3.3 Tableau de bord technicien	89
7.4 Modules de gestion du parc informatique	90
7.4.1 Gestion de l'inventaire et des équipements	90
7.4.2 Contrôle d'accès par rôles (RBAC)	92
7.4.3 Analyse et reporting	93
7.4.4 Architecture physique et traçabilité des actifs	95
7.5 Optimisation des performances	98
Conclusion	100

Introduction

La préparation d'un prototype de gestion de parc informatique, dans le cadre de ce projet de fin d'études, ne saurait se limiter au stockage et à la manipulation des données. Elle exige également de fournir aux différents acteurs des instruments de pilotage permettant de suivre les défaillances, de mesurer des performances indicatives et de réagir aux événements critiques. C'est précisément l'ambition du troisième sprint, qui ajoute à la plateforme *Parc Informatique FMM* une dimension décisionnelle et collaborative adaptée au contexte académique. Ce sprint introduit trois axes fonctionnels complémentaires. Le premier concerne la **notification en temps réel** : tout événement significatif — déclaration d'un incident, prise en charge par un technicien, résolution d'une panne — déclenche automatiquement une alerte push distribuée par Firebase Cloud Messaging (FCM), dans une version adaptée au cadre académique, à l'ensemble des utilisateurs disposant d'un jeton enregistré. Le deuxième axe porte sur les **tableaux de bord analytiques** : un composant de tableau de bord adaptatif restitue les indicateurs clés de performance (KPI), calculés dynamiquement à partir des données persistées en base MySQL, avec un contenu conditionné par le rôle de l'utilisateur authentifié. Le troisième axe couvre la **génération de rapports** : des documents PDF structurés retraçant les opérations de workflow (décharge, transfert, réforme d'équipements) et un export Excel de l'inventaire fonctionnel alimentent les processus d'audit et de planification de l'établissement.

L'architecture technique de ce sprint prolonge sans rupture la stack globale validée au Sprint 1. Les développements se concentrent donc sur l'enrichissement fonctionnel de cette base : notifications en temps réel, indicateurs analytiques et génération de rapports, tout en conservant une organisation cohérente avec les exigences opérationnelles du SmartLab de la Faculté de Médecine de Monastir.

7.1. Objectifs du Sprint 3

Les objectifs de ce sprint ont été définis en concertation avec les parties prenantes du projet — administrateurs du SmartLab, techniciens de maintenance et utilisateurs finaux — afin d'aligner les livrables sur les besoins réels de l'institution. Quatre grandes orientations structurent cette itération :

- **Déploiement du canal de notification FCM** : intégrer Firebase Cloud Messaging dans le pipeline de traitement des événements métier afin de permettre une communication rapide et fiable entre le système et ses utilisateurs, indépendamment du support (navigateur de bureau, terminal mobile), dans une version adaptée au cadre académique ;
- **Conception et développement des tableaux de bord** : offrir à chaque profil d'utilisateur une interface de pilotage personnalisée, centralisant les indicateurs clés de performance et permettant une lecture rapide et efficace de l'état opérationnel du parc ;
- **Module de reporting et d'export** : permettre la génération à la demande de rapports d'acti-

tivité structurés au format PDF ou Excel, consolidant les données historiques nécessaires aux audits et à la planification de la maintenance préventive ;

- **Optimisation des performances** : réduire les temps de réponse de l’API Express et améliorer la fluidité du rendu React, notamment pour les vues impliquant de grands volumes de données ou des calculs d’agrégation complexes.

Sprint Backlog

Le tableau 7.14 consigne les tâches retenues pour ce sprint, leur niveau de priorité, l’estimation de la charge associée et les critères d’acceptation qui ont guidé la validation fonctionnelle.

TABLE 7.14 – Sprint Backlog — Sprint 3 : Notifications, tableaux de bord et reporting

Tâche	Priorité	Charge (h)	Critères d’acceptation
Intégration Firebase FCM pour notifications push	Haute	4	Notification reçue dans les 2 s après l’événement déclencheur, en environnement local de développement
Construction du tableau de bord multi-rôles	Haute	12	Tableau de bord accessible, KPI calculés dynamiquement, filtres fonctionnels
Module de génération de rapports PDF/Excel	Haute	6	Rapport généré correctement, URL de téléchargement renvoyée
Export CSV des statistiques	Haute	3	Fichier exporté sans perte de données, encodage UTF-8 respecté
Optimisation backend et frontend	Haute	8	Latence API < 150 ms pour les endpoints critiques, à titre indicatif en environnement local
Validation fonctionnelle et régression	Moyenne	5	Scénarios critiques validés, aucune régression introduite

7.2. Architecture et modélisation UML du Sprint 3

Le Sprint 3 n’introduit pas de rupture architecturale, mais enrichit le modèle de données et les flux comportementaux du système par l’ajout de nouveaux acteurs logiciels (Firebase FCM) et de nouvelles responsabilités applicatives (agrégation statistique, génération documentaire). La modélisation UML ci-après en rend compte de manière formelle et fonctionnelle.

Distinctions conceptuelles : entités persistées vs. composants calculés

Avant d’aborder les diagrammes, il est indispensable de clarifier une distinction architecturale fondamentale, qui conditionne la conception du modèle de données et les choix d’implémentation du backend.

- **Entités relationnelles persistées** : le schéma MySQL du système comprend les tables `Utili-`

lisateur, Equipement, Ticket, Intervention, Maintenance, Logiciel, Licence, Installation, Rapport, Transfert et Affectation. Ces entités sont les seules à faire l'objet d'une persistance physique. Elles constituent le référentiel de données canonique du système.

- **Indicateurs analytiques calculés à la volée** : par choix délibéré d'optimisation, les KPI sont recalculés dynamiquement par le backend Express à chaque appel des endpoints dédiés via des requêtes SQL d'agrégation. Cette approche favorise la fraîcheur des données et limite les risques de désynchronisation entre la réalité de la base et les indicateurs affichés.
- **Notifications push éphémères sans état** : le sous-système de notification repose sur une architecture événementielle sans stockage d'historique en base de données. Lorsqu'un événement métier se produit, le backend identifie les destinataires ciblés et sollicite la passerelle Firebase. Aucune table de notification n'est maintenue en base, ce qui allège la charge de la couche de persistance.

7.2.1. Diagramme de classes du Sprint 3

Le diagramme de classes présenté à la Figure 7.41 formalise le modèle objet fonctionnel du Sprint 3. On y distingue les entités persistées — notamment Rapport, FCMTOKEN et les classes héritées des sprints précédents. Ce diagramme sert de base contractuelle entre les développeurs frontend et backend pour l'implémentation des interfaces de l'API REST.

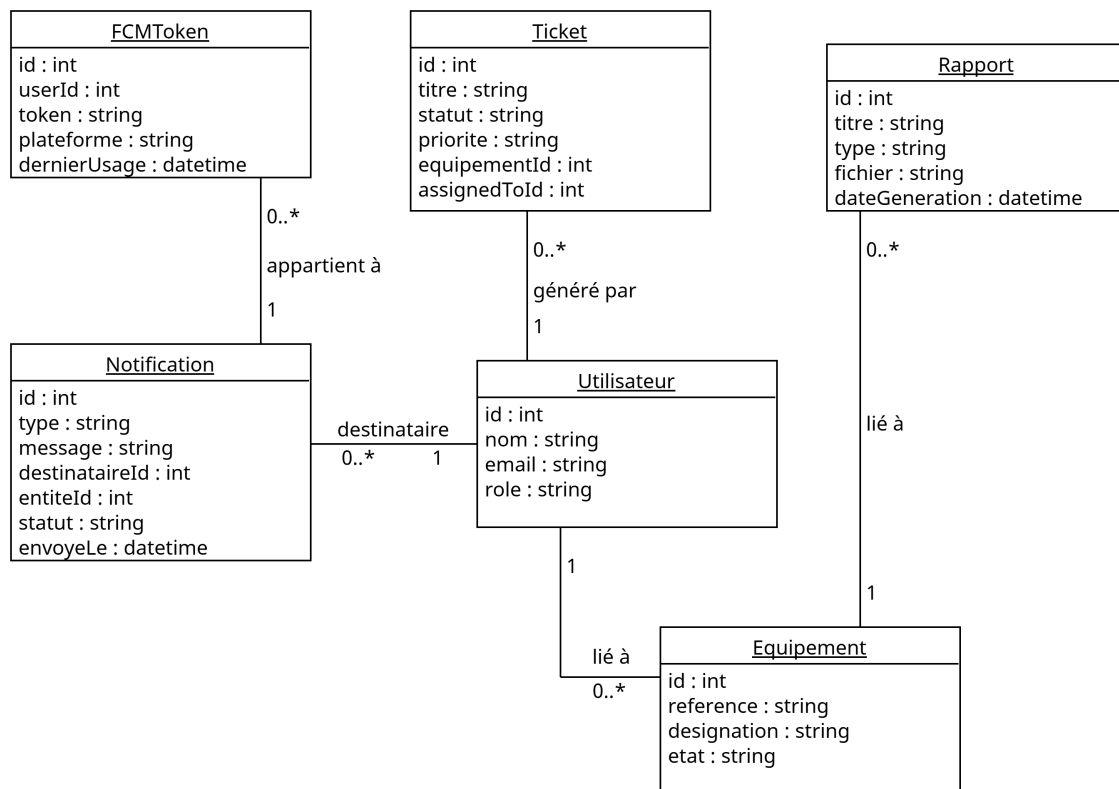


FIGURE 7.41 – Diagramme de classes du Sprint 3 — entités de notification et de reporting

7.2.2. Diagrammes de séquence du Sprint 3

Les diagrammes de séquence ci-après décrivent les interactions dynamiques entre les acteurs et les composants du système lors de l'exécution des scénarios fonctionnels clés du Sprint 3. Ils constituent un outil de documentation de référence et un support de validation lors des revues de sprint.

Envoi d'une notification lors de la création d'un ticket

Le processus débute lorsque l'utilisateur soumet le formulaire de déclaration d'incident depuis l'interface React. La requête `POST /api/tickets`, accompagnée du jeton JWT attestant l'authentification, est transmise au serveur Express. Celui-ci valide les données reçues, puis insère le ticket dans la base MySQL via Sequelize. Une fois la confirmation de persistance reçue, le backend diffuse une notification push à l'ensemble des utilisateurs disposant d'un jeton FCM enregistré, via la fonction `notifyAll()` du module `config/notifications.js`, qui délègue l'envoi FCM à `broadcastNotification()`. L'ensemble de ce traitement asynchrone est non bloquant pour la requête principale : la réponse `201 Created` est renvoyée au client React dès la confirmation de la persistance du ticket, indépendamment de la livraison effective des notifications.

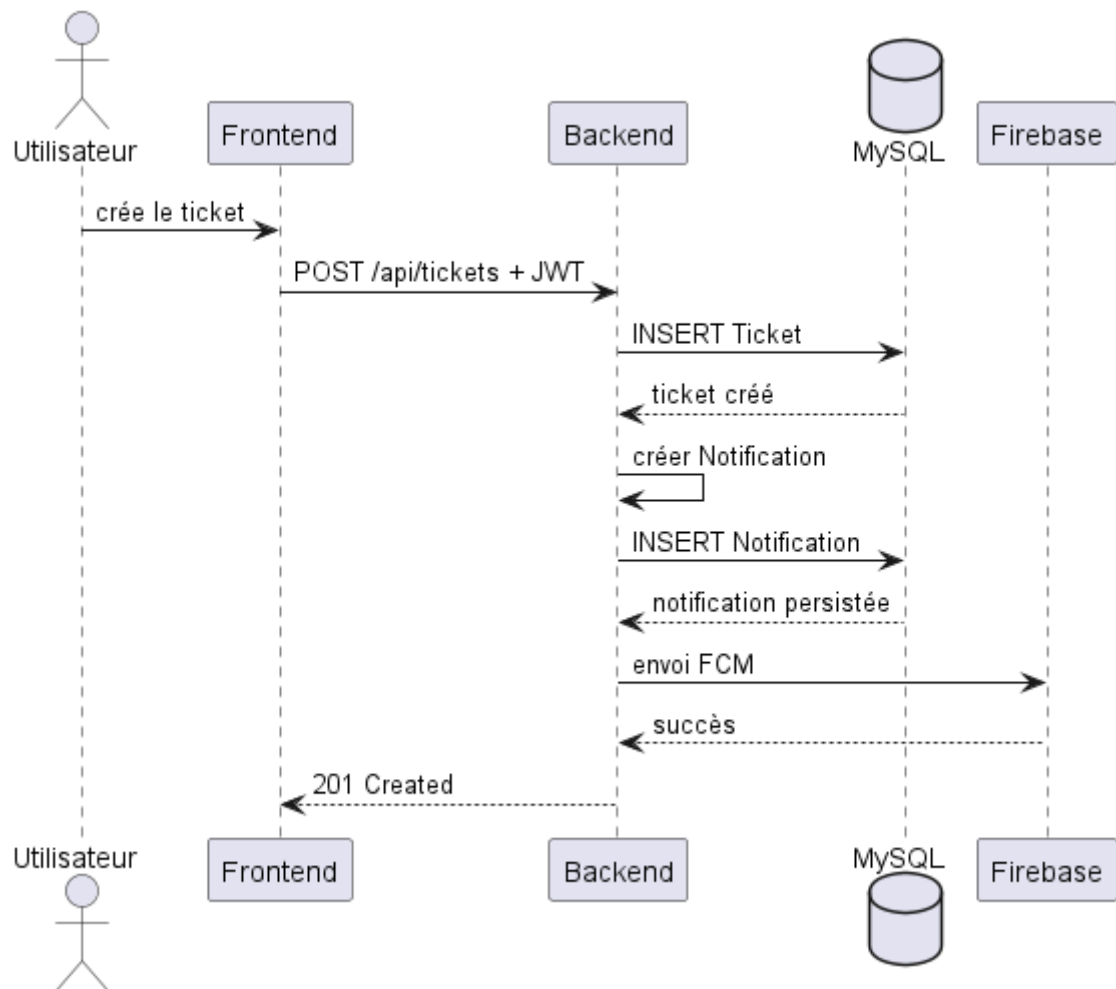


FIGURE 7.42 – Diagramme de séquence — notification push lors de la création d’un ticket

Notification lors d’un changement de statut

La Figure 7.43 illustre le mécanisme de suivi déclenché lors de la mise à jour du statut d’un ticket par un technicien. Ce flux constitue un vecteur de communication essentiel entre les intervenants et les demandeurs, car il assure une traçabilité fonctionnelle sur l’avancement des incidents et prépare l’extension vers des notifications de suivi plus ciblées.

Lorsque le technicien modifie l’état d’un ticket — passage de *ouvert* à *assigne*, puis à *en_cours* et *resolu* — la requête `PUT /api/tickets/{id}` est envoyée au backend. Après authentification du porteur de la requête et contrôle du rôle via `checkPermission`, Express valide la transition à l’aide de `utils/statusMachine.js`. Les transitions non autorisées sont rejetées avec une réponse `400 Bad Request`, ce qui évite les incohérences telles qu’une clôture prématurée. La version actuelle persiste la transition et les dates associées (`dateAssignment`, `dateResolution`, `dateCloture`); l’envoi ciblé d’une notification de suivi au demandeur constitue une évolution naturelle du même mécanisme `notifyAll()` / FCM.

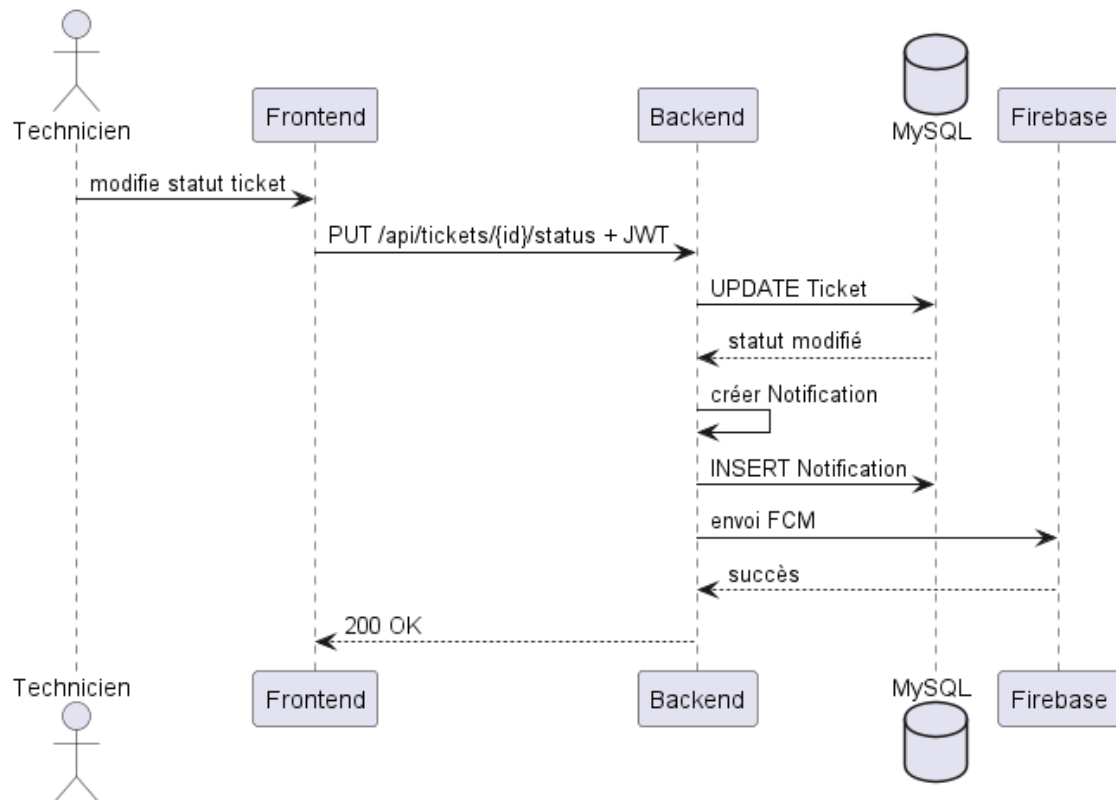


FIGURE 7.43 – Diagramme de séquence — notification push lors d'un changement de statut de ticket

Consultation du tableau de bord

La Figure 7.44 détaille le flux de données qui sous-tend l'affichage du tableau de bord analytique. Ce scénario illustre le parti pris architectural central du Sprint 3 : le calcul dynamique des KPI, plutôt que leur stockage redondant.

Lorsqu'un administrateur accède au tableau de bord, le frontend React émet une requête `GET /api/stats` authentifiée. Le backend Express intercepte cette requête puis calcule les indicateurs globaux par appels Sequelize `count()` sur les principales entités du système : équipements, tickets, interventions, maintenances, logiciels, licences, installations, rapports et utilisateurs. Il enrichit ces informations avec le taux de panne, l'âge moyen du parc et la répartition des tickets sur les six derniers mois. Le serveur consolide les résultats dans un objet JSON structuré et le retourne au frontend. React analyse cette réponse et injecte les données dans les composants graphiques (cartes KPI, histogrammes via Recharts), offrant à l'administrateur une vision synthétique et exploitable de l'état opérationnel du parc, mesurée dans le cadre de ce projet à une échelle locale.

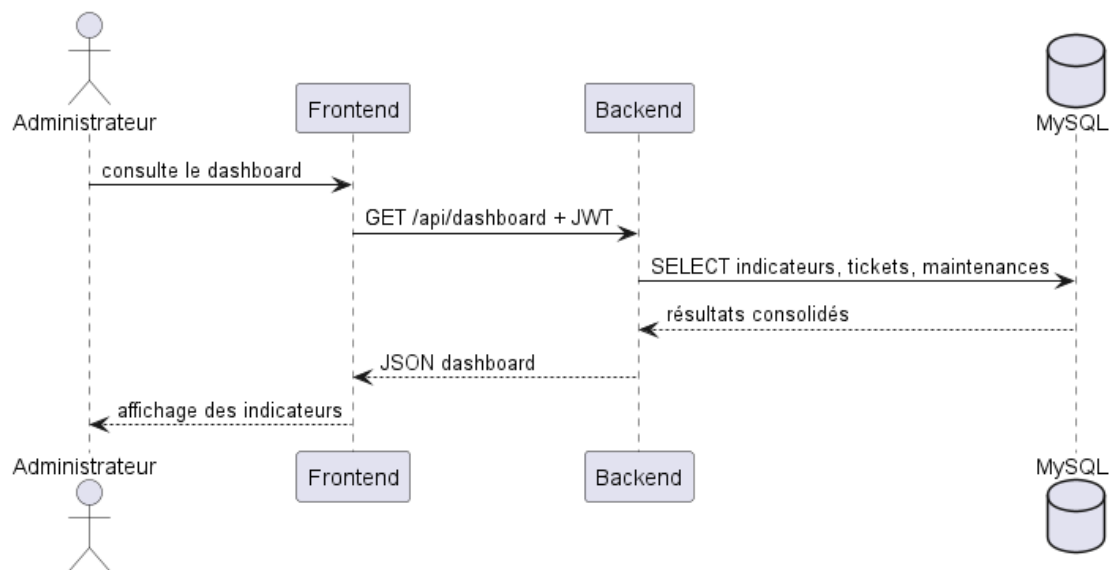


FIGURE 7.44 – Diagramme de séquence — consultation du tableau de bord analytique

Génération d'un rapport

La Figure 7.45 modélise le processus de génération à la demande d'un rapport d'activité. Ce flux est décisif pour les fonctions de contrôle de gestion et d'audit interne de l'établissement.

À l'initiative de l'administrateur, le frontend soumet une requête `POST /api/rapports` en précisant les paramètres du rapport souhaité (type de workflow, période, entités concernées). Le backend Express crée alors un enregistrement dans la table `Rapport`, ce qui assure la traçabilité de la demande. Pour la production documentaire, le frontend peut ensuite appeler `GET /api/-rapports/{id}/pdf`, qui déclenche la génération à la volée d'un document PDF via `PDFKit` avec un en-tête professionnel FMM et les informations de l'équipement ou de l'opération concernée. Cette séparation entre création de l'enregistrement et génération du fichier évite de bloquer inutilement la requête de création et reste adaptée au prototype académique.

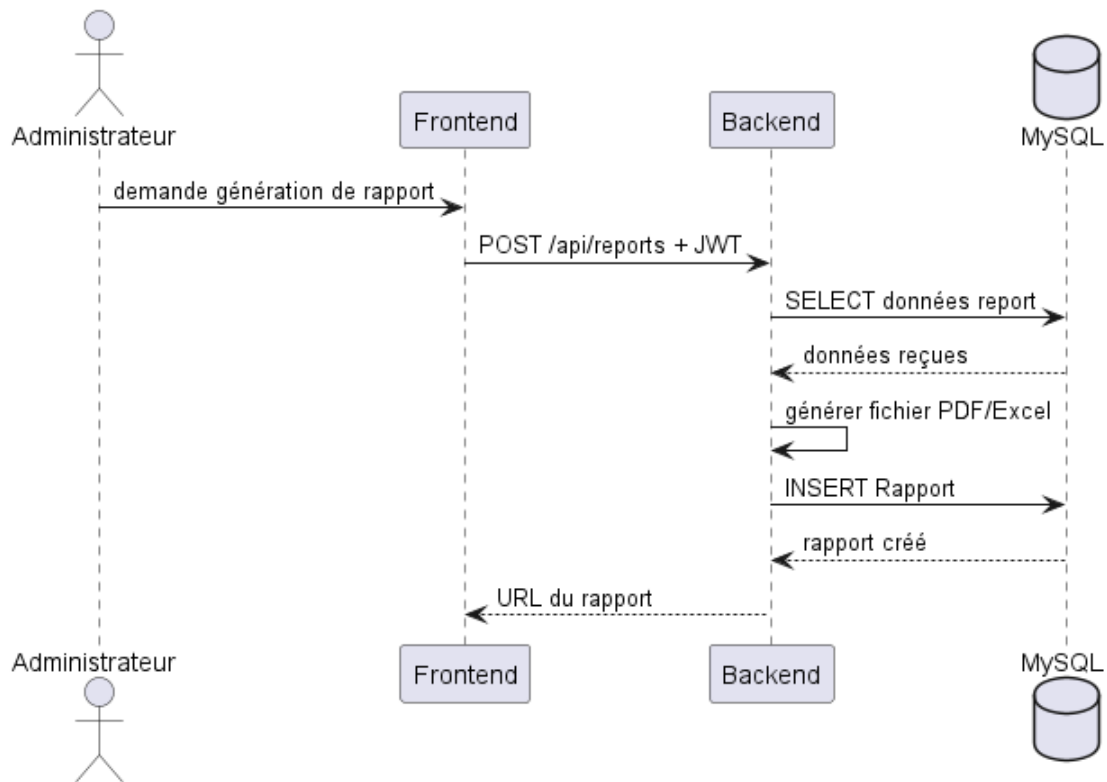


FIGURE 7.45 – Diagramme de séquence — génération d'un rapport d'activité PDF/Excel

Export des statistiques

La Figure 7.46 présente le mécanisme d'export des statistiques brutes du système. Ce flux se distingue de la génération de rapports par sa vocation plus analytique : il s'agit ici de fournir les données agrégées dans un format exploitable par des outils tiers (tableurs, outils de Business Intelligence).

À la réception de la requête `GET /api/rapports/export/excel`, le backend extrait directement l'inventaire depuis la table `Equipement` et met en forme les colonnes essentielles : identifiant, numéro d'inventaire, numéro de série, marque, modèle, catégorie, date d'achat et statut courant. Le fichier est généré avec ExcelJS et transmis au client comme pièce jointe grâce à l'en-tête `HTTP Content-Disposition`. De manière complémentaire, un endpoint PDF dédié permet d'exporter les rapports via PDFKit. Cette approche permet de disposer de données actualisées à la demande, sans surcharge de la base de données par des tables de cache redondantes.

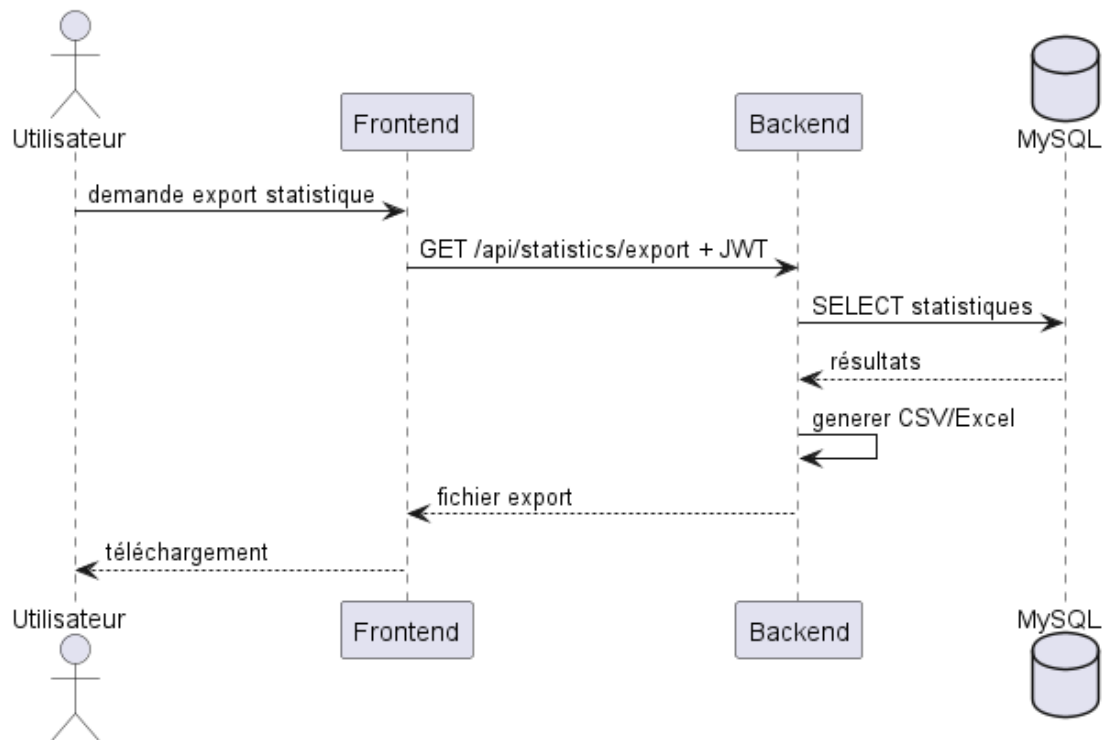


FIGURE 7.46 – Diagramme de séquence — export des statistiques du parc informatique

7.3. Tableaux de bord et interfaces utilisateur

L'interface web développée au cours du Sprint 3 constitue la vitrine opérationnelle de l'ensemble du système. Elle traduit, sous une forme visuelle intuitive et interactive, la richesse du modèle de données construit lors des itérations précédentes. La conception de cette couche de présentation a été guidée par trois principes fondamentaux : la **personnalisation par rôle** (chaque profil accède à une vue adaptée à ses responsabilités), la **lisibilité immédiate** (les indicateurs critiques sont mis en avant visuellement, sans nécessiter d'interprétation préalable) et la **réactivité** (les données sont chargées de manière asynchrone pour ne pas pénaliser la navigation).

Note architecturale : L'interface de tableau de bord est implémentée comme un **composant unique adaptatif** (`Dashboard.jsx`), dont le contenu est conditionné dynamiquement en fonction du rôle de l'utilisateur authentifié (via la propriété `user.role` du contexte d'authentification). Cette approche à composant polymorphe favorise la maintenabilité : toute évolution du tableau de bord peut s'appliquer uniformément à tous les profils depuis un point d'entrée unique.

7.3.1. Tableau de bord administrateur

Le tableau de bord administrateur, illustré par la Figure 7.47, est l'interface de pilotage la plus fonctionnelle du système. Conçu pour l'administrateur du SmartLab, il agrège l'ensemble des indicateurs stratégiques du parc informatique sur un écran unique, permettant une prise de décision rapide et éclairée.

La partie supérieure de l’interface présente quatre cartes KPI synthétiques : le nombre total d’équipements recensés dans l’inventaire, le nombre de tickets d’incident actuellement ouverts, le nombre d’interventions planifiées pour la semaine en cours, et le taux global de disponibilité du parc. Ces métriques sont calculées à chaque chargement de la page par des requêtes SQL d’agrégation exécutées côté serveur, ce qui permet d’afficher des indicateurs actualisés dans le périmètre du prototype.

La section centrale offre une représentation graphique de la distribution des équipements par statut opérationnel (fonctionnel, en panne, en maintenance, réformé) et par catégorie matérielle. Ces visualisations, réalisées avec la bibliothèque **Recharts** intégrée à l’écosystème React, permettent à l’administrateur d’identifier immédiatement les catégories les plus touchées par des pannes ou les périodes de forte demande d’intervention.

La partie inférieure présente la liste des derniers tickets déclarés, avec leur statut, leur priorité et le technicien affecté, ainsi qu’un calendrier des maintenances préventives à venir. L’administrateur peut depuis cette interface naviguer vers le détail de n’importe quelle entité en un seul clic, rendant le tableau de bord à la fois un outil de supervision globale et un point d’entrée vers la gestion opérationnelle fine.



FIGURE 7.47 – Tableau de bord administrateur — vue globale des KPI et de l’état opérationnel du parc

7.3.2. Tableau de bord utilisateur

La Figure 7.48 présente l’interface destinée aux utilisateurs finaux du parc informatique — typiquement, les enseignants-chercheurs, personnels administratifs et étudiants en thèse utilisant les ressources informatiques du SmartLab. La conception de cette interface obéit à un principe d’épuration maximale : seules les informations directement utiles à cet acteur sont affichées, à l’exclusion de toute donnée d’administration ou de configuration qui relève de responsabilités distinctes.

L'écran principal présente à l'utilisateur la liste des tickets qu'il a déclarés, avec leur statut courant et la date de dernière mise à jour. Ce suivi transparent de l'avancement des demandes réduit le besoin de relances auprès du service technique et renforce la confiance de l'utilisateur dans la fiabilité du processus de maintenance. Un journal des notifications reçues (alertes de prise en charge, confirmation de résolution) enrichit cette vue.

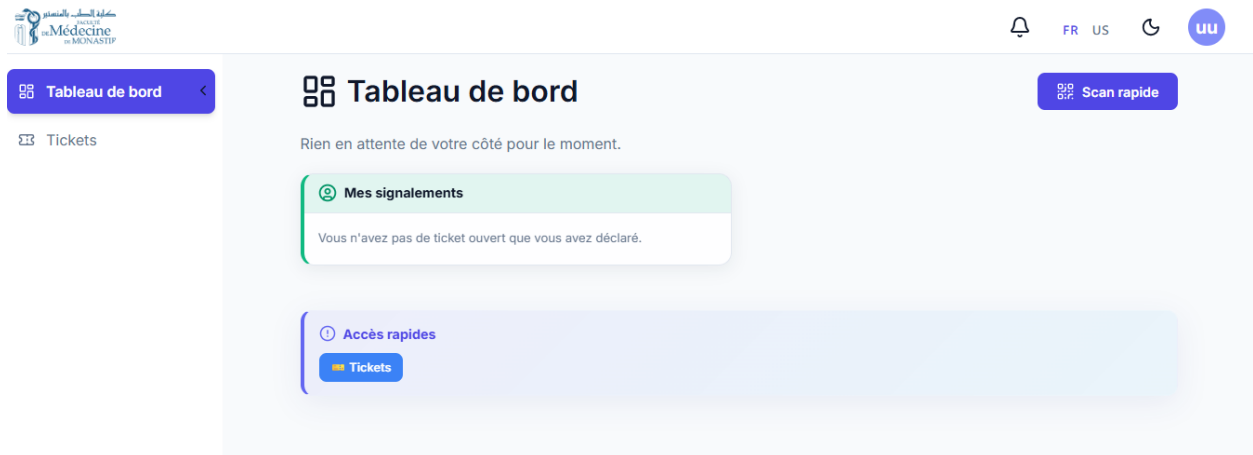


FIGURE 7.48 – Tableau de bord utilisateur — suivi des équipements affectés et des demandes d'intervention

7.3.3. Tableau de bord technicien

La Figure 7.49 illustre l'interface de travail quotidien du technicien de maintenance. Contrairement aux tableaux de bord précédents qui s'inscrivent dans une logique de supervision et de suivi, celui-ci est résolument orienté vers l'action et l'efficacité opérationnelle sur le terrain.

La vue principale liste l'ensemble des tickets affectés au technicien connecté, triés par ordre de priorité décroissante. Chaque ticket est présenté avec les informations essentielles à son traitement : désignation et localisation physique de l'équipement concerné (bloc, département, salle), description de la panne signalée par l'utilisateur, date de création et délai cible de résolution. Un code couleur intuitif — rouge pour les tickets critiques, orange pour les priorités hautes, vert pour les interventions planifiées — permet au technicien de hiérarchiser immédiatement son plan de travail de la journée.

Le technicien peut directement modifier le statut d'un ticket depuis cette interface (passage à *en cours* lors de la prise en charge, puis à *résolu* à l'issue de l'intervention), ce qui déclenche automatiquement la notification push vers l'utilisateur demandeur. Une section dédiée recense les maintenances préventives planifiées pour la semaine, permettant une organisation proactive du travail et évitant les conflits d'agenda entre interventions correctives et préventives.

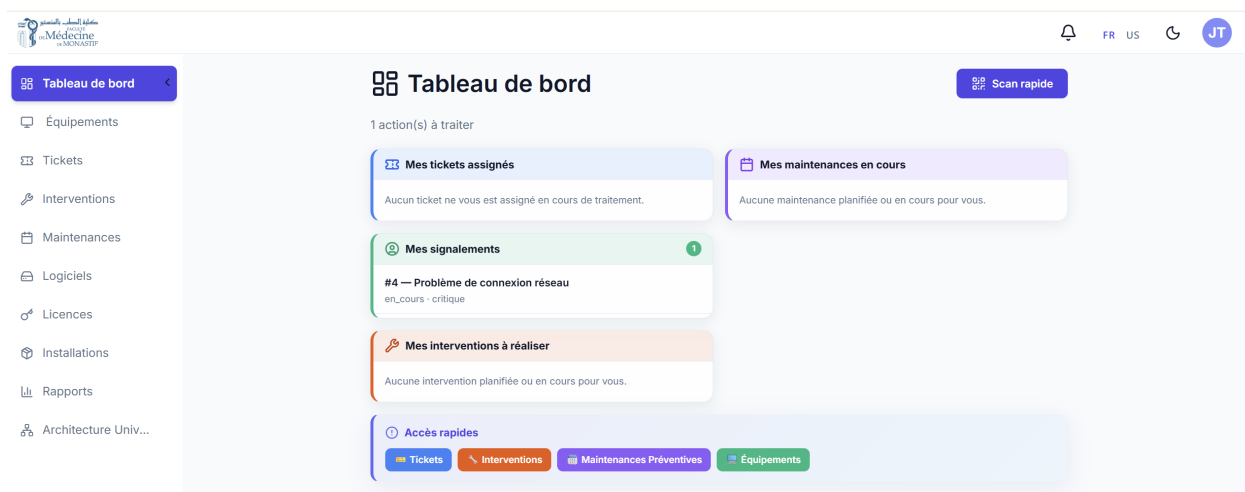


FIGURE 7.49 – Tableau de bord technicien — file d’attente des interventions classées par priorité

7.4. Modules de gestion du parc informatique

Au-delà des tableaux de bord de supervision, le Sprint 3 consolide et enrichit les modules de gestion opérationnelle introduits lors des sprints précédents. Ces modules constituent le cœur du système et permettent de gérer le cycle de vie des actifs informatiques de la FMM.

7.4.1. Gestion de l’inventaire et des équipements

Inventaire des équipements

La Figure 7.50 présente l’interface centrale de gestion de l’inventaire informatique. Cette vue est accessible exclusivement à l’administrateur du système et constitue le référentiel principal de l’ensemble du parc matériel de la FMM.

L’interface adopte une présentation tabulaire enrichie qui liste les équipements enregistrés dans la base de données, avec leurs attributs essentiels : référence constructeur, désignation, catégorie matérielle, état opérationnel, localisation physique (bloc et salle), et utilisateur affecté. Un système de filtres multicritères dynamiques permet de restreindre l’affichage selon plusieurs combinaisons de ces attributs — par exemple, afficher uniquement les ordinateurs portables du Département de Biologie Médicale dont le statut est *en panne*. Cette capacité de filtrage précise est particulièrement précieuse lors des inventaires physiques périodiques ou lors de la préparation de rapports sectoriels.

Depuis chaque ligne du tableau, l’administrateur dispose d’un menu d’actions contextuelles permettant de consulter la fiche détaillée de l’équipement, d’initier un transfert vers une autre localisation, de modifier ses attributs ou de le retirer de l’inventaire actif. La liste intègre également une pagination intelligente qui maintient la fluidité de l’interface même pour des inventaires de plusieurs centaines d’équipements.

Numéro de série	Numéro d'inventaire	Marque	Modèle	Source matérielle	Statut workflow	Date d'achat	État	Catégorie	Département	Assign
SN-0001	4458	Dell	Latitude 7420	Don	ASSIGNED	2024-01-15	reformé	Ordinateur portable	de2	Technik
eq1	14552	dd	gg	Don	ASSIGNED	2026-03-25	fonctionnel	gg	dep 1	technic
x5	41	x	x	Titre 2	ASSIGNED	2026-03-25	fonctionnel	x	dep 1	technic
SN-0005	588	Dell 2	Latitude 7450	Titre 1	ASSIGNED	2026-04-07	en_reparation	Ordinateur portable	de2	technic
SN-00056	542	xx	Latitude 7420	Don	ASSIGNED	2026-04-16	fonctionnel	x	dep 1	Technik
ggg	5174	fff	fff	Don	ASSIGNED	2026-04-08	fonctionnel	x	dep 1	Technik
cc4	425d	d	q	Titre 1	ASSIGNED	2026-04-14	en_panne	s	dep 1	technic
SN266	42544	MSI	Latitude	Titre 1	W&DEFINITION	2026-04-15	fonctionnel	Ordinateur	dep 1	zaishi

FIGURE 7.50 – Interface de gestion de l’inventaire — liste filtrée des équipements du parc

Enregistrement d’un nouvel équipement

La Figure 7.51 présente le formulaire d’enregistrement d’un nouvel actif informatique dans le système. Ce formulaire est le point d’entrée de tout équipement dans le cycle de vie géré par la plateforme, et sa conception reflète le souci de rigueur et de cohérence qui caractérise l’ensemble du système.

Le formulaire est structuré en sections thématiques pour faciliter la saisie et réduire les risques d’erreur : identification de l’équipement (référence, désignation, numéro de série, marque, modèle), caractéristiques techniques (catégorie, date d’acquisition, valeur d’achat, état initial), affectation organisationnelle (bloc, département, salle, utilisateur responsable) et configuration logicielle (systèmes d’exploitation et logiciels associés, gérés par liaison avec les entités Logiciel et Licence).

Chaque champ est soumis à des validations côté client et côté serveur : l’unicité du numéro de série est vérifiée en base de données avant l’insertion, la cohérence de la localisation (la salle doit appartenir au département, lui-même rattaché au bloc sélectionné) est contrôlée par le backend. À la validation du formulaire, le backend Express persiste l’entité `Equipement` et déclenche immédiatement la génération du QR code associé, encodant l’identifiant UUID unique de l’équipement. Ce QR code est stocké en base et rendu disponible pour impression et apposition physique sur la machine, dans une version implémentée de manière simplifiée afin de rester adaptée au contexte académique.

Nouvel Équipement

Numéro de série *

Numéro d'inventaire *

Material source *

Marque *

Modèle *

Date d'achat

État

Catégorie *

Block

Département

Salle / bureau

Assigné à

Assigné à

Notes d'assignation

Photo de l'équipement

Annuler Sauvegarder

FIGURE 7.51 – Formulaire d’enregistrement d’un nouvel équipement avec génération automatique du QR code

7.4.2. Contrôle d’accès par rôles (RBAC)

La Figure 7.52 illustre le module de gestion des permissions. Des mécanismes de sécurité de base ont été implémentés, notamment l’authentification JWT et le contrôle d’accès basé sur les rôles (RBAC). Cette dimension sécuritaire est importante dans un environnement institutionnel comme la FMM, où des données liées aux actifs matériels et aux identités des personnels sont traitées quotidiennement.

Le système définit trois rôles primaires — *Administrateur*, *Technicien* et *Utilisateur* — auxquels sont associées des matrices de permissions granulaires définissant les opérations autorisées sur chaque ressource de l’API (GET, POST, PUT, DELETE sur les endpoints `/api/equipements`, `/api/tickets`, `/api/users`, etc.). L’interface présentée dans la Figure 7.52 permet à l’administrateur de visualiser et de modifier cette matrice de permissions, d’affecter des rôles aux

utilisateurs et, si nécessaire, de créer des profils personnalisés combinant des sous-ensembles de permissions pour des cas d’usage spécifiques.

Le contrôle des accès est appliqué systématiquement par un middleware Express placé en amont de chaque route protégée. Ce middleware décode le jeton JWT présenté dans l’en-tête de la requête, extrait le rôle de l’utilisateur authentifié et vérifie sa conformité avec les permissions requises. En cas de violation, une réponse 403 *Forbidden* est renvoyée immédiatement, sans que la logique métier sous-jacente soit exécutée.

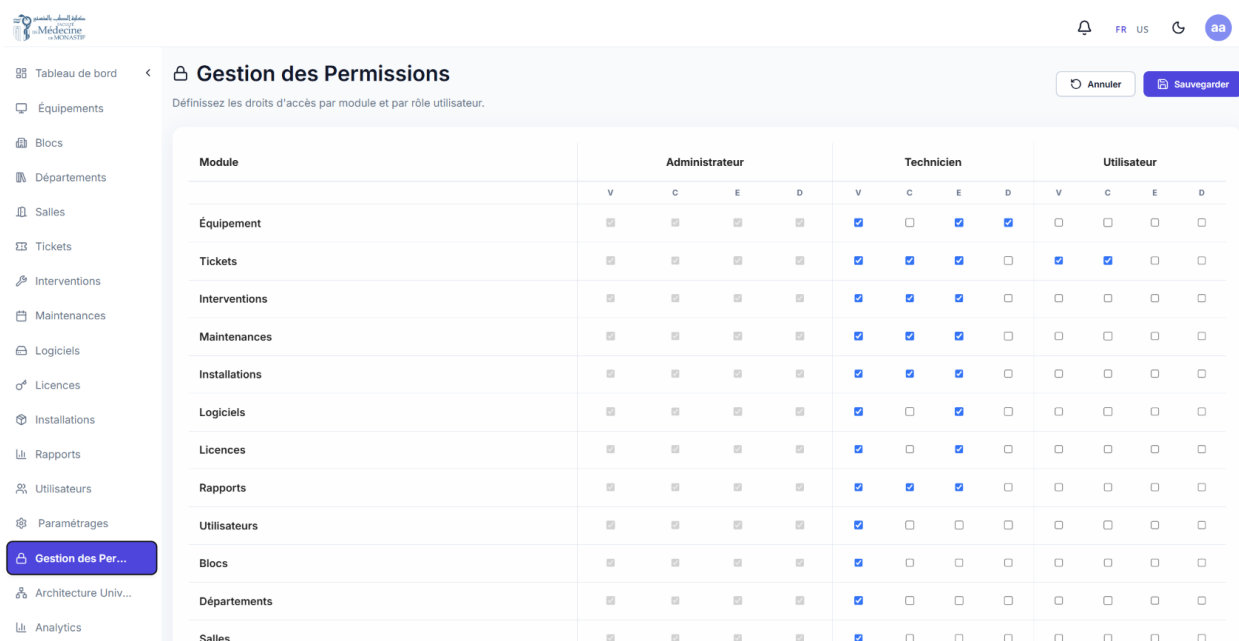


FIGURE 7.52 – Interface de gestion des permissions et rôles — modèle RBAC du système

7.4.3. Analyse et reporting

Module analytique du parc

La Figure 7.53 présente le module d’analyse décisionnelle du parc informatique. Cette interface se distingue du tableau de bord opérationnel par sa vocation prospective : là où le tableau de bord offre une photographie instantanée de l’état du système, le module analytique propose une lecture des tendances historiques et des patterns d’utilisation sur des périodes configurables.

Les visualisations disponibles couvrent plusieurs dimensions analytiques complémentaires. L’évolution mensuelle du volume de tickets sur les douze derniers mois permet d’identifier les périodes de forte sinistralité et d’anticiper les besoins en ressources humaines techniques. La distribution des pannes par catégorie d’équipement révèle les typologies matérielles les plus fragiles ou les plus sollicitées, orientant ainsi les décisions d’investissement et de renouvellement du parc. Le taux de résolution par technicien, exprimé en délai moyen de clôture (MTTR — *Mean Time To Repair*), fournit une mesure objective de la performance individuelle des équipes techniques.

Ces analyses sont produites par des requêtes SQL d’agrégation exécutées dynamiquement par

le backend, sans recours à une couche de cache ou à une table de faits dédiée. Ce choix permet de présenter des résultats proches de l'état courant de la base de données, sans décalage temporel volontaire.

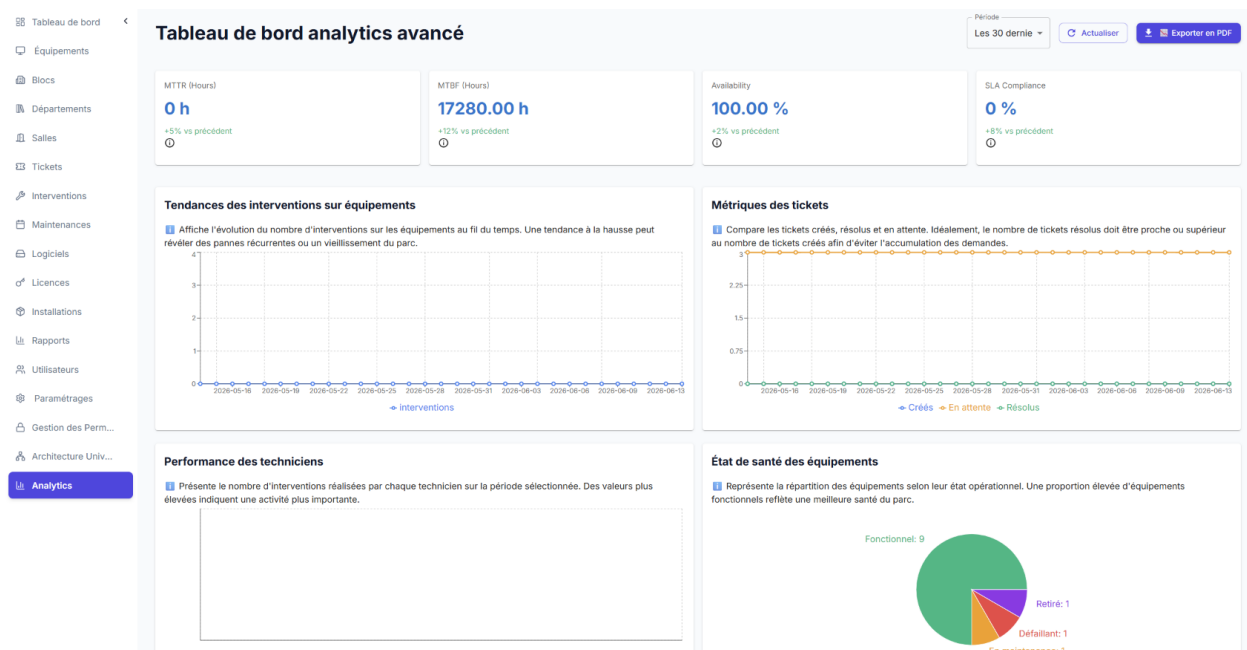


FIGURE 7.53 – Module analytique — tendances de pannes et indicateurs de performance du parc

Génération et consultation des rapports

La Figure 7.54 illustre le module de génération de rapports de workflow et d'inventaire. Ce module répond à une exigence institutionnelle forte : assurer la traçabilité formelle des opérations sur les actifs informatiques de la FMM.

Le module gère deux catégories de documents. La première couvre les **rapports de workflow** : à chaque opération de décharge (affectation d'équipement), de transfert ou de mise en réforme, le système génère automatiquement un document PDF structuré, avec en-tête FMM professionnel (logo, date, opérateur), fiche descriptive de l'équipement et résumé de l'opération. Un enregistrement correspondant est créé dans la table `Rapport`, assurant la traçabilité fonctionnelle de chaque mouvement d'actif. La deuxième catégorie couvre l'**export inventaire** : un export Excel du parc est disponible via `GET /api/rapports/export/excel`, généré avec la bibliothèque `ExcelJS`. Ce document consolide l'ensemble des actifs avec leurs attributs, leur localisation et leur état, facilitant les audits physiques ponctuels.

Titre	Workflow type	Equipment	Current assignment	Date de génération	Format	Utilisateur	Actions
Decharge - Equipment #16	DECHARGE	fff fff Inv: 5174 S/N: ggg	Technicien Jean dep 1 lab1	03/05/2026 12:57	PDF	admin admin	View PDF Export PDF
Decharge - Equipment #14	DECHARGE	Dell 2 Latitude 7450 Inv: 588 S/N: SN-0005	technicien technicien de2 lab1	03/05/2026 12:57	PDF	admin admin	View PDF Export PDF
Disposal - Equipment #1	DISPOSAL	Dell Latitude 7420 Inv: 4458 S/N: SN-0001	Technicien Jean de2 Room 24	03/05/2026 12:42	PDF	admin admin	View PDF Export PDF
Transfer - Equipment #1	TRANSFER	Dell Latitude 7420 Inv: 4458 S/N: SN-0001	Technicien Jean de2 Room 24	03/05/2026 12:29	PDF	admin admin	View PDF Export PDF
Transfer - Equipment #14	TRANSFER	Dell 2 Latitude 7450 Inv: 588 S/N: SN-0005	technicien technicien de2 lab1	03/05/2026 12:22	PDF	admin admin	View PDF Export PDF
Transfer - Equipment #16	TRANSFER	fff fff Inv: 5174 S/N: ggg	Technicien Jean dep 1 lab1	03/05/2026 12:15	PDF	admin admin	View PDF Export PDF
Decharge - Equipment #1	DECHARGE	Dell Latitude 7420 Inv: 4458 S/N: SN-0001	Technicien Jean de2 Room 24	03/05/2026 12:10	PDF	admin admin	View PDF Export PDF

FIGURE 7.54 – Module de reporting — génération de rapports périodiques PDF/Excel et consultation des archives

7.4.4. Architecture physique et traçabilité des actifs

Visualisation de l'architecture physique

La Figure 7.55 représente la vue d'architecture physique du parc informatique, qui matérialise dans l'interface la hiérarchie organisationnelle réelle de la FMM : **Bloc** → **Département** → **Salle** → **Équipement**. Cette représentation arborescente est l'une des fonctionnalités les plus distinctives du système, car elle permet de contextualiser chaque actif informatique dans son environnement physique réel, bien au-delà d'un simple inventaire tabulaire.

L'administrateur peut naviguer dans cet arbre en développant successivement les nœuds de la hiérarchie, jusqu'à accéder à la liste des équipements présents dans une salle donnée. Depuis chaque nœud, des métriques agrégées sont affichées : nombre total d'équipements dans le bloc ou le département, nombre d'équipements opérationnels vs. en panne, nombre de tickets ouverts associés à cette zone. Cette visualisation facilite grandement les audits physiques en permettant de préparer une tournée d'inspection de manière structurée, salle par salle.

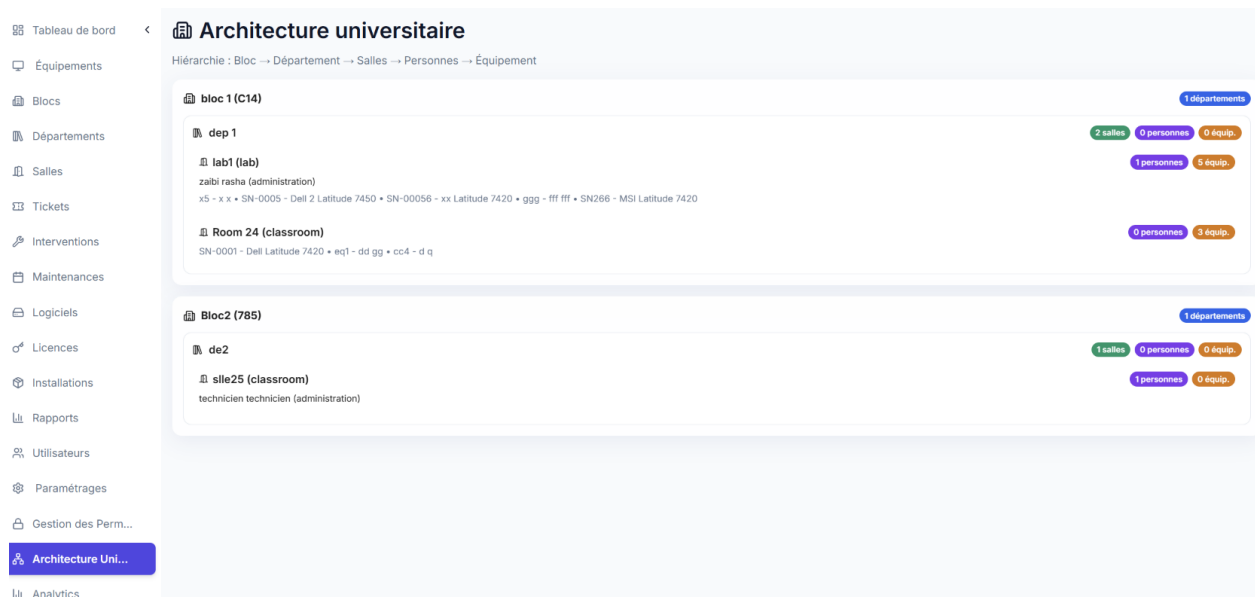


FIGURE 7.55 – Vue d’architecture physique — hiérarchie Bloc/Département/Salle/Équipement de la FMM

Génération et affichage du QR code d’un équipement

La Figure 7.56 présente l’interface d’affichage du QR code associé à un équipement spécifique. Chaque actif informatique enregistré dans le système se voit attribuer un identifiant UUID unique, encodé dans un QR code généré par le backend au moment de l’enregistrement de l’équipement. Ce code à deux dimensions constitue le pivot du mécanisme d’identification physique rapide, qui représente une fonctionnalité utile de la plateforme dans le contexte institutionnel tunisien.

L’interface présente le QR code dans une résolution optimale pour l’impression, accompagné des informations d’identification essentielles de l’équipement (référence, désignation, localisation). Un bouton d’impression directe permet de générer une étiquette formatée à apposer sur la machine physique. Une fois cette étiquette collée sur l’équipement, il devient possible d’interagir instantanément avec sa fiche système sans aucune saisie manuelle, simplement en le scannant avec un terminal mobile.

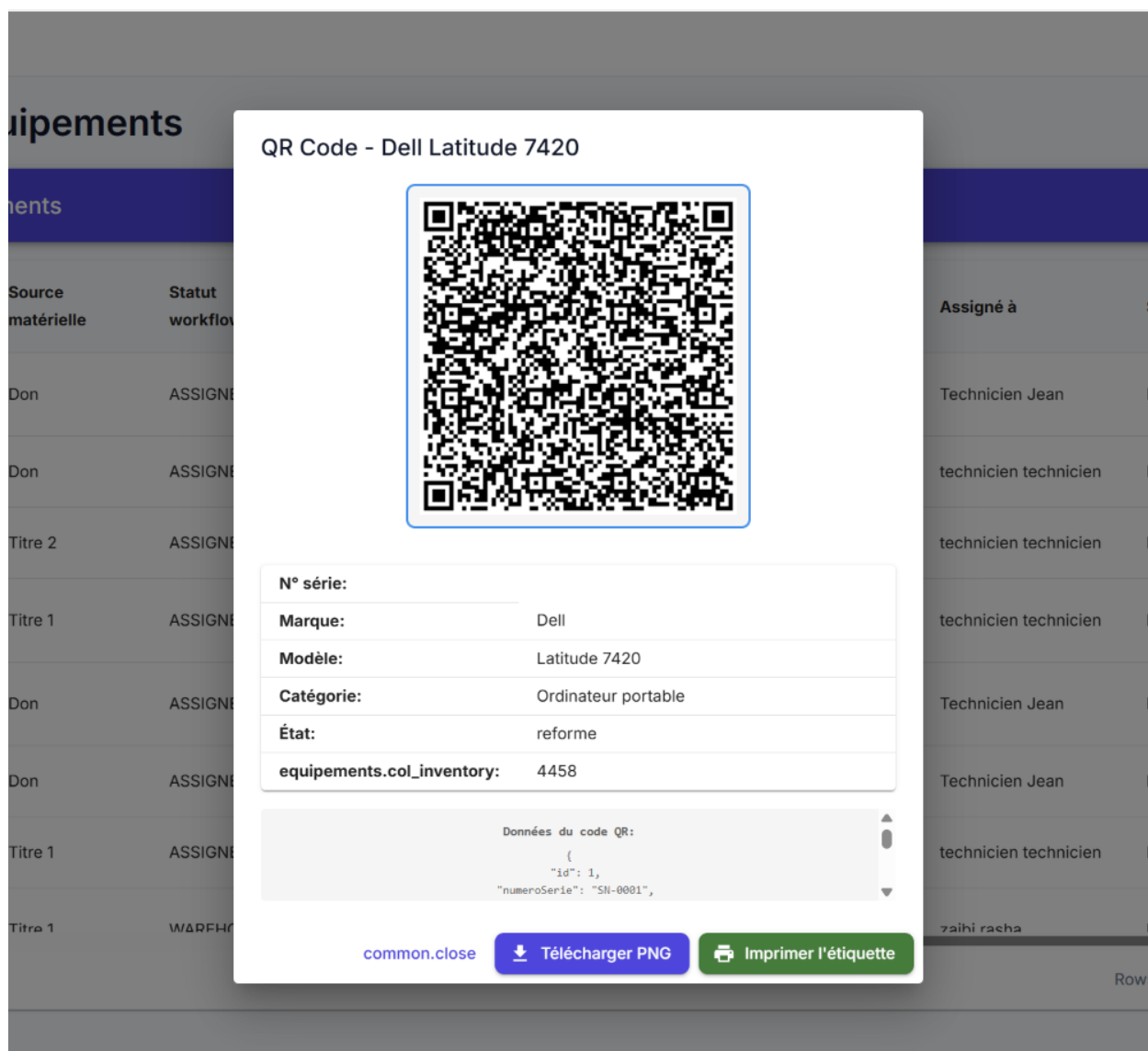


FIGURE 7.56 – Interface d’affichage et d’impression du QR code d’un équipement

Scan du QR code sur le terrain

La Figure 7.57 illustre l’interface de scan du QR code, conçue pour un usage mobile sur le terrain. Lorsqu’un technicien est appelé à intervenir sur une machine ou qu’un administrateur réalise un audit physique, il lui suffit de pointer la caméra de son terminal mobile vers l’étiquette QR code apposée sur l’équipement. L’application web, optimisée pour les navigateurs mobiles, active la caméra et décode le QR en temps réel.

Le décodage du QR code extrait l’UUID de l’équipement et déclenche une navigation automatique vers la fiche détaillée correspondante dans le système. En quelques secondes, le technicien dispose de l’ensemble des informations relatives à la machine : historique fonctionnel des interventions passées, état actuel (opérationnel, en panne, en maintenance), tickets en cours associés, spécifications techniques et localisation officielle. Cette capacité d’accès rapide à l’historique d’un équipement sur le terrain constitue un appui important pour le processus de diagnostic et peut

contribuer à réduire le temps de résolution des pannes en évitant les allers-retours vers un poste fixe.

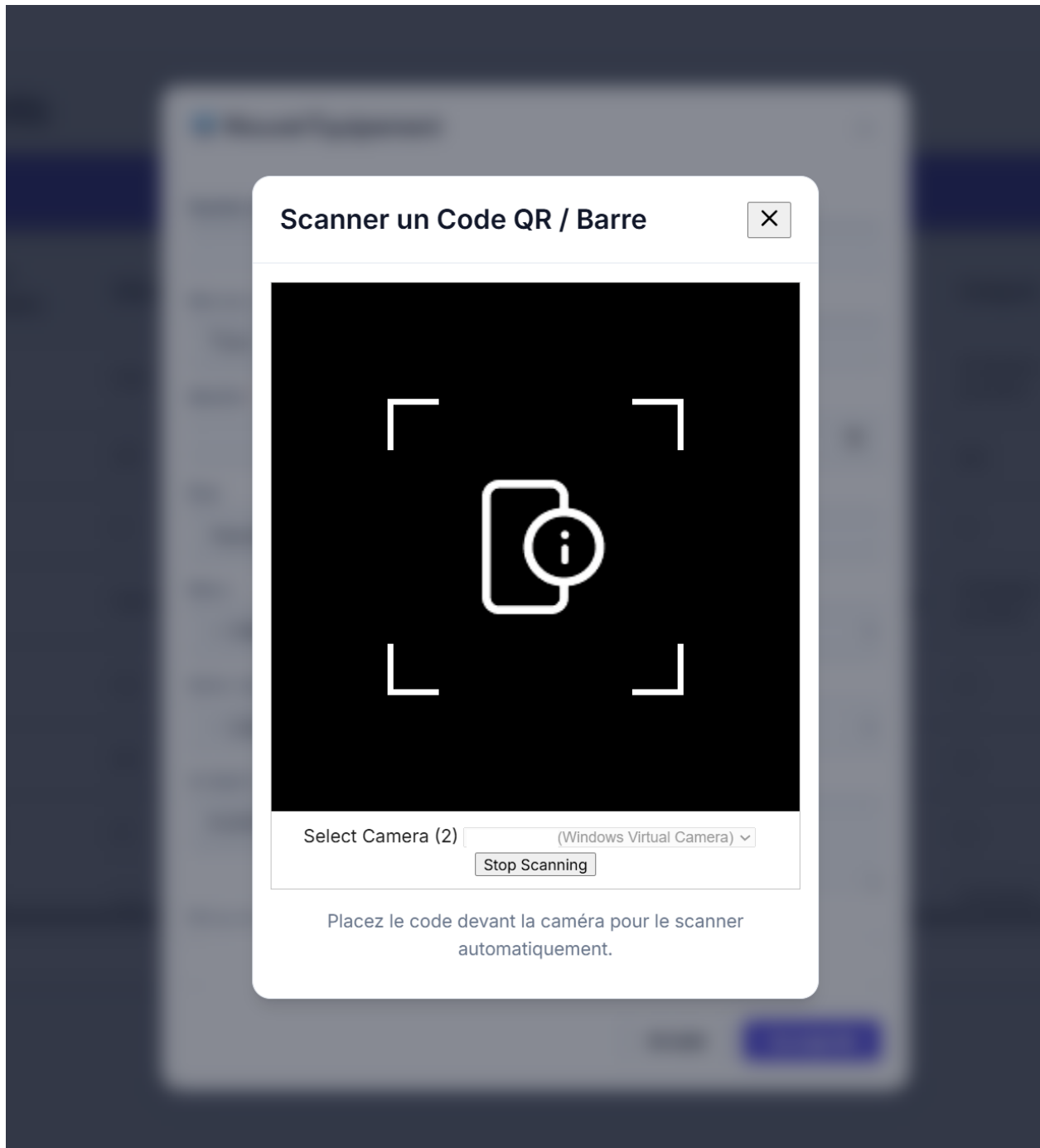


FIGURE 7.57 – Interface de scan mobile du QR code — accès instantané à la fiche d’un équipement sur le terrain

7.5. Optimisation des performances

La fiabilité fonctionnelle d’un système d’information ne saurait être dissociée de ses performances opérationnelles. Un tableau de bord dont le chargement dépasse plusieurs secondes, ou une API dont les temps de réponse se dégradent sous charge, peuvent nuire à l’adoption de la so-

lution et à sa crédibilité auprès des utilisateurs. C'est pourquoi le Sprint 3 a accordé une attention particulière à l'optimisation des performances, tant côté backend que côté frontend.

Optimisations côté Backend (API Express)

- **Indexation standard des clés étrangères** : Sequelize crée automatiquement des index sur les colonnes de jointure (`equipmentId`, `ticketId`, `TechnicienId`, etc.) lors de la synchronisation du schéma. Ces index optimisent les requêtes de jointure sur les tables à fort volume telles que `Ticket` et `Intervention`;
- **Projection des attributs** : les endpoints de liste n'exposent que les attributs strictement nécessaires à l'affichage, via la clause `attributes` de Sequelize, réduisant le volume de données transférées;
- **Agrégation SQL native** : les calculs de KPI sont réalisés directement au niveau de la base de données via des fonctions d'agrégation MySQL (`COUNT`, `AVG`, `SUM`, `GROUP BY`), évitant tout traitement itératif coûteux côté Node.js;
- **Gestion asynchrone des notifications** : l'envoi des notifications FCM, implémenté dans une version simplifiée adaptée au cadre académique, est découplé du flux de traitement principal de la requête API via des promesses non bloquantes. La réponse HTTP est renvoyée au client dès la confirmation de la persistance en base, sans attendre la confirmation de livraison de la notification.

Ces optimisations restent adaptées au volume de données manipulé dans le cadre du prototype. À plus grande échelle, l'absence d'un cache applicatif dédié pour les statistiques les plus consultées et la génération de documents PDF/Excel côté serveur devront être réévaluées afin d'éviter les pics de charge lors d'exports volumineux.

Optimisations côté Frontend (React)

- **Mémorisation des calculs** : `useMemo` est appliqué aux composants de tableau de bord pour éviter les re-calculs inutiles lors des mises à jour partielles de l'état applicatif;
- **Chargement asynchrone des données** : `@tanstack/react-query` gère le cycle de vie des requêtes API (mise en cache, rechargement automatique, états de chargement et d'erreur) sans bloquer l'interface utilisateur;
- **Rendu conditionnel par rôle** : le composant `Dashboard.jsx` adapte dynamiquement son contenu (sections affichées, données chargées) en fonction du rôle de l'utilisateur authentifié, réduisant la charge mémoire du navigateur;
- **Séparation des vues par rôle** : le routage de l'application charge uniquement les composants correspondant au rôle de l'utilisateur authentifié, réduisant à la fois la surface d'attaque côté sécurité et la charge mémoire du navigateur client.

Conclusion

Le Sprint 3 achève la construction d'un prototype fonctionnel de gestion de parc informatique cohérent, capable de couvrir les principales étapes du cycle de vie des actifs informatiques du SmartLab, de leur affectation à leur déclassement, en passant par le suivi opérationnel quotidien des incidents et des maintenances.

L'introduction de la dimension décisionnelle — tableaux de bord analytiques différenciés par profil, module de reporting exportable, indicateurs calculés en temps réel — permet à la plateforme de jouer le rôle d'outil de pilotage, et non plus seulement d'outil de saisie et de consultation. Le système de notification push, alimenté par Firebase FCM dans une version simplifiée adaptée au cadre académique, permet d'informer en temps réel les acteurs de la chaîne de maintenance des événements qui les concernent.

Les optimisations de performance réalisées au cours de cette itération indiquent la viabilité de l'architecture choisie dans le périmètre du prototype. Ces résultats ont été obtenus dans un environnement local de développement et doivent être considérés comme indicatifs. La plateforme constitue ainsi, dans le cadre de ce projet de fin d'études à une échelle locale, une contribution concrète à la modernisation de la gestion technique du patrimoine informatique de l'institution. Les performances estimées ainsi que la stabilité observée de l'architecture soutiennent la pertinence des choix techniques adoptés et ouvrent la voie à des évolutions futures du prototype dans un environnement universitaire.

Conclusion et perspectives

Le projet *Parc Informatique FMM* a permis de concevoir et de développer, dans le cadre de ce projet de fin d'études à une échelle locale, un prototype fonctionnel de plateforme web pour la gestion du parc informatique de la Faculté de Médecine de Monastir. Grâce à une approche agile (Scrum) et à l'intégration de technologies telles que les codes QR, les notifications Firebase FCM dans une version simplifiée adaptée au cadre académique, la génération de rapports PDF/Excel et le contrôle d'accès avec des mécanismes de sécurité de base, le prototype répond aux limites de la gestion traditionnelle fondée sur les fichiers Excel et les registres papier.

Les résultats obtenus indiquent que le prototype fonctionnel peut améliorer la réactivité du personnel de maintenance, faciliter le suivi des équipements et renforcer la disponibilité opérationnelle des ressources informatiques. La stack globale validée au Sprint 1, associée à une séparation claire des responsabilités, constitue un atout important pour la maintenabilité et l'évolutivité du système.

Pour aller plus loin, il sera nécessaire de réaliser des tests opérationnels en conditions réelles, d'intégrer le prototype avec des outils de gestion d'actifs (CMDB) et d'explorer l'utilisation de l'intelligence artificielle pour prédire les défaillances d'équipements et optimiser la planification de la maintenance préventive.

Limites du projet

Tout projet de développement logiciel présente des limites inhérentes à son périmètre et aux contraintes de temps. Les limites identifiées pour la version actuelle du prototype sont les suivantes :

- La validation du prototype a principalement été réalisée à travers des **tests fonctionnels manuels** via Swagger UI et l'interface web. Cette approche constitue une limite du projet. L'intégration de tests automatisés représente donc une perspective d'amélioration.
- Le prototype n'a pas encore été évalué dans un **environnement de production réel** avec un grand nombre d'utilisateurs simultanés. Les performances ont été estimées dans un environnement local de développement et doivent être considérées comme indicatives, sans tests de charge à grande échelle.
- L'absence d'un **cache applicatif dédié** pour les statistiques les plus consultées, par exemple Redis, pourrait devenir une limite en cas de montée en charge importante du module analytique.
- La génération des documents **PDF et Excel** est réalisée au niveau du serveur applicatif. Cette approche est suffisante pour le prototype, mais elle peut provoquer des pics de consommation CPU lors d'exports massifs.

- Les notifications reposent sur **Firestore FCM**, implémenté dans une version simplifiée adaptée au cadre académique, ce qui introduit une dépendance à un service externe pouvant affecter la disponibilité du module de notification.
- La gestion des jetons FCM est simplifiée avec un **seul jeton actif par utilisateur** (champ `fcmToken` dans le modèle `Utilisateur`), ce qui limite la prise en charge de sessions multi-appareils dans ce cadre académique.
- Le module d'**analyse prédictive des pannes par intelligence artificielle** est identifié comme perspective et n'est pas intégré dans ce prototype local.
- Des mécanismes de sécurité de base ont été implémentés, notamment l'authentification JWT et le contrôle d'accès basé sur les rôles (RBAC). Des audits de sécurité approfondis resteraient nécessaires avant tout déploiement institutionnel à grande échelle.

Tableau de synthèse

Le tableau ci-dessous récapitule les objectifs initiaux du projet, leur état d'avancement et les perspectives d'évolution identifiées.

TABLE 8.15 – Synthèse des objectifs atteints et perspectives d'évolution

Objectif	Statut	Perspective / Remarque
Gestion des équipements (CRUD, QR code, transfert)	✓ Atteint	Évolution vers des catégories dynamiques
Gestion des tickets d'incidents	✓ Atteint	Machine à états (Sprint 2)
Maintenance préventive et corrective	✓ Atteint	Alertes automatiques par calendrier
Notifications en temps réel (Firestore FCM)	✓ Atteint	Gestion multi-appareils à prévoir
Reporting PDF / Excel	✓ Atteint	Tableaux de bord analytiques enrichis
Contrôle d'accès RBAC / JWT	✓ Atteint	Intégration SSO / LDAP institutionnel
Tests automatisés	✗ Perspective	À mettre en place
Tests de charge	✗ Perspective	À prévoir
Analyse prédictive par IA	✗ Perspective	Module IA de détection des pannes
Audit de sécurité	✗ Perspective	À prévoir

En résumé, le prototype *Parc Informatique FMM* pose les bases d'une modernisation de la gestion technique du parc informatique du SmartLab de la Faculté de Médecine de Monastir. Son évolution dépendra d'un engagement continu des acteurs informatiques, gestionnaires et techniques afin de garantir une amélioration durable de l'efficacité opérationnelle et de la rentabilité des investissements en infrastructure.

Sur le plan académique et professionnel, ce projet a constitué une opportunité d'approfondir la maîtrise de technologies modernes du développement web, des concepts de gestion de projets agiles et des architectures de bases de données relationnelles. La collaboration étroite avec les parties prenantes de la Faculté de Médecine de Monastir a permis de confronter les contraintes théoriques de l'ingénierie logicielle aux réalités du terrain opérationnel. Cette démarche structurée contribue à la modernisation des outils de maintenance de l'institution, en jetant les bases d'une gestion plus rationnelle et réactive de son infrastructure informatique.

Bibliographie

- [1] Faculté de Médecine de Monastir. Rapport annuel 2020, 2020. Available : <http://www.fmm.rnu.tn/>.
- [2] International Organization for Standardization. Iso 21001 :2018 - educational organizations, 2018. Available : <https://www.iso.org/standard/69596.html>.
- [3] GLPI Project. Glpi : Gestionnaire libre de parc informatique, 2023. Available : <https://www.glpi-project.org/>.
- [4] Combodo. itop : It operations portal, 2023. Available : <https://combodo.com/>.
- [5] Asset Panda. Asset panda : Cloud-based it asset management, 2023. Available : <https://www.assetpanda.com/>.
- [6] Express.js Community. Express.js - fast, unopinionated web framework for node.js. <https://expressjs.com/>, 2024.
- [7] Meta Platforms, Inc. React - a javascript library for building user interfaces. <https://react.dev/>, 2024.
- [8] Oracle MySQL Community. Mysql - the world's most popular open source database. <https://www.mysql.com/>, 2024.
- [9] Google. Firebase - build apps, faster. <https://firebase.google.com/>, 2024.
- [10] MUI (Material-UI) Team. Material-ui - react components library. <https://mui.com/>, 2024.
- [11] Atlassian. What is scrum ? <https://www.atlassian.com/agile/scrum>, 2023.
- [12] Atlassian. Scrum artifacts. <https://www.atlassian.com/agile/scrum/artifacts>, 2023.
- [13] Ken Schwaber and Jeff Sutherland. *Scrum : The art of doing twice the work in half the time*. Crown Business, 2004.
- [14] Atlassian. Scrum roles. <https://www.atlassian.com/agile/scrum/roles>, 2023.
- [15] ClickUp. Clickup docs : Product management and agile tools. <https://docs.clickup.com/en/articles/2095287-getting-started-with-clickup>, 2025.
- [16] Slack Technologies. Slack documentation. <https://slack.com/help/categories/200111606>, 2025.
- [17] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.
- [18] Sequelize Contributors. Sequelize - promise-based node.js orm. <https://sequelize.org/>, 2024.
- [19] Evan You and Vite Contributors. Vite - next generation frontend tooling. <https://vitejs.dev/>, 2024.